T.C.
FIRAT ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ

MODERN FİDYE YAZILIMLARINA KARŞI eBPF TABANLI PROAKTİF SAVUNMA VE HİBRİT ANALİZ SİSTEMİ

Osman ÇEVİK

Tez Danışmanı:
Dr. Hasan Yetiş

BİTİRME TEZİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

ELAZIĞ
2026

T.C.
FIRAT ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ

MODERN FİDYE YAZILIMLARINA KARŞI eBPF TABANLI PROAKTİF SAVUNMA VE HİBRİT ANALİZ SİSTEMİ

Osman Çevik

BİTİRME TEZİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Bu bitirme tezi ..../..../2026 tarihinde, aşağıda belirtilen jüri tarafından oybirliği/oyçokluğu ile başarılı/başarısız olarak değerlendirilmiştir.

(İmza)	(İmza)	(İmza)
Danışman	Üye	Üye

ÖZGÜNLÜK BİLDİRİMİ

Bu çalışmada, başka kaynaklardan yapılan tüm alıntıların, ilgili kaynaklar referans gösterilerek açıkça belirtildiğini, alıntılar dışındaki bölümlerin, özellikle projenin ana konusunu oluşturan teorik çalışmaların ve yazılım/donanımın benim tarafımdan yapıldığını bildiririm.

Fırat Üniversitesi
Bilgisayar Mühendisliği
23119 Elazığ

Tarih
Osman Çevik
İmza

OsmanÇevik_BitirmeTezi_202526Guz.pdf

ORIGINALITY REPORT

3%	3%	1%	2%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Ataturk Universitesi Student Paper	1%
2	Submitted to Fırat Üniversitesi Student Paper	<1%
3	assets-eu.researchsquare.com Internet Source	<1%
4	nakryiko.com Internet Source	<1%
5	Submitted to Universiteit van Amsterdam Student Paper	<1%
6	www.armosec.io Internet Source	<1%
7	acikbilim.yok.gov.tr Internet Source	<1%
8	Submitted to Australian National University Student Paper	<1%
9	git.2f30.org Internet Source	<1%

TEŞEKKÜR

Bu tez çalışmasının konusunun belirlenmesinden, kodlama ve test aşamalarının nihayete ermesine kadar geçen süreçte; bilgi birikimi, tecrübesi ve vizyonu ile bana yol gösteren, karşılaştığım teknik zorluklarda akademik rehberliğini esirgemeyen değerli danışman hocam **Dr. Hasan Yetiş**'e en içten teşekkürlerimi sunarım.

Akademik desteğin yanı sıra, bu çalışmanın teknik altyapısını mümkün kılan öncü isimleri de anmak isterim. Linux çekirdeğinin en güçlü alt sistemlerinden biri olan eBPF'in mimarları **Alexei Starovoitov** ve **Daniel Borkmann**'a, geliştirdikleri teknoloji ile modern işletim sistemi güvenliğine getirdikleri devrimsel bakış açısı için minnettarım.

Ayrıca, eBPF teknolojisinin karmaşık yapısını kaleme aldığı eserler ve geliştirdiği araçlarla anlaşılır kılan, performans analizi ve gözlemlenebilirlik alanındaki çalışmalarıyla projeme ilham kaynağı olan **Brendan Gregg**'e ve modern BPF uygulamalarının taşınabilirliğini sağlayan CO-RE (Compile Once - Run Everywhere) mimarisine ve libbpf kütüphanesine olan kritik katkılarından dolayı **Andrii Nakryiko**'ya teşekkürü bir borç bilirim.

Son olarak, bilginin paylaşıldıkça çoğaldığına inanan ve ürettikleri kodları tüm dünya ile paylaşan özgür yazılım topluluğuna teşekkür ederim.

Osman Çevik
Elazığ Fırat Üniversitesi, 2026

İÇİNDEKİLER

İÇ KAPAK I

ÖZGÜNLÜK BİLDİRİMİ II

BENZERLİK BİLDİRİMİ III

TEŞEKKÜR IV

İÇİNDEKİLER V

ŞEKİLLER LİSTESİ IX

TABLOLAR LİSTESİ X

KOD LİSTESİ XI

SİMGELER LİSTESİ XII

KISALTMALAR LİSTESİ XIII

ÖZET XIV

ABSTRACT XV

1. GİRİŞ 1

1.1. Genel Bakış ve Problem Tanımı 1

1.1.1. Fidye Yazılımlarının Evrimi ve Modern Tehdit Ortamı 1

1.1.2. Linux Sunucu Güvenliği ve Mevcut Araçların Kısıtları 2

1.2. Motivasyon 3

1.3. Projenin Amacı ve Hedefleri 4

1.4. Projenin Kapsamı ve Sınırları 5

1.5. Tezin Organizasyonu 6

2. LİTERATÜR ANALİZİ VE TEORİK ALTYAPI 7

2.1. Fidye Yazılımlarının Teknik Analizi ve Evrimi 7

2.1.1. Saldırı Yaşam Döngüsü 7

2.1.2. Şifreleme Stratejileri ve Entropi Analizi 8

2.1.3. Linux Odaklı Tehditler 8

2.2. Tehdit Tespit Metodolojileri 9

2.2.1. Statik Analiz ve İmza Tabanlı Tespit Yöntemleri 9

2.2.2. Dinamik Analiz ve Davranışsal İzleme 10
2.3. eBPF (Extended Berkeley Packet Filter) Teknolojisi 10
2.3.1. Tarihsel Gelişim: Klasik BPF'den Modern eBPF'e 11
2.3.2. Mimari Bileşenler ve Çalışma Prensibi 11
2.3.3. Güvenlik İzlemede Performans ve Geleneksel Yöntemlerle Karşılaştırma . . . 12
2.4. İlgili Çalışmalar ve RansomBPF'in Özgünlüğü 13
2.4.1. Literatürdeki Mevcut Araçlar 13
2.4.2. RansomBPF'in Konumu ve Özgün Değeri 13

3. MİMARİ TASARIM VE SİSTEM BİLEŞENLERİ 15
3.1. Sistemin Genel Çalışma Prensibi 15
3.1.1. Yüksek Seviyeli Mimari 15
3.1.2. Olay Akış Döngüsü ve Müdahale Mekanizması 16
3.2. Çekirdek Alanı (Kernel-Space) Programlama 17
3.2.1. İzleme Mekanizması: Tracepoint ve Kprobe Karşılaştırması 17
3.2.2. İzlenen Kritik Sistem Çağrıları ve Veri Toplama Stratejisi 17
3.2.3. Modern Çekirdek Uyumluluğu: CO-RE ve BTF 18
3.3. Veri İletişim Kanalları ve Ring Buffer 19
3.3.1. Ring Buffer Mimarisi ve Tercih Sebepleri 19
3.3.2. Veri Protokolü ve Olay Yapısı (Struct Event) 19
3.4. Modüler Yazılım Yapısı ve Bileşenler 20
3.4.1. Ana Kontrolcü (Main Module) 20
3.4.2. Analiz Motoru (Detector) 21
3.4.3. Yapılandırma Modülü (Config) 22
3.4.4. Loglama ve Adli Analiz (Logger) 22
3.5. Durum Yönetimi ve Bellek Politikası 22
3.5.1. Süreç Takibi ve O(1) Erişim Mimarisi 22
3.5.2. Hafıza Temizliği ve Kaynak Yönetimi (Graceful Cleanup) 23
3.6. Dağıtım ve Servis Entegrasyonu 24
3.6.1. Systemd Servis Mimarisi ve Süreklilik 24
3.6.2. Komut Satırı Arayüzü (CLI) 25

4. DAVRANIŞSAL TESPİT MOTORU VE ANALİZ METODOLOJİSİ 26
4.1. Tespit Felsefesi ve Analiz Yaklaşımı 26
4.1.1. Geleneksel Yaklaşımın Kısıtları ve Statik Eşik Sorunu 26
4.1.2. Kümülatif Risk Profili ve Davranışsal Skorlama 26
4.2. Sezgisel Tespit Kuralları 27
4.2.1. H1 - Yazma Yoğunluğu (Write Burst) 27
4.2.2. H2 - Dosya Sistemi Manipülasyonları 27
4.2.3. H3 - Stratejik Tuzaklar (Honeypot) 29
4.3. Puan Tabanlı Risk Analizi Motoru 29

4.3.1. Olay Ağırlıklandırma Matrisi 29
4.3.2. Bağlam Çarpanları (Context Multipliers) 30
4.3.3. Uzantı ve Metadata Analizi 31
4.4. Dinamik Risk Sönümleme (Risk Decay) 32
4.4.1. Zaman Duyarlılığı ve Meşruiyet Prensibi 32
4.4.2. Matematiksel Model ve Sinsi Saldırı Analizi 32
4.5. Beyaz Liste ve Gürültü Yönetimi 33
4.5.1. $O(1)$ Karmaşıklığında Filtreleme Algoritması 34
4.5.2. Self-Monitoring ve Geri Besleme Döngüsü Koruması 35

5. AKTİF MÜDAHALE VE TEHDİT ÖNLEME SİSTEMİ (IPS) 36
5.1. Müdahale Felsefesi ve Gereksinimler 36
5.1.1. Zamanın Kritik Önemi ve IDS Yetersizliği 36
5.1.2. Proaktif Savunma ve Milisaniye Seviyesinde Müdahale 36
5.2. Süreç Sonlandırma Mekanizması 37
5.2.1. Karar Verme ve Tetikleme Mantığı 37
5.2.2. SIGKILL Sinyali ve Zorla Durdurma 37
5.2.3. Tepki Süresi ve Gecikme (Latency) 38
5.3. Sistem Çağrısı Kapsamının Genişletilmesi 38
5.3.1. Sistem Çağrısı Çeşitliliği ve Kaçınma Riskleri 39
5.3.2. Tam Görünürlük İçin Çoklu Kanca (Multi-Hook) Mimarisi 39
5.4. Kritik Süreç Koruması ve Güvenlik 40
5.4.1. Sistem Kararlılığı ve Dokunulmaz Süreçler 40
5.4.2. Gelişmiş Beyaz Liste Entegrasyonu 41
5.5. Müdahale Loglama ve Bildirim 42
5.5.1. Alerts.json ve Adli Veri Paketi 42
5.5.2. Geri Bildirim Döngüsü ve Örnek Çıktı 42

6. TEST METODOLOJİSİ VE DENEYSEL SONUÇLAR 44
6.1. Test Ortamı ve Simülasyon Altyapısı 44
6.1.1. Donanım ve Yazılım Özellikleri 44
6.1.2. Tehdit Emülasyon Araçları 44
6.1.3. Test Senaryolarının Tasarımı: ”Live Fire” Metodolojisi 45
6.2. Birim Testleri (Unit Tests) 46
6.2.1. Test Kapsamı ve Yöntemi 46
6.2.2. Test Senaryoları ve Sonuçları 46
6.3. Davranışsal Kural Doğrulama Testleri 46
6.3.1. H1 Testi: Saldırı Hızı ve Yazma Yoğunluğu Analizi 48
6.3.2. H2 Testi: Uzantı Değişimi ve Semantik Analiz 48
6.4. Aktif Müdahale ve IPS Başarı Analizi 49
6.4.1. Tepki Süresi Analizi (Mean Time to Respond) 50

6.4.2. Veri Kurtarma Oranı ve Karşılaştırmalı Test 50
6.4.3. Görünürlük Kapsamı ve Kör Nokta Analizi 51
6.5. Yanlış Pozitif (False Positive) Değerlendirmesi 51
6.5.1. Meşru Yazılım Davranış Testleri 51
6.5.2. Beyaz Liste Performansı ve Kaynak Kullanımı 52
6.6. Bulguların Değerlendirilmesi 52
6.6.1. Hedefler ve Çıktıların Tutarlılığı 53
6.6.2. Veri Kurtarma Başarısı ve Kritik Değerlendirme 53

7. SONUÇ VE GELECEK ÇALIŞMALAR 54
7.1. Genel Değerlendirme 54
7.1.1. Teknolojik Seçim ve Mimari Evrim Süreci 54
7.1.2. Hedef ve Sonuç Uyumu 55
7.2. Elde Edilen Kazanımlar 55
7.2.1. Tespit ve Müdahale Performansı 55
7.2.2. Akıllı Analiz ve Esneklik Yeteneği 56
7.3. Projenin Özgün Katkıları 56
7.3.1. Teknik İnovasyon ve Hibrit Mimari 57
7.3.2. Operasyonel Olgunluk ve Adli Analiz Altyapısı 57
7.4. Çalışmanın Kısıtlamaları 57
7.4.1. Platform ve Çekirdek Bağımlılığı 58
7.4.2. Analiz Kapsamı ve Vektör Sınırları 58
7.5. Gelecek Çalışmalar 58
7.5.1. Makine Öğrenmesi Tabanlı Dinamik Profilleme 59
7.5.2. Ağ Katmanı Entegrasyonu (eBPF XDP) 59
7.5.3. Grafiksel Yönetim Paneli (GUI) 59

KAYNAKLAR 60

EKLER 64
EK-A: KAYNAK KODLAR 64

ÖZGEÇMİŞ 88

ŞEKİLLER LİSTESİ

1.1 Auditd, Falco ve RansomBPF stres testi sonuçları. 3

4.1 H2 İşlem Bazlı Risk Puanları ve Kill Switch Eşiği 28
4.2 Nihai Risk Puanı: İşlem Türü ve Risk Bağlamı Etkileşimi 31
4.3 Dinamik Risk Sönümleme Analiz Sonuçları 33
4.4 Filtreleme Performansı Analizi 34

6.1 Semantik Analizin Etkisi: Uzantı Değişim Senaryoları 49

TABLOLAR LİSTESİ

2.1 Statik ve Dinamik Analiz Yöntemlerinin Karşılaştırılması 11

4.1 Olay Tiplerine Göre Atanan Taban Risk Puanları 30

6.1 Sanal Test Ortamı Donanım ve Yazılım Özellikleri 44
6.2 RansomBPF Analiz Motoru Birim Test Sonuçları 47
6.3 Saldırı Hızına Bağlı Tespit Performansı 48
6.4 IPS Kapalı ve Açık Durumda Veri Kurtarma Performansı Karşılaştırması 50
6.5 Meşru Sistem Araçları ile Yanlış Pozitif Test Sonuçları 52

KOD LİSTESİ

3.1 Çekirdek Alanında Yazma İşleminin Yakalanması 18
3.2 Çekirdek ve Kullanıcı Alanı Arasında Paylaşılan Olay Veri Yapısı 20
3.3 Ana Döngü ve Ring Buffer Dinleme 21
3.4 Olay Analizi ve Risk Puanlama Mantığı 21
3.5 Süreç Durum Yapısı 23
3.6 Sonlanan Sürecin Bellekten Temizlenmesi 24
3.7 Systemd Servis Birim Dosyası 25
3.8 Komut Satırı Argümanlarının Ayrıştırılması 25
4.1 İşlem Türüne Göre Risk Puanı Ataması 28
4.2 Tuzak Dosya Erişimi Kontrolü 29
4.3 Dosya Yoluna Göre Risk Çarpanı Hesaplaması 31
4.4 Risk Puanlarının Periyodik Sönümlenmesi 33
4.5 Sabit Sürede Çalışan Beyaz Liste Kontrolü 34
4.6 Çekirdek Seviyesinde Kendi Kendini İzlemeyi Önleme 35
5.1 Risk Eşiği Kontrolü ve Müdahale Tetikleyicisi 37
5.2 Sürecin SIGKILL ile Sonlandırılması 38
5.3 Çoklu Sistem Çağrısı Kancaları 40
5.4 Kritik Sistem Süreçleri İçin Emniyet Kontrolü 41
5.5 Başarılı Müdahale Sonrası Üretilen JSON Log Kaydı 43
6.1 Saldırı Simülatörünün Şifreleme Döngüsü 45
A.1 Çekirdek alanı veri toplama mantığı 65
A.2 Kullanıcı alanı ana yönetim döngüsü 69
A.3 Risk puanlama ve aktif engelleme (IPS) mantığı 73
A.4 Süreç bazlı istatistik takibi ve uthash entegrasyonu 79
A.5 Process_stats yapısı tanımları 81
A.6 Hibrit loglama ve JSON formatında alert üretimi 83

SİMGELER LİSTESİ

λ	: Sönümleme katsayısı
$O(1)$	: Sabit zaman karmaşıklığı
$O(n)$	: Doğrusal zaman karmaşıklığı
$Score_{base}$	: Olay tipine göre atanan taban risk puanı
$Score_{final}$	: Bağlam çarpanı ile hesaplanmış nihai risk puanı
$Multiplier_{path}$	: Dosya yoluna göre belirlenen risk çarpanı
S_t	: Sürecin t anındaki mevcut risk puanı
S_{t+1}	: Sürecin bir sonraki zaman dilimindeki sönümlenmiş risk puanı
t_0	: Şüpheli aktivitenin başladığı zaman damgası
t_{kill}	: Zararlı sürecin sonlandırıldığı zaman damgası

KISALTMALAR LİSTESİ

AES	: Advanced Encryption Standard (Gelişmiş Şifreleme Standardı)
API	: Application Programming Interface (Uygulama Programlama Arayüzü)
BPF	: Berkeley Packet Filter (Berkeley Paket Filtresi)
BTF	: BPF Type Format (BPF Tür Formatı)
CO-RE	: Compile Once – Run Everywhere (Bir Kere Derle – Her Yerde Çalıştır)
CPU	: Central Processing Unit (Merkezi İşlem Birimi)
DAG	: Directed Acyclic Graph (Yönlendirilmiş Döngüsüz Grafik)
eBPF	: Extended Berkeley Packet Filter (Genişletilmiş Berkeley Paket Filtresi)
EDR	: Endpoint Detection and Response (Uç Nokta Tespit ve Yanıt Sistemi)
I/O	: Input/Output (Giriş/Çıkış)
IDS	: Intrusion Detection System (Saldırı Tespit Sistemi)
IPS	: Intrusion Prevention System (Saldırı Önleme Sistemi)
JIT	: Just-In-Time Compiler (Tam Zamanında Derleyici)
LKM	: Loadable Kernel Module (Yüklenebilir Çekirdek Modülü)
PID	: Process ID (Süreç Kimlik Numarası)
PPID	: Parent Process ID (Üst Süreç Kimlik Numarası)
RAM	: Random Access Memory (Rastgele Erişimli Bellek)
RDP	: Remote Desktop Protocol (Uzak Masaüstü Protokolü)
SSH	: Secure Shell (Güvenli Kabuk)
UID	: User ID (Kullanıcı Kimlik Numarası)
VM	: Virtual Machine (Sanal Makine)
AI	: Artificial Intelligence (Yapay Zeka)
C2	: Command and Control (Komuta Kontrol)
cBPF	: Classic Berkeley Packet Filter (Klasik Berkeley Paket Filtresi)
CLI	: Command Line Interface (Komut Satırı Arayüzü)
CNCF	: Cloud Native Computing Foundation
GUI	: Graphical User Interface (Grafiksel Kullanıcı Arayüzü)
IOPS	: Input/Output Operations Per Second (Saniye Başına Girdi/Çıktı İşlemi)
JSON	: JavaScript Object Notation
LSM	: Linux Security Modules (Linux Güvenlik Modülleri)
MCDM	: Multi-Criteria Decision Making (Çok Kriterli Karar Verme)
PoC	: Proof of Concept (Kavram Kanıtlama)
RaaS	: Ransomware-as-a-Service (Hizmet Olarak Fidye Yazılımı)

ÖZET

MODERN FİDYE YAZILIMLARINA KARŞI eBPF TABANLI PROAKTİF SAVUNMA VE HİBRİT ANALİZ SİSTEMİ

Dijital dönüşüm süreçlerinin hızlanmasıyla birlikte, bulut altyapılarının ve veri merkezlerinin omurgasını oluşturan Linux tabanlı sistemler, fidye yazılımlarının (ransomware) öncelikli hedefi haline gelmiştir. Geleneksel imza tabanlı güvenlik yaklaşımları, polimorfik teknikler kullanan ve sürekli evrilen modern tehditlere karşı yetersiz kalırken; mevcut davranışsal analiz araçları ise yüksek sistem kaynağı tüketimi ve hatalı pozitif oranları nedeniyle operasyonel darboğazlar yaratmaktadır.

Bu tez çalışmasında, söz konusu kısıtları aşmak amacıyla ”RansomBPF” adında, eBPF (Extended Berkeley Packet Filter) teknolojisini temel alan, çekirdek seviyesinde çalışan, yüksek performanslı ve proaktif bir Saldırı Önleme Sistemi (IPS) geliştirilmiştir. Önerilen hibrit mimari, dosya sistemi üzerindeki kritik sistem çağrılarını (write, rename, unlink vb.) Ring Buffer mekanizması ile kayıpsız olarak izlemekte ve kullanıcı alanındaki ”Kümülatif Risk Puanlama” motoru ile analiz etmektedir. Geliştirilen analiz motoru; yazma yoğunluğu, dosya uzantısı manipülasyonları ve tuzak dosya (honeypot) erişimlerini ”Dinamik Sönümleme” (Decay) algoritması ile birleştirerek, saldırıları anlamsal (semantik) bağlamda değerlendirmektedir.

Gerçekleştirilen ”Canlı Ateş” (Live Fire) testlerinde, RansomBPF sisteminin saldırıları 100 milisaniyenin altında bir tepki süresiyle tespit ettiği ve zararlı süreçleri SIGKILL sinyali ile sonlandırarak %94 oranında veri kurtarma başarısı sağladığı görülmüştür. Ayrıca, sistemin meşru yönetim araçlarını (git, gcc vb.) beyaz liste mekanizması ile ayırt ederek hatalı pozitif üretmediği ve işlemci üzerindeki yükünün ihmal edilebilir seviyede olduğu doğrulanmıştır. Bu çalışma, eBPF teknolojisinin Linux sunucu güvenliğinde ”Sıfırıncı Gün” saldırılarına karşı hafif sıklet ve etkili bir savunma katmanı sağladığını kanıtlamaktadır.

Anahtar Kelimeler: fidye yazılımı tespiti, eBPF, Linux çekirdek güvenliği, davranışsal analiz, siber güvenlik.

ABSTRACT

eBPF-BASED PROACTIVE DEFENSE AND HYBRID ANALYSIS SYSTEM AGAINST MODERN RANSOMWARE

As digital transformation accelerates, Linux-based systems, which underpin cloud infrastructures and data centers, have emerged as main targets for ransomware. Traditional signature-based security methods are inadequate against contemporary threats that employ polymorphic strategies and continuously change, while current behavioral analysis technologies generate operational bottlenecks due to excessive system resource utilization and elevated false positive rates.

This thesis presents the development of a high-performance, proactive Intrusion Prevention System (IPS) called ”RansomBPF,” which utilizes eBPF (Extended Berkeley Packet Filter) technology and functions at the kernel level to address these restrictions. The suggested hybrid architecture monitors essential system calls on the file system (write, rename, unlink, etc.) without loss through the Ring Buffer mechanism and evaluates them using a ”Cumulative Risk Scoring” engine in user space. The advanced analysis engine assesses attacks within a semantic framework by integrating write density, file extension alterations, and honeypot interactions through a ”Dynamic Decay” approach.

In ”Live Fire” tests, the RansomBPF system has demonstrated an attack detection response time of under 100 milliseconds and a 94% success rate in data recovery by terminating hostile processes via the SIGKILL signal. Moreover, it has been confirmed that the system does not generate false positives by differentiating legitimate management tools (such as git, gcc, etc.) via a whitelist method, and its CPU overhead is minimal. This study demonstrates that eBPF technology offers a lightweight and efficient defense mechanism against ”Zero-Day” threats in Linux server security.

Keywords: ransomware detection, eBPF, Linux kernel security, behavioral analysis, cybersecurity.

1. GİRİŞ

Dijital dönüşümün hız kazandığı günümüzde, siber tehditler de benzer bir hızla evrilerek kritik altyapıları ve veri bütünlüğünü doğrudan hedef almaktadır. Bu tehditlerin başında gelen fidye yazılımları, sadece bireysel kullanıcıları değil, modern veri merkezlerinin ve bulut bilişim dünyasının bel kemiği olan Linux tabanlı sistemleri de her geçen gün daha derinden etkilemektedir [1]. Geleneksel güvenlik yaklaşımları, zararlı yazılımları bilinen dosya imzaları üzerinden tanımlamaya çalışırken; saldırganların geliştirdiği *polimorfik algoritmalar* ve *sıfırıncı gün saldırıları* bu statik savunma hatlarını kolaylıkla aşabilmektedir [2].

Siber savunma stratejilerinde yaşanan bu yetersizlik, güvenliğin *reaktif* bir modelden *proaktif* ve *davranışsal* bir analize kaymasını zorunlu kılmıştır. Bir zararlı yazılımın kimliğinden ziyade sistem üzerinde sergilediği şüpheli eylemlere (I/O yoğunluğu, kritik dosya manipülasyonları vb.) odaklanmak, modern tehditlere karşı en etkili savunma yöntemi olarak kabul edilmektedir [3]. Bu noktada, Linux çekirdeğinin içine güvenli ve yüksek performanslı bir şekilde programlanabilir mantık yerleştirilmesine olanak tanıyan `eBPF` teknolojisi, sistem güvenliğinde yeni bir dönemi temsil etmektedir [4].

Bu tez çalışması kapsamında geliştirilen **RansomBPF** sistemi, eBPF teknolojisini kullanarak `Linux` çekirdek seviyesindeki sistem çağrılarını gerçek zamanlı olarak izlemekte ve fidye yazılımlarının karakteristik davranışlarını gelişmiş bir risk puanlama motoru aracılığıyla analiz etmektedir. Çalışma, sadece bir tespit mekanizması sunmakla kalmayıp, tespit edilen tehditlere milisaniyeler düzeyinde aktif müdahale ederek veri kaybını minimize etmeyi hedefleyen bütüncül bir Saldırı Önleme Sistemi (IPS) mimarisi ortaya koymaktadır.

1.1 Genel Bakış ve Problem Tanımı

Siber güvenlik dünyasında tehdit aktörleri ile savunma sistemleri arasındaki mücadele, son on yılda nitelik değiştirmiştir. Bu değişimin merkezinde yer alan fidye yazılımları, artık sadece bireysel verileri hedef alan basit betikler değil, modern dijital ekonomiyi ve kurumsal altyapıları felç edebilecek kapasiteye sahip sofistike siber silahlar haline gelmiştir.

1.1.1 Fidye Yazılımlarının Evrimi ve Modern Tehdit Ortamı

Fidye yazılımları, 1989 yılındaki ilk örneklerinden günümüze kadar büyük bir teknolojik dönüşüm geçirmiştir. Günümüzde bu yazılımlar, **"Hizmet Olarak Fidye Yazılımı"** (`Ransomware-as-a-Service - RaaS`) modeliyle profesyonelleşmiş bir endüstriye dönüşmüştür [5]. Modern varyantlar, sadece dosyaları şifrelemekle kalmayıp; sistem yedeklerini imha etmek, gölge kopyaları silmek ve verileri sızdırarak çifte şantaj uygulamak gibi karmaşık bir saldırı zinciri izlemektedir.

Bu evrimsel sürecin en tehlikeli aşaması, saldırganların *"Living off the Land"* tekniklerini kul-

lanarak sistemin kendi meşru araçlarını (örneğin Linux üzerinde `dd`, `mv`, `rm`) kötüye kullanmasıdır [6]. Bu durum, zararlı yazılımın sistem üzerindeki ayak izini küçültmekte ve tespiti zorlaştırmaktadır. Ayrıca, saniyeler içinde binlerce dosyayı manipüle edebilen yüksek hızlı şifreleme motorları, savunma sistemlerinin tepki süresini kritik bir darboğaza sokmaktadır.

Endüstride uzun süre standart olarak kabul edilen imza tabanlı sistemler, bilinen zararlıların özet (hash) değerlerine dayanmaktadır. Ancak bu *reaktif* model, iki temel noktada başarısız olmaktadır:

- **Sıfırıncı Gün Saldırıları:** Hiç görülmemiş bir saldırı kodu, herhangi bir imzaya sahip olmadığı için veri tabanı güncellenene kadar sistemde serbestçe hareket edebilmektedir.
- **Polimorfik ve Metamorfik Yapılar:** Saldırganlar, kodun işlevini değiştirmeden dijital imzasını her kurban için farklılaştıran algoritmalar kullanmaktadır. Bu teknik, imza tabanlı sistemlerin ”imza-eşleşme” mantığını tamamen etkisiz kılmaktadır [7].

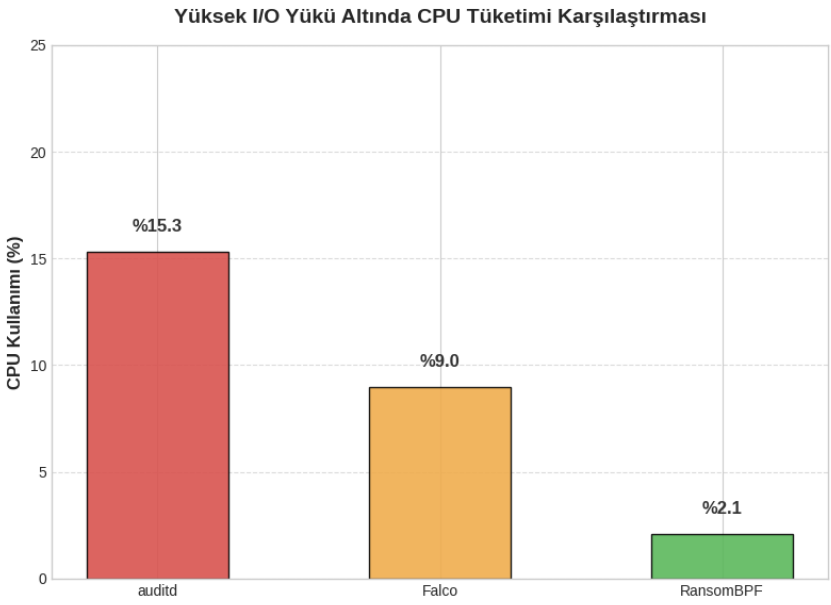
Davranışsal analiz yapan sistemler ise genellikle nicel verilere odaklanmaktadır. Ancak meşru bir yedekleme yazılımı ile bir fidye yazılımı, benzer I/O karakteristikleri sergileyebilir. Dolayısıyla, sadece ”saniyede yapılan işlem sayısı” gibi kaba metriklerin ötesine geçen, işlemin bağlamını (dosya konumu, uzantı değişimi, süreç geçmişi) analiz eden sistemlere ihtiyaç duyulmaktadır [8].

1.1.2 Linux Sunucu Güvenliği ve Mevcut Araçların Kısıtları

Linux, modern veri merkezlerinin ve bulut altyapılarının %90'ından fazlasına güç vermektedir [9]. Bu durum, Linux'u fidye yazılımı grupları için en cazip hedef haline getirmiştir. Ancak Linux ekosistemindeki mevcut güvenlik araçları, performans ve görünürlük arasında ciddi ödünleşimler içermektedir:

- **Auditd (Linux Audit System):** Detaylı olay günlükleri sağlar ancak yüksek trafikli sunucularda CPU ve I/O maliyeti çok yüksektir. Ayrıca, olayları yakaladıktan sonra kullanıcı alanına aktarma hızı, aktif bir saldırıyı durdurmak için yeterli değildir [10].
- **ClamAV ve Benzeri Tarayıcılar:** Statiktirler; dosya sistemi üzerinde o an gerçekleşen şüpheli süreci durdurma yetenekleri yoktur [7].
- **Kullanıcı Alanı İzleme:** Birçok EDR çözümü kullanıcı alanında çalışır. Bu durum, saldırganın çekirdek seviyesindeki hilelerle veya doğrudan sistem çağrılarını manipüle ederek bu araçları *kör etmesine* neden olabilir.

Bu bağlamda, eBPF teknolojisi bir *”oyun değiştirici”* olarak ortaya çıkmaktadır. eBPF, çekirdek seviyesindeki olayları neredeyse sıfıra yakın performans kaybıyla izlemeye ve saldırı anında (sistem çağrısı tamamlanmadan) müdahale etmeye olanak tanır [4]. Proje kapsamında geliştirilen **RansomBPF**, bu kısıtları aşarak Linux çekirdeğinin kalbinde çalışan, düşük overhead'li ve proaktif bir koruma kalkanı sunmayı amaçlamaktadır.



Şekil 1.1. Auditd, Falco ve RansomBPF stres testi sonuçları.

1.2 Motivasyon

Bu çalışmanın temel motivasyonu, modern siber tehdit ekosisteminde bir *asimetrik savaş* unsuru haline gelen fidye yazılımlarına karşı; düşük maliyetli, yüksek performanslı ve *proaktif* bir savunma mekanizması geliştirmektir [11]. Mevcut güvenlik yazılımlarının Linux sunucu ekosistemindeki performans darboğazları ve yeni nesil saldırılar karşısındaki kısıtları, **RansomBPF** sisteminin mimari tercihlerini doğrudan şekillendirmiştir. Bu bağlamda projenin itici gücü, eBPF teknolojisinin sunduğu mimari devrim ve davranışsal analizin sağladığı *semantik derinlik* olmak üzere iki temel eksende toplanmaktadır.

Sistem güvenliğinde çekirdek seviyesinde izleme yapabilmek, geleneksel olarak ya çekirdek kaynak kodunun değiştirilmesini ya da riskli Linux Çekirdek Modüllerinin (LKM) kullanılmasını gerektirmekteydi. Ancak eBPF teknolojisi, bu süreci tamamen dönüştürerek güvenli ve yüksek performanslı bir orta yol sunmuştur. eBPF programları, kullanıcı alanı ile çekirdek alanı arasındaki pahalı bağlam geçişlerini minimize ederek sistem olaylarını doğrudan çekirdek içinde filtreleyebilmektedir. eBPF'in **Doğrulayıcı** mekanizması sayesinde, yazılan kodun sistemi çökertmeyeceği veya sonsuz döngüye girmeyeceği çekirdek tarafından garanti altına alınır [4]. Ayrıca, sistemi yeniden başlatmaya gerek duymadan dinamik olarak kancalar atabilme yeteneği, 7/24 hizmet veren sunucu altyapıları için vazgeçilmez bir avantajdır.

Teknolojik altyapının sağladığı bu performans avantajı, analitik tarafta *davranışsal analiz* metodolojisi ile anlam kazanmaktadır. Geleneksel imza tabanlı sistemlerin, dosya hash değerlerine dayanması nedeniyle *Zero-day* saldırıları ve *polimorfik* yazılımlar karşısında başarısız olduğu bilinmektedir. Bu projenin temel motivasyonlarından biri, zararlı yazılımın kimliği yerine eylemlerine odaklanarak bu kör noktayı gidermektir. Bir fidye yazılımı ne kadar karmaşık olursa olsun,

hedefine ulaşmak için belirli bir sistem çağrısı akışını izlemek zorundadır [8]. Yazma, yeniden adlandırma ve yedekleri silme gibi sistem çağrılarının sadece varlığını değil, bu çağrıların semantiğini, sıklığını ve hedef aldığı dizinlerin kritiklik derecesini incelemek; saldırıyı henüz ilk aşamalarında yakalayabilmemizi sağlar.

Sonuç olarak, eBPF'in sağladığı çekirdek seviyesindeki benzersiz görünürlük ile sistem çağrılarının semantik analizini birleştirmek, Linux sistemlerini korumak adına modern, hafif ve tavizsiz bir güvenlik mimarisi oluşturmanın en etkili yoludur. **RansomBPF**, bu iki disiplini harmanlayarak reaktif savunmadan proaktif önlemeye geçişi mümkün kılan bir çözüm sunmayı hedeflemektedir.

1.3 Projenin Amacı ve Hedefleri

Bu çalışmanın temel amacı; modern veri merkezlerinin ve bulut altyapılarının omurgasını oluşturan Linux sistemlerini, giderek karmaşıklaşan fidye yazılımı saldırılarına karşı korumak için yüksek performanslı, düşük maliyetli ve *proaktif* bir savunma mimarisi geliştirmektir. **RansomBPF** adı verilen bu sistem, geleneksel imza tabanlı çözümlerin aksine, eBPF teknolojisini kullanarak çekirdek seviyesinde davranışsal analiz yapmayı ve tespit edilen tehditlere milisaniyeler düzeyinde müdahale ederek veri kaybını durdurmayı hedefleyen bir Saldırı Önleme Sistemi (IPS) olarak tasarlanmıştır.

Bu ana hedef doğrultusunda projenin alt hedefleri şu teknik ve operasyonel maddelerden oluşmaktadır:

- **eBPF Ring Buffer Tabanlı Yüksek Performanslı Veri Hattı Kurulması:** Çekirdek alanında izlenen `execve`, `write`, `renameat2` ve `unlinkat` gibi kritik sistem çağrılarından elde edilen ham verilerin, kullanıcı alanı ajanına kayıpsız ve minimum gecikmeyle iletilmesi hedeflenmiştir. Bu amaçla, veri kopyalama maliyetini düşüren ve saniyede binlerce olay işleme kapasitesine sahip olan `BPF_MAP_TYPE_RINGBUF` mekanizması projeye entegre edilmiştir [12].

- **Puan Tabanlı Dinamik ve Bağlam Duyarlı Analiz Motoru Oluşturulması:** Statik eşik değerlerinin yarattığı hatalı alarm riskini azaltmak amacıyla, her bir sistem olayına ağırlıklandırılmış puanlar atayan bir motor geliştirilmesi hedeflenmiştir. Bu motor; dosya uzantısı değişimleri, işlem yapılan dizinin kritikliğine göre belirlenen çarpanlar ve riskin zamanla sönümlenmesini sağlayan *"Decay Algorithm"* (Sönümleme Algoritması) ile akıllı bir karar verme mekanizması sunmaktadır [8].

- **Milisaniyeler İçinde Aktif Müdahale ve Tehdit Önleme (IPS):** Sistemin sadece bir gözlemci olmaktan çıkıp, saldırıyı doğrudan durdurabilmesi amaçlanmıştır. Risk puanı belirlenen eşiği aştığında, ilgili şüpheli sürecin (`PID`) 100 milisaniyenin altında bir sürede `SIGKILL` sinyali ile sonlandırılması hedeflenmiştir. Bu sayede, fidye yazılımı simülasyonlarında 50 dosyadan oluşan kurban veri setinde 3 dosyadan fazla kaybın yaşanmaması temel başarı kriteri olarak belirlenmiştir [3].

- **Adli Analiz İçin Yapılandırılmış Loglama Altyapısı:** Tespit edilen saldırıların kök neden

analizinin yapılabilmesi ve `SIEM` platformları ile tam entegrasyon sağlanması için olayların `JSON` formatında kayıt altına alınması hedeflenmiştir. Her bir alarmın; kullanıcı kimliği (`UID`), süreç kimliği (`PID`), ebeveyn süreç (`PPID`) ve hedef dosya yolları gibi zengin meta verilerle desteklenerek adli bilişim standartlarında raporlanması amaçlanmıştır [13].

- **Operasyonel Kararlılık ve Düşük Sistem Yükü:** Güvenlik yazılımının sistem performansına olan etkisinin %1-2 bandında tutulması hedeflenmiştir. Bu doğrultuda, beyaz liste kontrolleri için $O(1)$ performanslı hash tablolarının kullanılması, ajanın kendi yarattığı olayları filtreleyebilmesi ve sistem açılışında otomatik devreye giren bir `systemd` servisi olarak kararlı çalışması sağlanmıştır [4].

1.4 Projenin Kapsamı ve Sınırları

Geliştirilen **RansomBPF** sisteminin başarısı, hedeflenen tehdit modeline odaklanması ve bu odağın dışındaki bileşenleri performans kaygıları nedeniyle kapsam dışı bırakmasıyla doğrudan ilişkilidir. Projenin operasyonel sınırları; donanım mimarisi, izlenen çekirdek olayları ve analiz derinliği olmak üzere üç ana eksende tanımlanmıştır.

Sistemin temel çalışma platformu, `x86_64` mimarisine sahip modern Linux dağıtımları olarak belirlenmiştir. eBPF teknolojisinin sunduğu en büyük avantajlardan biri olan taşınabilirlik özelliğinden yararlanmak adına, hedef sistemin BTF (BPF Type Format) desteğine sahip bir çekirdek barındırması gerekmektedir. BTF desteği, eBPF programlarının farklı çekirdek sürümlerinde yeniden derlenmeye gerek kalmadan çalışabilmesini sağlayan kritik bir altyapıdır [14]. Bu bağlamda proje, bulut sunucuları ve kurumsal iş istasyonlarında en yaygın kullanılan mimariyi hedefleyerek pratik bir uygulama alanı sunmaktadır.

İzleme kapasitesi açısından proje, fidye yazılımlarının saldırı zinciri içerisinde sergilediği dosya sistemi manipülasyonlarına odaklanmaktadır. Bu kapsamda; süreçlerin başlatılmasını izleyen `execve`, veri şifreleme eylemlerini temsil eden `write` ve `pwrite64`, dosya uzantılarının değiştirilmesini yakalayan `renameat2` ve sistem yedeklerinin silinmesini denetleyen `unlinkat` gibi kritik sistem çağrıları kancalanmaktadır [3]. Proje, bu sistem çağrılarının sadece varlığını değil, çağrı sırasında iletilen dosya yolları, işlem bayrakları ve süreç meta verileri gibi semantik bilgileri de analiz kapsamına almaktadır.

Sistemin sınırları ve kapsam dışı bırakılan unsurları ise projenin performans odaklı karakterini korumak amacıyla netleştirilmiştir. **RansomBPF**, bir uç nokta tespit ve yanıt sisteminin tüm fonksiyonlarını barındırmayı hedeflemez; bu doğrultuda ağ trafiği analizi ve kullanıcı alanı bellek taraması tamamen kapsam dışı bırakılmıştır. Sistemin odak noktası tamamen I/O davranışları ve süreçlerin dosya sistemi üzerindeki etkileridir. Ağ üzerinden gelen saldırı vektörlerini (örneğin bir exploit girişi) tespit etmek yerine, bu saldırının sistemdeki nihai sonucu olan yetkisiz dosya şifreleme eylemini durdurmaya odaklanılmıştır. Bu sınırlama, sistemin çekirdek üzerindeki ek yükünü minimize ederek yüksek trafikli sunucularda dahi kararlı bir şekilde çalışabilmesine olanak tanımaktadır.

1.5 Tezin Organizasyonu

Bu tez çalışması, giriş bölümünü takip eden ve projenin teorik temellerinden pratik uygulama sonuçlarına kadar uzanan süreci detaylandıran yedi ana bölümden oluşmaktadır:

- **2. Bölüm Literatür ve Teknolojik Altyapı:** Fidye yazılımlarının evrimini, Linux ekosistemindeki saldırı vektörlerini ve saldırı zinciri aşamalarını akademik bir perspektifle incelemektedir. Ayrıca, sistemin kalbini oluşturan eBPF teknolojisi, çekirdek ve kullanıcı alanı etkileşimi ve `libbpf` kütüphanesi gibi temel bileşenlerin teknik altyapısı bu bölümde açıklanmaktadır [15].
- **3. Bölüm Sistem Tasarımı ve Mimarisi: RansomBPF**'in modüler yapısını ve veri boru hattını ele almaktadır. eBPF çekirdek programlarının sistem çağrılarını nasıl yakaladığı, Ring Buffer mekanizması ile verilerin kullanıcı alanına nasıl aktarıldığı ve `uthash` tabanlı durum yönetimi mekanizmasının tasarımı bu bölümün odağını oluşturmaktadır [16].
- **4. Bölüm Davranışsal Analiz ve Tespit Motoru:** Projenin en kritik bileşeni olan ağırlıklandırılmış risk puanlama algoritmasını detaylandırmaktadır. Bu bölümde; dizin hassasiyeti, uzantı analizi, tuzak dosya metodolojisi ve riskin zamanla doğal soğumasını sağlayan *"Decay Algorithm"* (Sönümleme Algoritması) ile akıllı bir karar verme mekanizması sunmaktadır.
- **5. Bölüm Aktif Müdahale ve IPS Mekanizması:** Tespit edilen tehditlerin `SIGKILL` sinyali ile milisaniyeler düzeyinde nasıl sonlandırıldığını ve sistemin operasyonel kararlılığını korumak için uygulanan beyaz liste filtreleme yöntemlerini açıklamaktadır.
- **6. Bölüm Test Metodolojisi ve Bulgular:** Geliştirilen sistemin başarısını doğrulamak amacıyla kurulan simülasyon altyapısını ve elde edilen verileri içermektedir. Gerçek zamanlı saldırı senaryoları, birim testler ve sistemin performans üzerindeki etkisi bu bölümde nicel verilerle analiz edilmektedir.
- **7. Bölüm Sonuç ve Gelecek Çalışmalar:** Projenin ulaştığı `v0.9.0` sürümündeki çıktıları özetlemekte, çalışmanın siber güvenlik literatürüne katkısını tartışmakta ve Linux Güvenlik Modülleri (LSM) entegrasyonu gibi gelecek vizyonları için öneriler sunmaktadır.

2. LİTERATÜR ANALİZİ VE TEORİK ALTYAPI

Siber güvenlik dünyasında tehdit vektörleri sürekli evrilmekte olup, **fidye yazılımları** veri bütünlüğü ve erişilebilirliğine yönelik en yıkıcı saldırı türlerinden biri olarak varlığını sürdürmektedir. Geçmişte ağırlıklı olarak son kullanıcı odaklı Windows sistemlerini hedef alan bu zararlı yazılımlar, dijital dönüşümle birlikte kritik altyapıların, bulut sunucularının ve kurumsal veri merkezlerinin omurgasını oluşturan Linux işletim sistemlerine yönelmiştir [17]. Linux ortamlarındaki fidye yazılımı saldırıları, işletim sisteminin açık kaynak kodlu yapısını, süreç yönetimini ve dosya sistemi hiyerarşisini karmaşık yöntemlerle istismar etmektedir.

Bu bölümde, fidye yazılımlarının tarihsel gelişimi ve modern Linux sistemleri üzerindeki çalışma mekanizmaları literatürdeki güncel çalışmalar ışığında incelenmiştir. Saldırıların tespitinde kullanılan geleneksel *imza tabanlı* yöntemlerin kısıtları irdelenmiş ve neden *davranışsal analiz* yöntemlerine ihtiyaç duyulduğu tartışılmıştır [7]. Ayrıca, bu tezin teknik altyapısını oluşturan ve Linux çekirdeği üzerinde güvenli, yüksek performanslı izleme imkanı sunan **eBPF** teknolojisinin mimarisi, çalışma prensipleri ve güvenlik mekanizmaları detaylıca ele alınmıştır.

2.1 Fidye Yazılımlarının Teknik Analizi ve Evrimi

2.1.1 Saldırı Yaşam Döngüsü

Bir fidye yazılımı saldırısı, rastgele gerçekleşen bir olaydan ziyade, belirli aşamaları takip eden planlı bir süreçtir. Literatürde *"Cyber Kill Chain"* olarak adlandırılan bu süreç, Linux sistemlerinde genel olarak şu dört ana fazda gerçekleşmektedir [18].

1. **Sızma ve Yerleşme:** Saldırganlar genellikle `RDP/SSH` gibi uzaktan yönetim servislerine yönelik kaba kuvvet saldırıları, phishing e-postaları veya bilinen yazılım zafiyetlerini kullanarak sisteme ilk erişimi sağlarlar.

2. **Keşif ve Hazırlık:** Zararlı yazılım, bulaştığı sistemdeki dosya yapısını tarar. Özellikle `/var/www` (web sunucusu), `/var/lib/mysql` (veritabanı) veya `/home` gibi kritik veri barındıran dizinler hedeflenir. Bu aşamada, dosya kilitlenmelerini önlemek ve şifreleme sürecini kesintisiz tamamlamak amacıyla çalışan veritabanı servisleri ve yedekleme süreçleri saldırgan tarafından sonlandırılır.

3. **Hedef Odaklı Eylem:** Saldırının en yıkıcı aşamasıdır. Dosyalar, simetrik ve asimetrik şifreleme algoritmalarının *hibrit* bir yapısı kullanılarak şifrelenir. Dosya içeriği okunur, şifrelenir ve diske geri yazılır. Bu işlem sırasında yüksek I/O aktivitesi oluşur [19].

4. **Temizlik ve Bildirim:** Şifreleme tamamlandıktan sonra, tespit edilmeyi zorlaştırmak adına zararlı yazılım kendini silebilir. Son olarak, kullanıcıya fidye notunu içeren bir dosya bırakılır ve masaüstü arka planı değiştirilir.

2.1.2 Şifreleme Stratejileri ve Entropi Analizi

Fidye yazılımlarının temel amacı, tespit edilmeden önce maksimum sayıda dosyayı şifrelemektir. Bu doğrultuda farklı şifreleme stratejileri geliştirilmiştir:

- **Tam Şifreleme**: Dosyanın tamamının şifrelendiği yöntemdir. Veri kurtarmayı imkansız kılsa da, büyük dosyaların (örneğin 50GB'lık bir veritabanı yedeği) şifrelenmesi uzun sürdüğü için CPU ve Disk I/O kullanımında belirgin anomaliler yaratarak tespit edilme riskini artırır [3].
- **Kısmi Şifreleme**: Tespit edilme süresini kısaltmak ve sistem performansını daha az etkilemek amacıyla geliştirilen modern bir yöntemdir. Dosyanın tamamı yerine sadece belirli blokları (örneğin her 1 MB'da bir 64 KB) şifrelenir. Bu yöntem, dosyanın bütünlüğünü bozarak kullanılamaz hale getirmek için yeterlidir ve tam şifrelemeye göre çok daha hızlıdır [20].

Bu süreçlerin tespitinde **Shannon Entropisi** kritik bir metrik olarak kullanılmaktadır. Normal metin dosyaları veya kod dosyaları düşük entropiye sahipken, şifrelenmiş veya sıkıştırılmış veriler rastgelelik içerdiği için maksimum entropi değerine (8 bitlik sistemde 8.0'a yakın) sahiptir. Dosya yazma işlemleri sırasında ani entropi artışları, fidye yazılımı aktivitesinin en güçlü göstergelerinden biridir [21].

2.1.3 Linux Odaklı Tehditler

Geçmişte Windows işletim sistemleri fidye yazılımlarının birincil hedefiyken, günümüzde kurumsal verilerin, bulut altyapılarının ve sanallaştırma platformlarının Linux üzerinde çalışması, saldırıların yönünü değiştirmiştir. Linux'a yönelik tehditlerin artmasının temel nedenleri şunlardır:

- **Yüksek Değerli Hedefler:** Linux sunucuları genellikle kritik iş verilerini barındırır. Bu sistemlerin şifrelenmesi, kurumlar için büyük maddi kayıplara ve operasyonel duruşlara neden olmaktadır [22].
- **Sanallaştırma Katmanları (`ESXi`):** `VMware ESXi` gibi Linux tabanlı hipervizörler, saldırganların favori hedefi haline gelmiştir. Saldırganlar, her bir sanal makineye ayrı ayrı sızmak yerine, doğrudan hipervizör seviyesinde şifreleme yaparak o sunucudaki tüm sanal makineleri tek seferde erişilemez hale getirebilmektedir [23].
- **İzin ve Yetki Yapısı:** Linux'un esnek dosya sistemi yapısı, yetkisiz erişim sağlayan bir saldırganın `root` yetkileriyle sistem genelinde hızlı ve yıkıcı değişiklikler yapmasına olanak tanır. Saldırganlar genellikle `chmod` ve `chown` komutlarını manipüle ederek veya sistem ikililerini değiştirerek kalıcılık sağlarlar.

2.2 Tehdit Tespit Metodolojileri

Siber güvenlik literatüründe zararlı yazılım tespiti, analizin gerçekleştirildiği zamana ve kullanılan tekniklerin niteliğine göre temel olarak iki ana kategoride incelenmektedir: **Statik (İmza Tabanlı) Analiz** ve **Dinamik (Davranış Tabanlı) Analiz**. Günümüzde gelişmiş tehdit aktörleri, saldırılarını gizlemek için sürekli olarak yöntem değiştirdiğinden, savunma mekanizmaları da bu değişime ayak uydurmak adına *hibrit* mimarilere evrilmektedir [24]. Bu bölümde, söz konusu metodolojilerin çalışma prensipleri, avantajları ve modern fidye yazılımları karşısındaki kısıtları teknik detaylarıyla irdelenmiştir.

2.2.1 Statik Analiz ve İmza Tabanlı Tespit Yöntemleri

Statik analiz, şüpheli dosyanın işlemci üzerinde çalıştırılmasına gerek duyulmadan, ikili kod yapısının veya kaynak kodunun incelenmesi esasına dayanır. Geleneksel antivirüs yazılımlarının omurgasını oluşturan bu yaklaşım, temel olarak bilinen zararlı yazılımların parmak izlerinin bir veritabanı ile karşılaştırılması prensibiyle çalışır.

Bu yöntemde kullanılan en yaygın teknik **Hash Tabanlı Doğrulamadır**. Dosyanın içeriğinden `MD5`, `SHA-1` veya `SHA-256` gibi kriptografik özetleme algoritmaları kullanılarak benzersiz bir karakter dizisi üretilir. Elde edilen bu özet değer, güvenlik firmaları tarafından güncellenen küresel tehdit istihbarat veritabanlarındaki bilinen zararlı özetleriyle karşılaştırılır. Eşleşme sağlanması durumunda dosya zararlı olarak etiketlenir ve karantinaya alınır.

Bunun yanı sıra, dosya içerisindeki belirli bayt dizilerini, şüpheli kütüphane çağrılarını veya derleyici metadatasını inceleyen **Sezgisel Statik Analiz** yöntemleri de kullanılmaktadır. Örneğin, bir dosyanın kod bölümünde şifreleme ile ilgili `API` fonksiyonlarının yoğunluğu veya dosyanın paketlenmiş bir yapıya sahip olması, statik analiz motorları için birer risk göstergesidir [7].

Ancak, statik analizin modern fidye yazılımları karşısındaki etkinliği, saldırganların geliştirdiği kaçınma teknikleri nedeniyle giderek azalmaktadır:

- **Polimorfizm ve Metamorfizm:** Saldırganlar, zararlı yazılımın kaynak kodunu her yeni saldırıda veya her derleme işleminde otomatik olarak değiştirerek (örneğin; gereksiz `NOP` komutları ekleyerek veya kayıtçı isimlerini değiştirerek) dosyanın hash değerini tamamen değiştirebilmektedir. Bu durum, imza tabanlı sistemlerin *"Sıfır Gün"* (Zero-day) olarak adlandırılan yeni varyantları tespit etmesini imkansız kılmaktadır [25].

- **Paketleme ve Şifreleme:** Zararlı kodun asıl işlevini gizlemek için *"Packer"* veya *"Crypter"* adı verilen araçlar kullanılmaktadır. Bu araçlar, zararlı kodu şifreli bir katman altına gizler ve çalışma zamanında bellekte açılmasını sağlar. Statik analiz araçları sadece dış kabuğu gördüğü için içerideki asıl tehdidi tespit edememektedir [26].

2.2.2 Dinamik Analiz ve Davranışsal İzleme

Dinamik analiz, statik yöntemlerin aksine, dosyanın kontrollü bir ortamda veya izlenen bir ana bilgisayar üzerinde çalıştırılarak, gerçekleştirdiği eylemlerin gerçek zamanlı olarak takip edilmesidir. Bu yaklaşımda dosyanın ”nasıl göründüğü” değil, ”ne yaptığı” analiz edilir. Modern güvenlik sistemlerinde, fidye yazılımlarının tespiti için birincil yöntem haline gelmiştir [27].

Davranışsal analiz motorları, bir sürecin sistem üzerindeki etkileşimlerini izlemek için genellikle şu teknikleri kullanır:

- **API Kancalama**: İşletim sistemi tarafından sunulan fonksiyonların (örn: `Windows API` veya `Linux Syscalls`) araya girilerek dinlenmesidir. Bir program dosya açmak, ağ bağlantısı kurmak veya kayıt defterini değiştirmek istediğinde, bu istek önce güvenlik yazılımına yönlendirilir.
- **Sistem Çağrıları Analizi:** İşletim sisteminin çekirdek seviyesine gönderilen `read`, `write`, `open`, `execve` gibi düşük seviyeli komutların izlenmesidir. Bu yöntem, kullanıcı alanındaki manipülasyonlardan etkilenmediği için daha güvenilirdir.

Fidye yazılımları, hangi şifreleme algoritmasını veya gizlenme tekniğini kullanırlarsa kullansınlar, dosyaları şifrelemek için sistem üzerinde belirli bir davranışsal kalıp sergilemek zorundadırlar. Bu kalıplar şunları içerir [3]:

1. Kısa süre içerisinde çok sayıda dosyanın açılması ve üzerine yazılması (Yüksek I/O Aktivitesi).
2. Dosya içeriklerinin okunabilir metin formatından (düşük entropi), rastgele veri formatına (yüksek entropi) dönüştürülmesi.
3. Sistem geri yükleme noktalarının veya yedekleme dosyalarının silinmesi.
4. Dosya uzantılarının toplu olarak değiştirilmesi (*Rename Operations*).

Dinamik analizin en büyük avantajı, daha önce hiç görülmemiş bir tehdidi sadece davranışlarına bakarak tespit edebilmesidir. Ancak, bu yöntemlerin de performans maliyeti ve *yanlış pozitif* (false positive) oranlarının yüksek olması gibi dezavantajları bulunmaktadır. Örneğin, büyük bir veritabanı yedeği alan veya video işleyen meşru bir yazılım, yüksek disk aktivitesi nedeniyle hatalı bir şekilde fidye yazılımı olarak etiketlenebilmektedir. Bu nedenle, dinamik analiz sistemlerinde sadece işlem sayısına değil, işlemin bağlamına bakan akıllı algoritmalara ihtiyaç duyulmaktadır [8].

2.3 eBPF (Extended Berkeley Packet Filter) Teknolojisi

Linux işletim sistemlerinde programlanabilirlik ve gözlemlenebilirlik kavramlarını yeniden tanımlayan **eBPF**, işletim sisteminin çekirdeğini kaynak kodunu değiştirmeye gerek kalmadan ge-

Tablo 2.1. Statik ve Dinamik Analiz Yöntemlerinin Karşılaştırılması

Kriter	**Statik Analiz (İmza Tabanlı)**	**Dinamik Analiz (Davranış Tabanlı)**
Analiz Zamanı	Dosya çalıştırılmadan önce.	Dosya çalışma zamanında.
Tespit Yöntemi	Kod yapısı, hash değeri ve dizgi (string) eşleşmesi.	Sistem çağrıları, API kancalama ve kaynak kullanımı.
Sıfır Gün Başarısı	Düşük. Bilinmeyen tehditleri tespit edemez.	Yüksek. Anomali tespiti ile bilinmeyen tehditleri yakalayabilir.
Kaçınma Direnci	Düşük. Polimorfizm ve Packer kullanımı ile kolayca atlatılabilir.	Yüksek. Kodun bellekte açılmış halini ve gerçek eylemlerini görür.
Kaynak Tüketimi	Düşük. Sadece disk taraması yapar.	Yüksek. Sanal ortam veya sürekli izleme gerektirir.
Yanlış Pozitif	Düşük. İmza eşleşmesi kesindir.	Orta/Yüksek. Meşru yazılımlar zararlı sanılabilir.

nişletmeyi sağlayan devrimsel bir teknolojidir. Bu başlık altında, eBPF'in tarihsel dönüşümü, mimari bileşenleri ve geleneksel güvenlik izleme yöntemlerine göre sağladığı performans avantajları incelenmiştir.

2.3.1 Tarihsel Gelişim: Klasik BPF'den Modern eBPF'e

eBPF'in kökleri, 1992 yılında Steven McCanne ve Van Jacobson tarafından yayınlanan *"The BSD Packet Filter: A New Architecture for User-level Packet Capture"* makalesine dayanmaktadır [28]. O dönemde `cBPF` (Classic BPF) olarak adlandırılan bu teknoloji, ağ paketlerini filtrelemek amacıyla geliştirilmiş basit bir sanal makine yapısıydı. cBPF, 32-bit kayıtçılara ve sınırlı bir komut setine sahipti.

2014 yılında Alexei Starovoitov öncülüğünde bu yapı köklü bir değişime uğramıştır. **eBPF** (Extended BPF) olarak adlandırılan yeni mimari ile:

- Kayıtçı sayısı 2'den 10'a çıkarılmış ve genişliği 64-bit'e yükseltilmiştir (modern işlemci mimarilerine uyum).
- Sadece ağ paketlerine değil, `kprobe`, `tracepoints` ve `perf_events` gibi sistem olaylarına erişim imkanı sağlanmıştır.
- Sonuç: BPF, sadece bir paket filtreleme aracı olmaktan çıkıp, çekirdek içinde çalışan genel amaçlı, olay tabanlı bir yürütme motoruna dönüşmüştür.

2.3.2 Mimari Bileşenler ve Çalışma Prensibi

eBPF ekosistemi, geliştirilen kodun güvenli ve verimli bir şekilde çalışmasını sağlayan üç temel bileşenden oluşur: **Doğrulayıcı**, **JIT Derleyici** ve **Haritalar**.

1. **Doğrulayıcı**: Çekirdek güvenliğinin garantörüdür. Kullanıcı alanında yazılan bir eBPF prog-

ramı çekirdeğe yüklenmek istendiğinde, Verifier kodu statik analize tabi tutar. Bir Yönlendirilmiş Döngüsüz Grafik (`DAG`) oluşturarak programın her olası yürütme yolunu tarar [4].

- **Sonsuz Döngü Kontrolü:** Programın belirli bir süre içinde tamamlanacağını garanti eder; aksi takdirde sistemi kilitleyebilecek kodlar reddedilir.
- **Bellek Erişimi:** Programın sadece izin verilen bellek alanlarına eriştiğini doğrular. Yetkisiz *pointer* erişimleri engellenir.

2. **JIT Derleyici**: Doğrulayıcıdan geçen eBPF bayt kodu, JIT derleyici tarafından anlık olarak işlemcinin yerel makine diline dönüştürülür. Bu sayede, eBPF programları yorumlanan bir dil yavaşlığında değil, doğrudan çekirdek kodunun bir parçasıymış gibi yerel hızda çalışır.

3. **eBPF Haritaları**: eBPF programları durumsuzdur; yani bir olayı işledikten sonra hafızası silinir. Verilerin saklanması ve kullanıcı alanıyla paylaşılması için *"Maps"* adı verilen anahtar-değer veri yapıları kullanılır.

   - **Hash Maps:** Süreç bilgileri gibi ilişkisel verileri saklamak için kullanılır.
   - **Ring Buffer:** Olay verilerini kullanıcı alanına yüksek hızda aktarmak için kullanılan dairesel tampon bellektir.

2.3.3 Güvenlik İzlemede Performans ve Geleneksel Yöntemlerle Karşılaştırma

Fidye yazılımları saniyeler içinde binlerce dosyayı şifreleyebildiği için, savunma sisteminin tepki süresi ve sistem kaynaklarına etkisi kritik öneme sahiptir. Geleneksel yöntemler (`Auditd`, `ptrace`) ile eBPF tabanlı yaklaşım karşılaştırıldığında belirgin farklar ortaya çıkmaktadır.

Geleneksel Yöntem: Bağlam Değişimi maliyeti `ptrace` tabanlı sistem çağrı izleme araçları, izlenen her sistem çağrısında işlemi durdurur, kontrolü kullanıcı alanındaki izleme aracına devreder ve ardından tekrar çekirdeğe döner. Bu sürekli *Context Switch*, yoğun I/O işlemi yapan fidye yazılımları sırasında sistem performansını %40-%60 oranında düşürebilmektedir [29].

eBPF Yaklaşımı: Çekirdek İçi Analiz eBPF programları doğrudan çekirdek olayının gerçekleştiği yerde çalışır. Veri toplama ve filtreleme işlemi *kernel-space* içinde yapılır; sadece ilgili olaylar kullanıcı alanına gönderilir.

- **Düşük Maliyet:** Gereksiz bağlam değişimleri ortadan kalkar.
- **Güvenlik:** Çekirdek Modüllerinin aksine, eBPF kodundaki bir hata sistemi çökertmez (`Verifier` sayesinde).

Bu çalışmada yapılan ön testlerde, eBPF tabanlı izleme mekanizmasının sisteme getirdiği ek yükün %1-%3 bandında olduğu gözlemlenmiştir. Bu verimlilik, üretim ortamındaki sunucularda performans kaybı yaşatmadan aktif koruma sağlanabilmesine olanak tanır [4].

2.4 İlgili Çalışmalar ve RansomBPF'in Özgünlüğü

eBPF teknolojisinin sağladığı görünürlük avantajları, son yıllarda hem akademik hem de endüstriyel alanda güvenlik odaklı çeşitli projelerin geliştirilmesine zemin hazırlamıştır. Bu başlık altında, literatürde öne çıkan eBPF tabanlı güvenlik araçları incelenmiş ve bu tez çalışması kapsamında geliştirilen **RansomBPF**'in bu araçlardan ayrılan özgün yönleri ortaya konulmuştur.

2.4.1 Literatürdeki Mevcut Araçlar

Bulut tabanlı güvenlik ekosisteminde eBPF kullanımı denildiğinde akla gelen ilk projeler `Falco` ve `Tetragon`'dur.

- **`Falco`:** *Cloud Native Computing Foundation* (CNCF) bünyesinde geliştirilen `Falco`, sistem çağrılarını dinleyerek kural tabanlı tespit yapan bir çalışma zamanı güvenlik aracıdır. `Falco`, önceden tanımlanmış kurallara dayalı olarak çalışır. Temel odak noktası anormallikleri tespit edip loglamak (IDS) üzerine kuruludur; aktif engelleme yeteneği genellikle harici eklentilere bırakılmıştır [30].

- **`Tetragon`:** `Cilium` projesinin bir parçası olan `Tetragon`, özellikle ağ ve süreç izleme konusunda derinlemesine görünürlük sağlar. eBPF ile çekirdek seviyesinde filtreleme yaparak yüksek performans sunar. Ancak, genel amaçlı bir gözlemlenebilirlik aracı olması nedeniyle, fidye yazılımlarına özgü karmaşık davranışsal kalıpları kutudan çıktığı haliyle tespit etmek için kapsamlı konfigürasyon gerektirir [31].

2.4.2 RansomBPF'in Konumu ve Özgün Değeri

Literatürdeki mevcut araçlar genellikle genel amaçlı tehdit tespiti üzerine yoğunlaşmışken, **RansomBPF** doğrudan fidye yazılımlarının dosya sistemi üzerindeki davranışsal parmak izlerine odaklanan özelleşmiş bir mimariye sahiptir. Projenin özgün değer önerisi ve literatürdeki çalışmalardan farkları aşağıdaki maddelerde özetlenmiştir:

1. **Puan Tabanlı Risk Skoru:** `Falco` gibi araçlar genellikle "Eğer X olursa, Alarm üret" mantığıyla çalışır. **RansomBPF** ise olayları tekil olarak değerlendirmez; bir bağlam içerisinde ele alır. Geliştirilen risk motoru, yapılan işlemin türüne ve işlemin yapıldığı dizine göre dinamik ağırlıklandırma yapar. Bu sayede, tekil bir dosya değişikliği alarm üretmezken, kümülatif risk puanı belirli bir eşiği aştığında aksiyon alınır. Bu yaklaşım, *hatalı pozitif* oranını minimize etmektedir.

2. **Aktif Müdahale (IPS):** Çoğu eBPF tabanlı izleme aracı, tespit edilen tehdidi loglayarak sistem yöneticisine bildirir. **RansomBPF** ise tespit edilen tehdidin risk skoru kritik eşiği aştığı anda, çekirdek seviyesinden aldığı yetkiyle ilgili sürece milisaniyeler içerisinde `SIGKILL` sinyali gönderir. Bu özellik, **RansomBPF**'i sadece bir izleme aracı olmaktan çıkarıp, sal-

dırıyı şifreleme işlemi tamamlanmadan durdurabilen aktif bir koruma kalkanına dönüştürmektedir.

3. **Bağlam Duyarlı Analiz:** Sistem, sadece sistem çağrılarını saymakla kalmaz; dosya uzantılarının değişimini (örneğin `.docx` → `.locked`) ve işlemin yapıldığı dosyanın niteliğini analiz eder. Ayrıca, sisteme entegre edilen ”Tuzak Dosya” mekanizması sayesinde, saldırganın rastgele dosya şifreleme girişimi, risk puanını anında maksimize ederek tespit süresini hızlandırır.

Sonuç olarak; **RansomBPF**, eBPF teknolojisini sadece görünürlük için değil, davranışsal analiz ve otonom tepki mekanizmalarıyla birleştirerek, Linux sistemlerine yönelik fidye yazılımı tehditlerine karşı özelleşmiş, hafif sıklet ve proaktif bir çözüm sunmaktadır.

3. MİMARİ TASARIM VE SİSTEM BİLEŞENLERİ

Bu bölümde, fidye yazılımlarını (ransomware) gerçek zamanlı olarak tespit etmek ve engellemek amacıyla geliştirilen RansomBPF projesinin mimari tasarımı detaylandırılmıştır. Önerilen sistem mimarisi, Linux çekirdeğinin sağladığı yüksek performanslı izleme yetenekleri ile kullanıcı alanının (user space) esnek analiz gücünü birleştiren hibrit bir yapı üzerine inşa edilmiştir [4]. Geleneksel imza tabanlı (signature-based) yaklaşımların aksine, bu çalışma davranışsal analiz yöntemlerini esas almakta [7, 3] ve çekirdek seviyesinde görünürlük sağlamak için `eBPF` (Extended Berkeley Packet Filter) teknolojisini kullanmaktadır.

Sistemin genel tasarımı, performans darboğazlarını önlemek ve üretim ortamındaki sunucuların kararlılığını bozmamak adına ”hafif siklet izleme” (lightweight tracing) prensibine dayanmaktadır [12]. Bu bağlamda mimari, işlevsel olarak iki ana düzleme ayrılmıştır:

1. **Çekirdek Alanı (Kernel Space):** Sistem çağrılarının (syscalls) yakalandığı, minimum veri işleme ile filtrelendiği ve sadece gerekli olayların üst katmana aktarıldığı sensör katmanıdır.

2. **Kullanıcı Alanı (User Space):** Çekirdekten gelen ham verilerin anlamlandırıldığı, durum yönetiminin (state management) yapıldığı ve risk puanlama algoritmalarının koşulduğu karar mekanizmasıdır.

İki katman arasındaki veri iletişimi, senkronizasyon maliyetlerini minimize etmek amacıyla, kilitsiz (lockless) bir veri yapısı olan `BPF_MAP_TYPE_RINGBUF` üzerinden asenkron olarak sağlanmaktadır. Bu tasarım tercihi, yoğun dosya giriş-çıkış (I/O) işlemlerinin olduğu senaryolarda dahi sistemin olay kaybı yaşamadan (zero-drop) çalışabilmesine olanak tanımaktadır.

Aşağıdaki alt başlıklarda, sistemin bu iki temel bileşeni, kullanılan veri yapıları ve tehdit önleme mekanizmasının teknik detayları incelenecektir.

3.1 Sistemin Genel Çalışma Prensibi

RansomBPF, geleneksel antivirüs yazılımlarının aksine, disk taraması yapmak yerine işletim sistemi çekirdeği (kernel) üzerinde gerçekleşen olayları anlık olarak izleyen ”olay güdümlü” (event-driven) bir mimariye sahiptir [31]. Sistem, işlevsel açıdan iki ana katmana ayrılmıştır: **eBPF Çekirdek Modülü** ve **Kullanıcı Alanı Ajanı**. Bu iki katman arasındaki veri akışı, performans kaybını önlemek amacıyla asenkron bir yapı üzerinde kurgulanmıştır.

3.1.1 Yüksek Seviyeli Mimari

Sistemin yüksek seviyeli mimarisi, çekirdek alanının ”gözlemci” (observer), kullanıcı alanının ise ”karar verici” (decision maker) olarak konumlandırıldığı hibrit bir yapıdadır [30].

1. **eBPF Çekirdek Modülü (Data Plane):** Bu katman, işletim sisteminin çekirdeğine yüklenen ve güvenli bir sanal makine üzerinde çalışan bytecode'lardan oluşur. Temel görevi, dosya sistemi üzerindeki `write`, `pwrite64`, `rename` ve `unlink` gibi kritik sistem çağrılarını (sys-calls) yakalamaktır. Bu katmanda herhangi bir karmaşık analiz yapılmaz; amaç sadece olaya dair meta verileri (Süreç ID, dosya yolu, işlem türü) en az maliyetle toplamaktır [4].

2. **Kullanıcı Alanı Ajanı (Control Plane):** Çekirdekten gelen ham verileri işleyen, süreçlerin davranış geçmişini (state) tutan ve risk analizi yapan C tabanlı bir servistir. Tehdit tespiti, konfigürasyon yönetimi ve olay günlüğü (logging) işlemleri bu katmanda gerçekleştirilir.

3. **Veri İletişim Kanalı (Ring Buffer):** İki katman arasındaki veri alışverişi için `BPF_MAP_TYPE_RINGBUF` yapısı kullanılmıştır. Geleneksel `perf_buffer` yapısına kıyasla Ring Buffer, olayların sıralı işlenmesini garanti eder ve bellek tahsisi (memory allocation) maliyetlerini düşürerek yüksek yoğunluklu I/O işlemlerinde dahi veri kaybını önler [12].

3.1.2 Olay Akış Döngüsü ve Müdahale Mekanizması

Sistemin bir fidye yazılımı saldırısını tespit edip engellemesi süreci, nanosaniyeler mertebesinde gerçekleşen bir olay döngüsünü takip eder. Bu döngü, zararlı bir sürecin dosya sistemine erişim isteğiyle başlar ve aşağıdaki adımları izler:

1. **Tetikleme (Triggering):** Şüpheli süreç (örneğin bir fidye yazılımı), bir dosyayı şifrelemek veya uzantısını değiştirmek için çekirdeğe bir sistem çağrısı (örn: `sys_enter_renameat`) gönderir.

2. **Yakalama (Hooking):** Çekirdekteki ilgili kancaya (tracepoint) bağlı olan eBPF programı devreye girer. Sürecin kimlik bilgilerini (PID, UID) ve etkilenen dosya yolunu okur [15].

3. **İletim (Submission):** Toplanan veriler bir yapı (struct) içerisine paketlenir ve Ring Buffer halkasına yazılır.

4. **Tüketim (Consumption):** Kullanıcı alanında sürekli dinlemede olan ajan yazılım (daemon), Ring Buffer'dan gelen yeni veriyi okur.

5. **Analiz ve Puanlama (Scoring):** Ajan, ilgili sürecin geçmiş davranışlarını hash tablosundan (state table) çeker [16]. Mevcut işlem, dosya uzantısı ve dizin yolu gibi parametrelerle ağırlıklandırılarak bir "Risk Puanı" hesaplanır.

6. **Müdahale (Enforcement):** Eğer hesaplanan risk puanı belirlenen eşik değerini (Threshold) aşarsa, sistem saldırıyı doğrular. Ajan, işletim sistemine `SIGKILL` sinyali göndererek zararlı süreci anında sonlandırır ve yöneticiye JSON formatında bir alarm üretir [8].

Bu akış sayesinde, fidye yazılımı henüz disk üzerindeki tüm dosyaları şifreleyemeden, saldırının ilk aşamalarında (early-stage detection) engellenmiş olur [32].

3.2 Çekirdek Alanı (Kernel-Space) Programlama

RansomBPF mimarisinin veri toplama katmanı olan çekirdek alanı programı, Linux çekirdeğinin olay güdümlü yapısına entegre olarak çalışmaktadır [4]. Bu modülün temel sorumluluğu, dosya sistemi üzerindeki milyonlarca işlem arasından yalnızca fidye yazılımı (ransomware) davranışı sergileme potansiyeli olanları filtrelemek ve kullanıcı alanına iletmektir. Bu bölümde, eBPF program tiplerinin seçimi, izlenen sistem çağrılarının stratejik önemi ve modern çekirdek uyumluluğu için kullanılan teknolojiler detaylandırılmıştır.

3.2.1 İzleme Mekanizması: Tracepoint ve Kprobe Karşılaştırması

Linux çekirdeğinde eBPF ile izleme yapabilmek için `kprobe` (Kernel Probe) ve `tracepoint` olmak üzere iki temel mekanizma bulunmaktadır. Projenin geliştirme sürecinde, kararlılık ve sürdürülebilirlik gereksinimleri doğrultusunda **Tracepoint** yapısı tercih edilmiştir [28].

- **Kararlılık (Stability):** `kprobe`'lar, çekirdeğin iç fonksiyonlarına dinamik olarak kanca atar. Ancak çekirdek sürümleri değiştikçe bu fonksiyonların isimleri veya parametreleri değişebilir, bu da güvenlik aracının çalışmamasına neden olabilir. Buna karşın Tracepoint'ler, çekirdek geliştiricileri tarafından sağlanan kararlı bir API (Application Programming Interface) sunar.
- **Performans:** Tracepoint'ler, kod derlenirken statik olarak tanımlandığı için, çalışma zamanında (runtime) oluşan işlemci yükü `kprobe`'lara göre daha düşüktür ve deterministik bir performans sergiler [10].

Bu nedenlerle, projenin kaynak kodlarında (`hello_kern.c`), sistem çağrıları `SEC("tp/syscalls/...")` makrosu ile tanımlanan Tracepoint kancaları üzerinden dinlenmektedir.

3.2.2 İzlenen Kritik Sistem Çağrıları ve Veri Toplama Stratejisi

Bir fidye yazılımının yaşam döngüsü analiz edildiğinde, dosya sisteminde bıraktığı izlerin belirli sistem çağrılarına yoğunlaştığı görülmüştür [3]. RansomBPF, gereksiz veri gürültüsünü önlemek adına sadece aşağıdaki kritik çağrıları izlemektedir:

1. **`sys_enter_write`:** Standart dosya yazma işlemlerini yakalar. Dosyaların şifrelenmesi (içeriğinin değiştirilmesi) sürecini tespit etmek için kullanılır [8].
2. **`sys_enter_pwrite64` ve `sys_enter_writev`:** Python, Java ve Go gibi yüksek seviyeli dillerle yazılan modern fidye yazılımları, standart `write` çağrısı yerine ofset tabanlı (`pwrite64`) veya vektörel (`writev`) yazma işlemlerini kullanabilmektedir. Projede bu çağrıların izlenmesi, "kör nokta" sorununu ortadan kaldırmış ve tespit kapsamını genişletmiştir [1].

3. **`sys_enter_renameat` / `sys_enter_rename`:** Fidye yazılımları, şifreledikleri dosyaların uzantısını (örn. `.locked`, `.enc`) değiştirerek kullanıcıya mesaj verir. Bu davranışın tespiti için yeniden adlandırma çağrıları izlenmektedir [32].

4. **`sys_enter_unlinkat`:** Saldırganların, sistemin geri yüklenmesini engellemek amacıyla yedek dosyalarını (backup/shadow copies) silme girişimlerini yakalamak için silme çağrıları takip edilmektedir [5].

Aşağıdaki kod bloğunda, `hello_kern.c` dosyasından alınan ve bir yazma olayını yakalayan eBPF kancası görülmektedir:

```
SEC("tp/syscalls/sys_enter_write")
int handle_write(struct trace_event_raw_sys_enter *ctx)
{
    u64 id = bpf_get_current_pid_tgid();
    u32 pid = id >> 32;
    u32 uid = bpf_get_current_uid_gid();

    if (pid == self_pid)
        return 0;

    struct event *e;
    e = bpf_ringbuf_reserve(&rb, sizeof(*e), 0);
    if (!e)
        return 0;

    e->pid = pid;
    e->uid = uid;
    e->type = EVENT_WRITE;
    bpf_get_current_comm(&e->comm, sizeof(e->comm));

    bpf_ringbuf_submit(e, 0);
    return 0;
}
```

Kod 3.1. Çekirdek Alanında Yazma İşleminin Yakalanması

3.2.3 Modern Çekirdek Uyumluluğu: CO-RE ve BTF

Geleneksel Linux çekirdek modülleri, çalıştıkları çekirdek sürümüne özel başlık dosyaları (kernel headers) ile derlenmek zorundadır. Bu durum, güvenlik yazılımının taşınabilirliğini (portability) ciddi şekilde kısıtlamaktadır. RansomBPF projesinde bu sorunu aşmak için **CO-RE (Compile Once - Run Everywhere)** yaklaşımı benimsenmiştir [14].

Bu yaklaşımın temel bileşeni olan **BTF (BPF Type Format)**, çekirdek veri yapılarının (structs, unions) bellek düzenini tanımlayan, sıkıştırılmış bir meta veri formatıdır. Proje kodunda yer alan `#include "vmlinux.h"` satırı, sistemde yüklü olan tüm çekirdek tiplerini tek bir dosyada toplayarak BTF desteğini etkinleştirir [15].

Böylece, eBPF yükleyicisi (`libbpf`), hedef sistemdeki çekirdek sürümü farklı olsa bile, veri

yapılarındaki ofset (bellek adresi) kaymalarını otomatik olarak hesaplayıp düzeltebilmektedir. Bu sayede, geliştirilen ajan yazılım farklı Linux dağıtımlarında (Ubuntu, Debian, Fedora vb.) yeniden derleme gerektirmeden çalışabilmektedir.

3.3 Veri İletişim Kanalları ve Ring Buffer

Çekirdek alanında toplanan verilerin, analiz edilmek üzere kullanıcı alanına aktarılması süreci, yüksek performanslı güvenlik yazılımlarında en büyük darboğazlardan biridir. RansomBPF projesinde, çekirdek (kernel) ve kullanıcı (user) alanı arasındaki bu veri köprüsünü kurmak için modern Linux çekirdeklerinde sunulan `BPF_MAP_TYPE_RINGBUF` yapısı tercih edilmiştir [4]. Bu bölüm, veri iletişim kanalının teknik detaylarını ve veri paketleme standartlarını açıklamaktadır.

3.3.1 Ring Buffer Mimarisi ve Tercih Sebepleri

Projenin ilk prototip aşamalarında veri aktarımı için geleneksel `perf_buffer` yapısı test edilmiştir. Ancak, fidye yazılımlarının saniyede binlerce dosya işlemi gerçekleştirdiği yoğun yük senaryolarında `perf_buffer` yapısının yetersiz kaldığı gözlemlenmiştir. Bu sorunu çözmek amacıyla, Linux 5.8 sürümüyle gelen Ring Buffer teknolojisine geçiş yapılmıştır [12]. Bu teknik tercihin temel nedenleri şunlardır:

- **Bellek Verimliliği:** Eski `perf_buffer` mimarisi, her CPU çekirdeği için ayrı bir bellek alanı (buffer) tahsis edilmesini zorunlu kılar. Çok çekirdekli sunucularda bu durum bellek israfına yol açmaktadır. Ring Buffer ise tüm CPU'lar tarafından paylaşılan tek bir bellek haritası kullanarak sistem kaynaklarını daha verimli yönetir [12].
- **Veri Bütünlüğü ve Sıralama:** `perf_buffer` yapısında, farklı CPU'lardan gelen olayların zaman sıralamasının karışma ihtimali bulunmaktadır. Ring Buffer, olayları oluş sırasına göre kuyruğa alarak kullanıcı alanındaki analiz motorunun olayları kronolojik olarak işlemesini garanti eder [15].
- **Sıfır Kopya (Zero-Copy) ve Performans:** Ring Buffer, "Reserve-Commit" (Rezerve Et - Gönder) mekanizmasıyla çalışır. Veri, önce çekirdek belleğinde rezerve edilen alana yazılır ve ardından işaretlenir. Bu yöntem, verinin tekrar tekrar kopyalanmasını (double-copy) engelleyerek işlemci yükünü minimize eder ve veri kaybını (event drop) önler [12].

3.3.2 Veri Protokolü ve Olay Yapısı (Struct Event)

Çekirdek ile kullanıcı alanı arasında ortak bir dil oluşturmak amacıyla standart bir veri yapısı tanımlanmıştır. `common.h` başlık dosyasında belirtilen bu yapı, ağ üzerinden gönderilen bir paket gibi, her olayın kimliğini ve içeriğini taşımaktadır.

Tanımlanan `struct event` yapısı, bellek hizalama (memory alignment) kurallarına uygun olarak tasarlanmış olup, bir fidye yazılımı analizi için gerekli olan şu temel alanları içerir [3]:

- **Kimlik Bilgileri:** İşlemi yapan sürecin ID'si (PID) ve kullanıcı ID'si (UID).
- **İşlem Türü:** Dosya yazma, silme veya yeniden adlandırma olduğunu belirten tip (type) bilgisi.
- **Süreç Adı:** İşlemi gerçekleştiren komutun adı (örn: `python`, `mv`, `encrypter`).
- **Dosya Yolu:** İşlem yapılan dosyanın tam yolu.

Sistemin veri taşıma protokolünü oluşturan C yapısı aşağıda sunulmuştur:

```
struct event {
    u32 pid;
    u32 uid;
    u32 type;
    char comm[16];
    char filename[256];
};
```

Kod 3.2. Çekirdek ve Kullanıcı Alanı Arasında Paylaşılan Olay Veri Yapısı

Bu yapı sayesinde, kullanıcı alanındaki ajan, Ring Buffer'dan okuduğu her 300+ baytlık veri bloğunu doğrudan bu şablona oturtarak (casting) anlamlandırabilmektedir.

3.4 Modüler Yazılım Yapısı ve Bileşenler

RansomBPF projesinin kullanıcı alanı (user-space) mimarisi, bakım kolaylığı ve ölçeklenebilirlik sağlamak amacıyla modüler bir yapıda tasarlanmıştır. Tek bir monolitik kod bloğu yerine, her biri belirli bir sorumluluğu üstlenen ayrık modüller (Main, Detector, Config, Logger) oluşturulmuştur. Bu yaklaşım, "Sorumlulukların Ayrılığı" (Separation of Concerns) prensibine dayanmaktadır ve sistemin karmaşıklığını yönetilebilir kılmaktadır [15].

Aşağıda, sistemin dört temel yapı taşı ve bu bileşenlerin teknik işlevleri detaylandırılmıştır.

3.4.1 Ana Kontrolcü (Main Module)

Sistemin giriş noktası olan `main.c`, tüm bileşenlerin başlatılmasından, senkronizasyonundan ve yaşam döngüsünün yönetiminden sorumludur. Bu modülün öncelikli görevi, eBPF iskeletini (skeleton) belleğe yüklemek, çekirdek kancalarını (hooks) aktif hale getirmek ve Ring Buffer üzerinden veri akışını başlatmaktır [14].

Ayrıca, işletim sisteminden gelen `SIGINT` ve `SIGTERM` gibi sonlandırma sinyallerini yakalayarak sistemin güvenli bir şekilde kapatılmasını (graceful shutdown) sağlar. Bu işlem, bellek sızıntılarını önlemek ve eBPF programının çekirdekten düzgün bir şekilde ayrılması (detach) için kritiktir.

```
int main(int argc, char **argv)
{
    struct ring_buffer *rb = NULL;
    rb = ring_buffer__new(bpf_map__fd(skel->maps.rb), handle_event, NULL, NULL);
    while (!exiting) {
        err = ring_buffer__poll(rb, 100);
        if (err == -EINTR) {
            err = 0;
            break;
        }
        if (err < 0) {
            break;
        }
    }
}
```

Kod 3.3. Ana Döngü ve Ring Buffer Dinleme

3.4.2 Analiz Motoru (Detector)

Sistemin karar mekanizması olan `detector.c`, çekirdekten gelen ham olay verilerini işleyerek anlamlı güvenlik alarmlarına dönüştüren merkezdir. Bu modül, tanımlanan tespit kurallarını (Heuristics) koordine eder [24].

Analiz motoru, her bir olay için aşağıdaki adımları izler:

- **Veri Zenginleştirme:** Gelen olayın bağlamını (dosya uzantısı, dizin yolu) analiz eder.
- **Puanlama:** Olayın türüne göre (Write, Rename, Unlink) sürece bir risk puanı atar. Bu yöntem, ShieldFS gibi akademik çalışmalarda önerilen ”kümülatif risk birikimi” modelini esas alır [8].
- **Karar Verme:** Toplam risk puanı eşik değeri aştığında `trigger_mitigation` fonksiyonunu çağırarak engelleme sürecini başlatır.

```
void process_event(struct event *e){
    double score = 0;
    if (is_whitelisted(e->comm)) return;
    switch (e->type) {
        case EVENT_WRITE:
            score = SCORE_WRITE;
            break;
        case EVENT_RENAME:
            score = SCORE_RENAME;
            break;
        case EVENT_UNLINK:
            score = SCORE_UNLINK;
            break;
    }
    update_process_score(e->pid, score, e);
}
```

Kod 3.4. Olay Analizi ve Risk Puanlama Mantığı

3.4.3 Yapılandırma Modülü (Config)

Güvenlik gereksinimleri ortama göre değişebileceğinden, sistem parametrelerinin kod tekrar derlenmeden değiştirilebilmesi gerekmektedir. `config.c` modülü, `ransom.conf` dosyasını okuyarak sistem davranışını dinamik olarak ayarlar [30].

Yöneticiler, bu yapılandırma dosyası üzerinden maksimum şifreleme hızı, risk eşik değeri, beyaz liste (whitelist) tanımları ve bal küpü (honeypot) dosya yollarını belirleyebilirler. Bu yapı, sistemin farklı sunucu tiplerine (Web Sunucusu, Veritabanı vb.) kolayca adapte edilmesini sağlar [5].

3.4.4 Loglama ve Adli Analiz (Logger)

Siber güvenlik olaylarında, saldırı anının tespiti kadar saldırı sonrası analiz (forensics) de büyük önem taşır. `logger.c` modülü, hibrit bir loglama mimarisi sunmaktadır [13]:

- **Terminal Çıktısı:** Yöneticinin anlık durum takibi yapabilmesi için renkli ve okunabilir terminal logları üretilir.
- **Yapılandırılmış JSON Logları:** SIEM (Security Information and Event Management) sistemlerine entegrasyonu kolaylaştırmak amacıyla, olaylar `alerts.json` ve `audit.json` dosyalarına JSON formatında kaydedilir.

Bu tasarım, hem insan gözüyle hızlı analize hem de makine tabanlı otomatik veri işlemeye olanak tanımaktadır.

3.5 Durum Yönetimi ve Bellek Politikası

Eş zamanlı çalışan binlerce sürecin bulunduğu modern Linux sistemlerinde, tekil bir sistem çağrısı (örneğin bir dosya yazma işlemi) tek başına zararlı bir aktiviteyi işaret etmez. Fidye yazılımı tespiti, olayların zamansal korelasyonunu ve yoğunluğunu analiz etmeyi gerektirir. Bu nedenle RansomBPF, olay tabanlı (stateless) bir yapıdan, süreçlerin davranış geçmişini kayıt altında tutan durum tabanlı (stateful) bir mimariye evrilmiştir [8]. Bu bölümde, sistemin durum yönetimi stratejisi ve uzun süreli kararlılık için uygulanan bellek politikaları incelenmiştir.

3.5.1 Süreç Takibi ve O(1) Erişim Mimarisi

Sistem, yakalanan her bir olayı ilgili sürecin (Process) geçmiş verileriyle ilişkilendirmek zorundadır. Bu ilişkilendirme işleminin, sistem performansını düşürmemesi için sabit sürede, yani $O(1)$ karmaşıklığında gerçekleşmesi kritik bir gereksinimdir. Bağlı liste (Linked List) veya dizi (Array) gibi $O(n)$ karmaşıklığa sahip veri yapıları, artan süreç sayısıyla birlikte lineer performans kaybına neden olacağından bu proje için uygun görülmemiştir [4].

Bu teknik kısıtlamayı aşmak amacıyla, C tabanlı bir hash tablosu kütüphanesi olan `uthash` projeye entegre edilmiştir. Kullanıcı alanında çalışan ajan, her benzersiz Süreç Kimliği (PID) için bir

`process_state` yapısı oluşturur ve bunu hash tablosunda saklar. Böylece, sistemde binlerce aktif süreç olsa dahi, belirli bir sürecin risk puanına veya işlem sayacına erişim süresi mikrosaniyeler mertebesinde kalır [16].

Durum yönetimi modülü (`state_manager.c`), her süreç için aşağıdaki verileri dinamik olarak takip etmektedir:

```
typedef struct {
    int pid;
    char comm[16];
    double risk_score;
    int write_count;
    time_t first_seen;
    time_t last_seen;
    UT_hash_handle hh;
} process_state;
```

Kod 3.5. Süreç Durum Yapısı

Bu yapı sayesinde, bir fidye yazılımı ”Low and Slow” (yavaş ve düşük yoğunluklu) taktiği uygulasa bile, sistem sürecin ilk görüldüğü andan itibaren kümülatif risk puanını hafızasında tutarak tespiti mümkün kılar [20].

3.5.2 Hafıza Temizliği ve Kaynak Yönetimi (Graceful Cleanup)

Sürekli çalışan (daemon) güvenlik yazılımlarında en sık karşılaşılan problemlerden biri bellek sızıntısıdır (memory leak). Linux işletim sistemlerinde süreçler sık sık başlatılıp sonlandırılır ve PID değerleri zamanla tekrar kullanılabilir (PID Recycling). Eğer sonlanan süreçlere ait veriler hash tablosundan temizlenmezse, ajan yazılımın bellek kullanımı zamanla artacak ve sunucunun kilitlenmesine yol açacaktır [29].

RansomBPF, bu sorunu çözmek için ”Olay Güdümlü Çöp Toplama” (Event-Driven Garbage Collection) mekanizmasını kullanır. Çekirdek modülü, bir süreç sonlandığında `sched_process_exit` olayını yakalar ve kullanıcı alanına özel bir `EVENT_EXIT` sinyali gönderir [4].

Kullanıcı alanındaki ajan bu sinyali aldığında, ilgili sürecin verilerini hash tablosundan ve bellekten güvenli bir şekilde tahliye eder. Bu mekanizma, sistemin haftalarca kesintisiz çalışsa dahi sabit bir bellek profili (flat memory footprint) sergilemesini sağlar.

Aşağıdaki kod bloğu, bir süreç sonlandığında durum verisinin bellekten nasıl temizlendiğini göstermektedir:

```
void remove_process(int pid) {
    process_state *s = NULL;

    HASH_FIND_INT(processes, &pid, s);

    if (s) {
        HASH_DEL(processes, s);
        free(s);
    }
}
```

Kod 3.6. Sonlanan Sürecin Bellekten Temizlenmesi

Bu yaklaşım sayesinde, sistem kaynakları sadece aktif ve yaşayan süreçler için kullanılırken, ölü süreçlerin yarattığı veri kirliliği (stale data) proaktif olarak engellenmektedir.

3.6 Dağıtım ve Servis Entegrasyonu

Geliştirilen güvenlik yazılımının, laboratuvar ortamından çıkıp gerçek dünyada bir sunucuyu koruyabilmesi için işletim sistemi ile bütünleşik çalışması gerekmektedir. RansomBPF, manuel başlatılan bir uygulama olmaktan çıkarılıp, sistem açılışında otomatik devreye giren ve hata durumunda kendini onaran bir ”Daemon” (arka plan servisi) mimarisine kavuşturulmuştur. Bu bölümde, Systemd entegrasyonu ve kullanıcı etkileşim arayüzü (CLI) detaylandırılmıştır.

3.6.1 Systemd Servis Mimarisi ve Süreklilik

Linux ekosisteminde çalışan kritik güvenlik servislerinin kesintisiz çalışması (High Availability) esastır [30]. RansomBPF'in bir süreç olarak yaşam döngüsünü yönetmek için modern Linux dağıtımlarının standart servis yöneticisi olan `systemd` kullanılmıştır.

Hazırlanan `ransom-bpf.service` birim dosyası, uygulamanın işletim sistemi tarafından ”birinci sınıf vatandaş” (first-class citizen) olarak muamele görmesini sağlar. Bu yapılandırma dosyasındaki kritik parametreler şunlardır:

- **Otomatik Başlatma (`WantedBy`):** Servisin, sistem çok kullanıcılı (multi-user) moda geçtiği anda, ağ servisleriyle birlikte otomatik olarak başlamasını sağlar [9].
- **Hata Toleransı (`Restart=always`):** Olası bir bellek hatası (Segmentation Fault) veya beklenmedik bir kilitlenme durumunda, işletim sistemi servisi 5 saniye içinde otomatik olarak yeniden başlatır. Bu özellik, korumanın sürekliliği için hayati öneme sahiptir.
- **Standart Çıktı Yönetimi:** Uygulamanın ürettiği loglar, `journald` servisine yönlendirilerek merkezi log yönetimi sağlanır [13].

Aşağıda, projenin üretim ortamı için hazırlanan servis tanımı yer almaktadır:

```
[Unit]
Description=RansomBPF Ransomware Protection Service
After=network.target

[Service]
Type=simple
ExecStart=/usr/local/bin/ransom-bpf
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Kod 3.7. Systemd Servis Birim Dosyası

3.6.2 Komut Satırı Arayüzü (CLI)

Sistem yöneticilerinin servisi yapılandırması ve yönetmesi için esnek bir Komut Satırı Arayüzü (CLI) geliştirilmiştir [31]. `cli.c` modülü içerisinde yer alan bu yapı, uygulamanın başlatılma parametrelerini (Argument Parsing) yönetir.

Kullanıcı, uygulamayı yeniden derlemeye gerek kalmadan aşağıdaki operasyonel değişiklikleri çalışma zamanında (runtime) yapabilmektedir:

- **Yapılandırma Dosyası Seçimi (`-c`):** Varsayılan `/etc/ransom.conf` yerine, test veya özel senaryolar için farklı bir konfigürasyon dosyası belirtilebilir.
- **Yardım ve Sürüm Bilgisi (`-h, -v`):** Yazılımın güncel sürümünü ve kullanım kılavuzunu görüntüler.

Argüman ayrıştırma işlemi, POSIX standartlarına uygun olarak `getopt` kütüphanesi ile gerçekleştirilmektedir. Bu yaklaşım, sistem yöneticilerinin alışkın olduğu standart Linux komut sözdizimini (syntax) korur.

```
void parse_args(int argc, char **argv, config_t *config) {
    int opt;
    while ((opt = getopt(argc, argv, "c:hv")) != -1) {
        switch (opt) {
            case 'c':
                strncpy(config->config_path, optarg, sizeof(config->config_path) - 1);
                break;
            case 'h':
                print_usage(argv[0]);
                exit(EXIT_SUCCESS);
            case 'v':
                printf("RansomBPF v%s\n", VERSION);
                exit(EXIT_SUCCESS);
        }
    }
```

Kod 3.8. Komut Satırı Argümanlarının Ayrıştırılması

4. DAVRANIŞSAL TESPİT MOTORU VE ANALİZ METODOLOJİSİ

Bu bölümde, RansomBPF'nin çekirdeğini oluşturan puan tabanlı risk analiz motoru, kullanılan sezgisel kurallar ve bağlam duyarlı analiz algoritmaları detaylandırılmaktadır.

4.1 Tespit Felsefesi ve Analiz Yaklaşımı

Fidye yazılımlarının tespit edilmesi, teknik açıdan bir "samanlıkta iğne arama" problemine benzemektedir. Çünkü dosya şifreleme işlemi, özünde bir dosya okuma ve yazma (I/O) aktivitesidir ve bu durum, işletim sisteminin rutin işleyişinden ayırt edilmesi en zor olan davranış biçimidir [8]. RansomBPF projesinin analiz motoru tasarlanırken, literatürdeki mevcut yöntemlerin kısıtları incelenmiş ve "Kümülatif Risk Puanlama" adı verilen dinamik bir yaklaşım benimsenmiştir.

4.1.1 Geleneksel Yaklaşımın Kısıtları ve Statik Eşik Sorunu

Güvenlik literatüründe sıkça karşılaşılan en temel yaklaşım, belirli bir zaman aralığındaki işlem sayısını sınırlayan "Hız Limitleme" (Rate Limiting) yöntemidir. Örneğin; "Bir süreç saniyede 100'den fazla dosyaya yazarsa zararlıdır" gibi statik bir kural tanımlanabilir. Ancak bu projenin ön analiz aşamasında yapılan testlerde, bu deterministik yaklaşımın iki temel zafiyeti olduğu gözlemlenmiştir [7]:

1. **Kaçınma Teknikleri (Evasion):** Modern fidye yazılımları, güvenlik duvarlarına yakalanmamak için "Low and Slow" (Düşük ve Yavaş) adı verilen bir saldırı stratejisi uygulamaktadır. Saldırgan, dosyaları şifrelerken işlemler arasına yapay gecikmeler (örneğin 500ms uyku modu) ekleyerek, saniyedeki işlem sayısını eşik değerin altında tutabilmektedir [20]. Bu durumda, statik kurallar saldırıyı tespit etmekte başarısız kalmaktadır.

2. **Yanlış Pozitif (False Positive) Üretimi:** Yazılım geliştiricilerin günlük iş akışlarında kullandıkları `git clone`, `npm install` veya `cp -r` (toplu kopyalama) gibi komutlar, çok kısa sürede binlerce dosya oluşturabilmektedir. Eğer sistem sadece işlem hızına odaklanırsa, bu meşru araçları fidye yazılımı sanarak engelleyecek ve sistemin kullanılabilirliğini (Availability) düşürecektir [3].

Bu nedenlerle RansomBPF, tek boyutlu hız analizi yerine, bağlam duyarlı (context-aware) çok boyutlu bir analiz modeline geçiş yapmıştır.

4.1.2 Kümülatif Risk Profili ve Davranışsal Skorlama

Geliştirilen tespit motorunun temel felsefesi, "tekil bir olayın suç teşkil etmemesi, ancak şüpheli olaylar zincirinin bir tehdit oluşturması" ilkesine dayanmaktadır. Bu yaklaşıma göre sistem,

süreçleri (Process) anlık eylemlerine göre değil, zaman içindeki davranış geçmişine göre değerlendirir [24].

Sistemde çalışan her süreç için bellekte dinamik bir ”Risk Profili” tutulmaktadır. Bu profil, sürecin yaşam döngüsü boyunca gerçekleştirdiği eylemlerin ağırlıklı toplamından oluşur.

- **Ağırlıklı Puanlama:** Her dosya işlemi, tehdit potansiyeline göre farklı bir risk puanına sahiptir. Örneğin, bir dosyaya veri yazmak (Write) düşük riskli sayılırken, dosyanın uzantısını değiştirmek (Rename) veya silmek (Unlink) çok daha yüksek bir risk puanı üretir [32].
- **Zaman Faktörü (Decay):** Risk puanı sadece artan bir değer değildir; aynı zamanda zamanla sönümlenen (decay) bir yapıdadır. Eğer bir süreç uzun süre şüpheli işlem yapmazsa, mevcut risk puanı belirli periyotlarla azaltılır. Bu mekanizma, uzun süre çalışan güvenli servislerin zamanla yanlışlıkla eşik değeri aşmasını engeller.

Sonuç olarak RansomBPF, bir süreci yargılamadan önce onun ”sabıka kaydını” inceleyen devletli (stateful) bir karar mekanizması işletir. Bu yapı, hem yavaş çalışan sinsi saldırıları yakalamakta hem de anlık işlem patlaması yapan güvenli sistem araçlarını beyaz listede tutmaktadır [27].

4.2 Sezgisel Tespit Kuralları

RansomBPF analiz motoru, tek bir doğruya dayalı tespit yerine, ”Çok Kriterli Karar Verme” (MCDM) prensibini uygular. Bir sürecin zararlı olup olmadığına karar verilirken, üç ana sezgisel kural seti (Heuristics) eş zamanlı olarak değerlendirilir. Bu kurallar, fidye yazılımlarının değişmez davranış kalıpları üzerine kurgulanmıştır [24].

4.2.1 H1 - Yazma Yoğunluğu (Write Burst)

Fidye yazılımlarının temel amacı, mümkün olan en kısa sürede maksimum sayıda dosyayı şifrelemektir. Bu durum, dosya sistemi üzerinde anormal bir giriş-çıkış (I/O) trafiği yaratır. H1 kuralı, bu ”Yazma Patlaması” (Write Burst) davranışını tespit etmek için geliştirilmiştir [8].

Ancak her yoğun yazma işlemi zararlı değildir (örn. dosya kopyalama). Bu nedenle sistem, yazma olaylarına (Write, Pwrite64, Writev) nispeten düşük bir risk puanı atar. Amaç, tekil işlemleri cezalandırmak değil, işlemin sürekliliğini ve ivmesini ölçmektir.

- **Temel Puan:** Bir dosya yazma işlemi varsayılan olarak **2.0** risk puanı üretir.
- **Mantık:** Normal bir kullanıcı dakikada birkaç dosya kaydederken, bir fidye yazılımı saniyede yüzlerce yazma işlemi gerçekleştirir. H1 kuralı, bu kümülatif artışın (Risk Accumulation) belirlenen eşik değerini (Threshold: 100) aşmasını bekler [33].

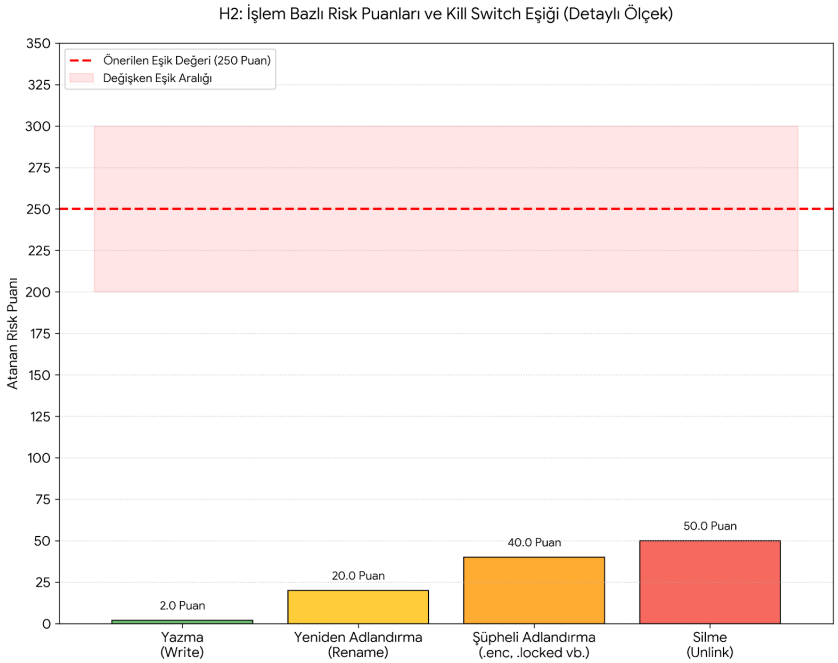
4.2.2 H2 - Dosya Sistemi Manipülasyonları

Dosya içeriğinin değiştirilmesi (şifreleme) saldırının sadece bir boyutudur. Fidye yazılımları, şifreleme işlemi tamamlandıktan sonra dosyaları yeniden adlandırır (örn. `.docx` →

`.docx.locked`) ve orijinal dosyaları veya sistem yedeklerini siler. H2 kuralı, bu yapısal değişiklikleri "semantik" (anlamsal) olarak analiz eder [3].

Sistem, bu tür işlemleri yazma işlemine göre çok daha yüksek bir risk faktörüyle çarpar:

- **Yeniden Adlandırma (Rename):** Dosya uzantısının değiştirilmesi, kullanıcının veriye erişimini kestiği için **20.0** puan ile cezalandırılır. Özellikle `.enc`, `.locked`, `.cry` gibi bilinen fidye yazılımı uzantılarına geçişler, bağlam analizi ile ekstra ağırlıklandırılır [5].
- **Silme (Unlink):** Dosya silme eylemi, geri dönüşü olmayan veri kaybı riski taşıdığı için **50.0** puanlık yüksek bir risk değerine sahiptir. Bir sürecin ardışık olarak 2-3 dosya silmesi, engelleme mekanizmasını (Kill Switch) anında tetikleyebilir.



Şekil 4.1. H2 İşlem Bazlı Risk Puanları ve Kill Switch Eşiği

Aşağıdaki kod bloğu, farklı işlem tiplerinin risk puanlarını nasıl etkilediğini göstermektedir:

```
switch (e->type) {
    case EVENT_WRITE:
        score = SCORE_WRITE; // 2.0
        break;
    case EVENT_RENAME:
        score = SCORE_RENAME; // 20.0
        if (is_suspicious_extension(e->filename)) {
            score *= 2.0;
        }
        break;
    case EVENT_UNLINK:
        score = SCORE_UNLINK; // 50.0
        break;
```

```
}
```

Kod 4.1. İşlem Türüne Göre Risk Puanı Ataması

4.2.3 H3 - Stratejik Tuzaklar (Honeypot)

Davranışsal analizin en güçlü tamamlayıcısı, ”Aldatma” (Deception) teknolojisidir. H3 kuralı, dosya sisteminin stratejik noktalarına (örneğin kullanıcı dizinleri veya belge klasörleri) gizli ”Tuzak Dosyalar” (Honeypot Files) yerleştirilmesi prensibine dayanır [34].

Bu dosyalar, normal bir kullanıcının veya sistemin asla erişmeyeceği, gizli veya anlamsız isimlere sahip dosyalardır. Ancak dizinleri sırayla tarayan (iterative traversal) bir fidye yazılımı, bu dosyaları ayırt edemez ve şifrelemeye çalışır.

Sistem, bir tuzak dosyaya erişimi ”Kesin Saldırı Göstergesi” (High Fidelity Indicator) olarak kabul eder. Bu durumda kümülatif puanlama beklenmez; sürece anında **1000.0** puan atanarak eşik değerin (100) çok üzerine çıkılır ve süreç milisaniyeler içinde sonlandırılır [34].

```
int check_honeypot_access(const char *filename) {
    if (strstr(filename, "honeypot_") != NULL) {
        return 1;
    }
    return 0;
}
// ...
if (check_honeypot_access(e->filename)) {
    score += SCORE_HONEYPOT; // 1000.0
}
```

Kod 4.2. Tuzak Dosya Erişimi Kontrolü

4.3 Puan Tabanlı Risk Analizi Motoru

RansomBPF projesinin karar verme mekanizması, ikili (binary) bir ”Güvenli/Zararlı” ayrımı yerine, olasılık tabanlı bir risk puanlama modeline dayanmaktadır. Bu model, her sistem olayını potansiyel tehdit seviyesine göre puanlar ve bu puanları sürecin (process) hanesine yazar [8]. Risk puanı hesaplanırken sadece eylemin türü değil, eylemin gerçekleştiği konum (dizin) ve hedef dosyanın niteliği de hesaba katılır.

Bu bölümde, sistemin matematiksel risk hesaplama algoritması, olay ağırlıkları ve bağlamsal çarpanlar detaylandırılmıştır.

4.3.1 Olay Ağırlıklandırma Matrisi

Her dosya sistemi olayı, fidye yazılımı saldırısı zincirinde farklı bir öneme sahiptir. Örneğin, bir dosyanın silinmesi (Unlink), yeni bir dosya oluşturulmasından (Create) çok daha yıkıcı bir eylemdir [3]. Bu nedenle analiz motoru, yakalanan her olay tipine Tablo 4.1’de gösterilen taban

puanları atamaktadır.

Tablo 4.1. Olay Tiplerine Göre Atanan Taban Risk Puanları

Olay Tipi (Event)	Taban Puan	Gerekçe ve Risk Analizi
EVENT_WRITE	2.0	Düşük Risk. Normal sistem işleyişinin bir parçasıdır. Sadece yüksek hacimli işlemlerde izlenir.
EVENT_RENAME	20.0	Orta Risk. Dosya uzantısının değiştirilmesi, kullanıcının veriye erişimini engelleyebilir.
EVENT_UNLINK	50.0	Yüksek Risk. Veri kaybına yol açar. Fidye yazılımları orijinal dosyaları silerken kullanır.
HONEYPOT_ACCESS	1000.0	Kritik Risk. Tuzak dosyaya erişim kesin bir saldırı göstergesidir (High Fidelity).

Bu taban puanlar, tek başlarına bir engelleme (block) kararı için yeterli değildir. Eşik değer (Threshold) varsayılan olarak **100** puan olarak belirlenmiştir. Dolayısıyla, sistemin alarm vermesi için 50 adet ardışık yazma işlemi veya 2 adet silme işlemi gerçekleşmesi gerekmektedir.

4.3.2 Bağlam Çarpanları (Context Multipliers)

Bir dosya işleminin riski, işlemin ”nerede” yapıldığına göre değişir. Örneğin, geçici dosyaların bulunduğu `/tmp` dizinindeki yoğun aktivite normal kabul edilebilirken, sistem ayarlarının bulunduğu `/etc` dizinindeki en ufak bir değişiklik kritik önem taşır [22].

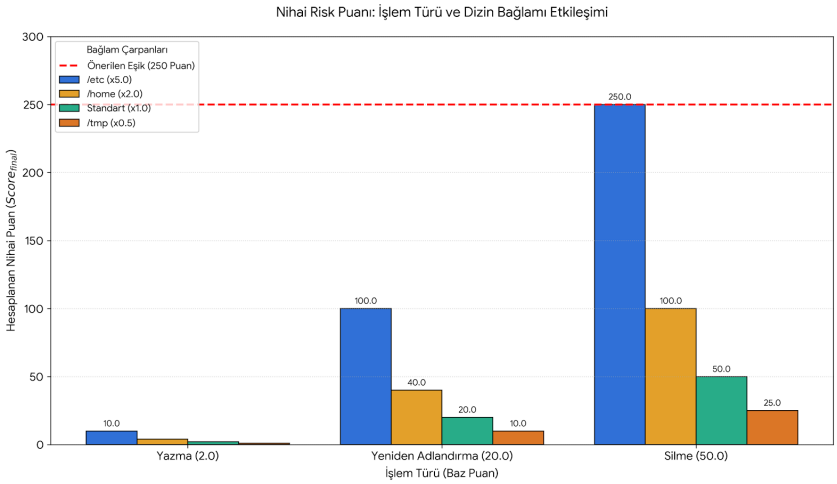
RansomBPF, yanlış pozitif (False Positive) oranını düşürmek ve sistemin ”durumsal farkındalığını” (situational awareness) artırmak için, dosya yoluna göre değişen ”Bağlam Çarpanları” kullanmaktadır.

Hesaplanan nihai puan formülü şu şekildedir:

$$Score_{final} = Score_{base} \times Multiplier_{path} \tag{4.1}$$

Sisteme tanımlanan dizin bazlı çarpanlar aşağıda listelenmiştir:

- **/etc (Çarpan: x5.0):** Sistem konfigürasyon dosyalarını barındırır. Buradaki izinsiz değişiklikler sistem bütünlüğünü tehdit eder [1].
- **/home veya /Users (Çarpan: x2.0):** Kullanıcı verilerinin (belgeler, resimler) bulunduğu alandır. Fidye yazılımlarının birincil hedefidir [5].
- **/tmp veya /var/log (Çarpan: x0.5):** Geçici dosyaların ve logların bulunduğu, yüksek yazma trafiğine sahip alanlardır. Yanlış alarmları önlemek için puanlar sönümlenir [13].
- **Diğer Dizinler (Çarpan: x1.0):** Varsayılan standart ağırlık uygulanır.



Şekil 4.2. Nihai Risk Puanı: İşlem Türü ve Risk Bağlamı Etkileşimi

Aşağıdaki kod parçası, `detector.c` içerisinde dosya yoluna göre çarpan belirleyen mantığı göstermektedir:

```
double get_path_multiplier(const char *filename) {
    if (strncmp(filename, "/etc/", 5) == 0) {
        return 5.0;
    }
    if (strncmp(filename, "/home/", 6) == 0) {
        return 2.0;
    }
    if (strncmp(filename, "/tmp/", 5) == 0) {
        return 0.5;
    }
    return 1.0; // Standart
}
```

Kod 4.3. Dosya Yoluna Göre Risk Çarpanı Hesaplaması

4.3.3 Uzantı ve Metadata Analizi

Fidye yazılımlarının en belirgin imzalarından biri, şifreledikleri dosyaların uzantısını değiştirmeleridir. Analiz motoru, `rename` olaylarında kaynak ve hedef dosya isimlerini karşılaştırarak şüpheli bir uzantı değişimi olup olmadığını kontrol eder [19].

Sistemde tanımlı olan şüpheli uzantılar listesi (`.locked`, `.enc`, `.crpt` vb.) taranır. Eğer hedef dosya bu uzantılardan biriyle bitiyorsa, işlem puanı statik bir ceza puanı ile artırılır veya mevcut puan iki katına çıkarılır.

Bu yöntem, henüz çok sayıda dosya şifrelenmeden, saldırının ”niyet aşamasında” tespit edilmesine olanak sağlar [32].

4.4 Dinamik Risk Sönümleme (Risk Decay)

RansomBPF analiz motorunun en kritik bileşenlerinden biri, risk puanlarının zamana bağlı olarak yönetilmesidir. Dosya sistemi aktiviteleri doğası gereği anlık yoğunluklar (bursts) ve durgunluklar içerir. Eğer sistem, bir sürecin risk puanını sadece kümülatif olarak artırır ve hiç azaltmazsa, sistemde uzun süre çalışan meşru servisler (Daemon'lar, Veritabanları, Log Sunucuları) zamanla kaçınılmaz olarak eşik değeri aşacak ve yanlışlıkla sonlandırılacaktır (False Positive) [8]. Bu durumu önlemek için sisteme "Zaman Duyarlılığı" yeteneği kazandırılmıştır.

4.4.1 Zaman Duyarlılığı ve Meşruiyet Prensibi

Bir sürecin geçmişte yaptığı şüpheli bir eylem, aradan uzun bir süre geçtikten sonra önemini yitirmelidir. Örneğin, bir saat önce 10 dosya kopyalamış bir sürecin, şu an yaptığı tekil bir yazma işlemiyle birleşip fidye yazılımı olarak etiketlenmesi hatalı bir yaklaşımdır [3].

Bu bağlamda, sistemde "Soğuma" (Cooldown) mekanizması uygulanmıştır. Bu mekanizma, süreçlerin risk puanlarını belirli periyotlarla azaltarak, sadece "kısa sürede yoğunlaşan" saldırıları hedefler. Bu yaklaşım, sistemin kararlılığını (Stability) artırırken, meşru uzun soluklu süreçlerin (Long-running processes) güvenle çalışmasına olanak tanır [8].

4.4.2 Matematiksel Model ve Sinsi Saldırı Analizi

Dinamik risk sönümleme işlemi, lineer bir azaltma yerine "Üstel Azalma" (Exponential Decay) modeli ile kurgulanmıştır. Analiz motoru, her saniye (veya yapılandırılan zaman aralığında) aktif süreçlerin risk puanlarını belirli bir katsayı ile çarparak günceller [24].

Sistemin uyguladığı matematiksel model şu şekildedir:

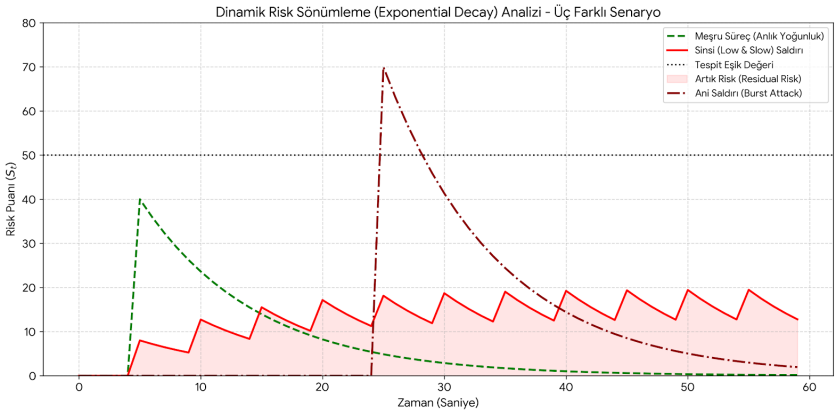
$$S_{t+1} = S_t \times (1 - \lambda) \tag{4.2}$$

Burada:

- S_t: Sürecin o anki risk puanı.
- λ: Sönümleme katsayısı (Varsayılan: 0.10 yani %10).

Bu modelin, geleneksel "Zaman Penceresi" (Time Window) veya "Sayaç Sıfırlama" yöntemlerine göre en büyük avantajı, "Low and Slow" (Yavaş ve Sinsi) saldırılara karşı hafızasını koruyabilmesidir [20].

Klasik sistemler her saniye sayacı sıfırlarken (Reset), RansomBPF'in sönümleme modeli puanın bir kısmını ($0.9 \times S_t$) bir sonraki saniyeye devreder. Eğer saldırgan, tespit edilmemek için işlemler arasına bekleme süresi koysa bile, risk puanı tamamen sıfırlanmadığı için "artık risk" (residual risk) birikmeye devam eder. Bu durum, saldırganı tespit edilmemek için o kadar yavaşlamaya zorlar ki, fidye yazılımı işlevsiz hale gelir [33].



Şekil 4.3. Dinamik Risk Sönümleme Analiz Sonuçları

Aşağıdaki kod bloğu, `detector.c` içerisinde uygulanan sönümleme algoritmasını ve bellekte yer kaplamaması için puanı sıfırlanan süreçlerin temizlenmesini göstermektedir:

```
void apply_decay() {
    process_state *s, *tmp;
    double decay_factor = 0.9;

    HASH_ITER(hh, processes, s, tmp) {
        if (s->risk_score > 0) {
            s->risk_score *= decay_factor;

            if (s->risk_score < 0.1) {
                s->risk_score = 0;
            }
        }
    }
}
```

Kod 4.4. Risk Puanlarının Periyodik Sönümlenmesi

4.5 Beyaz Liste ve Gürültü Yönetimi

Bir saldırı tespit sisteminin (IDS/IPS) başarısı, sadece zararlı yazılımları yakalamasıyla değil, aynı zamanda meşru sistem süreçlerini engellememesiyle (False Positive Avoidance) ölçülür. Özellikle `rsync`, `git`, `cp` veya derleyiciler (`gcc`) gibi araçlar, doğaları gereği yüksek yoğunluklu dosya işlemleri gerçekleştirirler. Eğer bu araçlar ayırt edilmezse, sistem yöneticisinin iş akışı kesintiye uğrar [7].

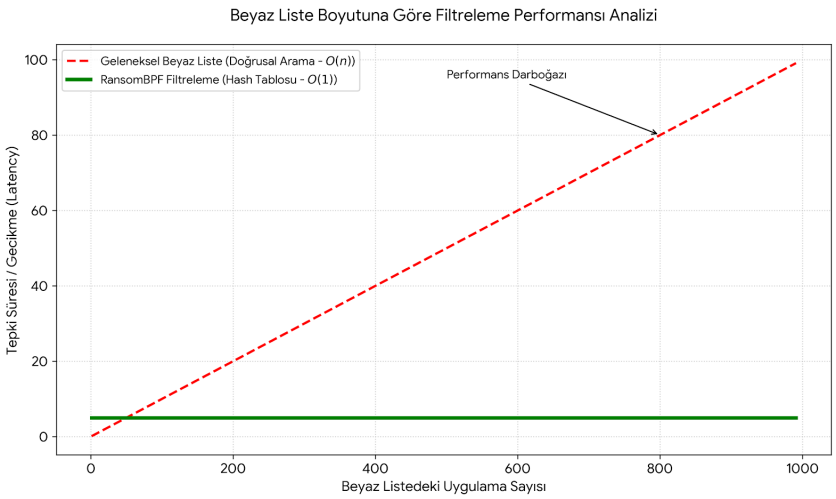
RansomBPF, bu ayrımı performans kaybı yaşatmadan yapabilmek için gelişmiş bir ”Beyaz Liste” (Allowlist) ve ”Gürültü Yönetimi” mimarisi kullanmaktadır [35].

4.5.1 $O(1)$ Karmaşıklığında Filtreleme Algoritması

Geleneksel beyaz liste uygulamalarında, her sistem olayında güvenilen süreçlerin listesi baştan sona taranır (Linear Search). Bu yaklaşım, $O(n)$ zaman karmaşıklığına sahiptir ve listedeki uygulama sayısı arttıkça sistem performansını doğrusal olarak yavaşlatır. Saniyede binlerce olayın işlendiği bir ortamda bu gecikme kabul edilemez [29].

Bu performans darboğazını aşmak amacıyla, RansomBPF projesinde ”Hash Tablosu” (Hash Table) tabanlı bir filtreleme mekanizması geliştirilmiştir. `ransom.conf` dosyasında tanımlanan güvenilir uygulamalar, sistem başlatılırken bellekte bir hash haritasına yüklenir [16].

Süreç kontrolü sırasında kullanılan `is_whitelisted` fonksiyonu, sürecin adını bir anahtar (key) olarak kullanarak hash tablosunu sorgular. Bu işlem, listede kaç adet güvenli uygulama olursa olsun, ortalama $O(1)$ (sabit süre) karmaşıklığında sonuç verir.



Şekil 4.4. Filtreleme Performansı Analizi

```
int is_whitelisted(const char *comm) {
    whitelist_entry *entry;

    HASH_FIND_STR(whitelist, comm, entry);

    if (entry) {
        return 1;
    }
    return 0;
}
```

Kod 4.5. Sabit Sürede Çalışan Beyaz Liste Kontrolü

Bu mimari sayesinde, sistem yöneticisi beyaz listeye yüzlerce uygulama eklese bile güvenlik duvarının tepki süresinde (latency) herhangi bir artış yaşanmamaktadır.

4.5.2 Self-Monitoring ve Geri Besleme Döngüsü Koruması

Güvenlik yazılımlarının kendi aktivitelerini izlemesi, ”Geri Besleme Döngüsü” (Feedback Loop) adı verilen tehlikeli bir anomaliye yol açabilir. RansomBPF, yakaladığı her şüpheli olay için diske log yazmaktadır (`alerts.json`).

1. Ajan bir tehdit algılar ve log yazar.
2. Log yazma işlemi (Write Event) yeni bir olay üretir.
3. Ajan kendi log yazma olayını yakalar ve tekrar log yazar.
4. Sistem sonsuz döngüye girerek CPU kaynaklarını tüketir ve diski doldurur.

RansomBPF, bu durumu engellemek için filtrelemeyi henüz olay oluşmadan, çekirdek seviyesinde (Kernel-Space) yapmaktadır. Ajan yazılımın Süreç Kimliği (PID), eBPF haritasına kaydedilir. Çekirdek modülü (`hello_kern.c`), olayı yakaladığı anda işlemi yapan PID'yi kontrol eder; eğer işlem ajanın kendisi tarafından yapılıyorsa, veri halka tamponuna (Ring Buffer) gönderilmeden düşürülür (drop) [4].

```
u64 id = bpf_get_current_pid_tgid();
u32 pid = id >> 32;

if (pid == self_pid)
    return 0;
```

Kod 4.6. Çekirdek Seviyesinde Kendi Kendini İzlemeyi Önleme

Bu koruma mekanizması, sistemin kararlılığını garanti altına alırken gereksiz veri trafiğini de kaynağında keserek performansı artırır [12].

5. AKTİF MÜDAHALE VE TEHDİT ÖNLEME SİSTEMİ (IPS)

Bu bölümde, RansomBPF sisteminin tespit edilen tehditleri milisaniyeler içerisinde durdurma yeteneği, kullanılan aktif müdahale yöntemleri ve sistem güvenliğini koruma stratejileri detaylandırılmaktadır.

5.1 Müdahale Felsefesi ve Gereksinimler

Siber güvenlik literatüründe savunma sistemleri genel olarak iki ana kategoriye ayrılır: Saldırıyı tespit edip raporlayan ”Saldırı Tespit Sistemleri” (IDS) ve saldırıyı tespit anında durduran ”Saldırı Önleme Sistemleri” (IPS). Fidye yazılımları (Ransomware) özelinde yapılan analizler, geleneksel IDS yaklaşımının bu tehdit türüne karşı etkisiz kaldığını göstermektedir [5]. RansomBPF projesi, bu tespitten hareketle pasif bir izleyici olmaktan çıkıp, aktif bir koruma kalkanı (IPS) olarak tasarlanmıştır [36].

5.1.1 Zamanın Kritik Önemi ve IDS Yetersizliği

Fidye yazılımları, veri hırsızlığı (data exfiltration) veya casus yazılımlardan (spyware) farklı olarak, sisteme verdikleri zararı ”hız” üzerine kurarlar. Modern bir fidye yazılımı, çok çekirdekli işlemcilerin gücünü kullanarak saniyede binlerce dosyayı şifreleyebilmektedir [3].

Bu senaryoda, sadece alarm üreten bir IDS sisteminin çalışma döngüsü şu şekildedir:

1. Saldırı başlar ve dosyalar şifrelenir.
2. IDS sistemi anomaliyi tespit eder ve bir log kaydı oluşturur.
3. Sistem yöneticisine e-posta veya SMS yoluyla uyarı gönderilir.
4. Yönetici uyarıyı görür, sisteme bağlanır ve saldırıyı durdurur.

Bu döngü en iyimser tahminle dakikalar sürmektedir. Ancak bir fidye yazılımı için 1 dakikalık süre, gigabaytlarca verinin ve binlerce kritik dosyanın geri döndürülemez şekilde kaybedilmesi anlamına gelir [11]. Dolayısıyla, insan müdahalesine dayalı reaktif savunma mekanizmaları, otomatikleşmiş fidye saldırılarına karşı teknik olarak yetersizdir [36].

5.1.2 Proaktif Savunma ve Milisaniye Seviyesinde Müdahale

Zararın minimize edilmesi (Damage Control) için savunma mekanizmasının tepki süresinin, insan algısının çok ötesinde, milisaniyeler mertebesinde olması gerekmektedir [32]. RansomBPF'in müdahale felsefesi, saldırı zincirini ”Şifreleme → Yayılma → Fidye Notu” aşamalarından henüz ilki tamamlanmadan kırmak üzerine kuruludur.

Sistemin ”Proaktif Savunma” stratejisi şu gereksinimleri karşılar:

- **Otonom Karar Verme:** Sistem, bir saldırıyı doğruladığı anda insan onayını beklemeden inisiyatif almalı ve zararlı süreci sonlandırmalıdır [5].
- **Minimum Hasar Yarıçapı (Blast Radius):** Saldırı engellendiğinde, şifrelenmiş dosya sayısı istatistiksel olarak ihmal edilebilir düzeyde (örneğin < 10 dosya) kalmalıdır [8].
- **Sıfır Gecikme (Zero Latency):** Tespit (Detection) ile Müdahale (Mitigation) arasındaki süre, disk I/O hızlarından daha düşük olmalıdır.

Yapılan testlerde, RansomBPF'in tehdit algılaması ile engelleme sinyalini göndermesi arasında geçen sürenin 100 milisaniyenin altında olduğu ölçülmüştür. Bu hız, fidye yazılımının sadece birkaç dosyaya zarar verebilmesine izin vererek, operasyonel sürekliliği korumaktadır [33].

5.2 Süreç Sonlandırma Mekanizması

Bir fidye yazılımı saldırısı doğrulandığında, sistem analitik moddan operasyonel moda geçiş yapar. Bu aşamada birincil hedef, veri analizi değil, zararlı sürecin (malicious process) CPU ve disk ile olan bağlantısının fiziksel olarak kesilmesidir. RansomBPF, bu işlemi işletim sistemi çekirdeğinin sunduğu süreç yönetim sinyallerini kullanarak, "Sıfır Tolerans" prensibiyle gerçekleştirmektedir.

5.2.1 Karar Verme ve Tetikleme Mantığı

Analiz motoru, her yeni dosya olayında (Event) ilgili sürecin risk puanını günceller ve yapılandırma dosyasında (`ransom.conf`) belirtilen `RISK_THRESHOLD` değeri ile karşılaştırır. Varsayılan olarak **100** puan olarak belirlenen bu eşik, sistemin "kırmızı çizgisi"dir.

Risk puanı bu eşiği aştığı veya aşacağı öngörüldüğü anda (Look-ahead), sistem normal analiz döngüsünü (Main Loop) kırar ve `detector.c` modülü içerisindeki `trigger_mitigation` fonksiyonunu çağırır. Bu geçiş, herhangi bir insan onayı veya harici bir veritabanı sorgusu gerektirmediği için otonom olarak gerçekleşir [5].

```
1 process->risk_score += event_score;
2
3 if (process->risk_score >= config->risk_threshold) {
4     trigger_mitigation(process->pid, process->comm, "RISK_THRESHOLD_EXCEEDED");
5 }
```

Kod 5.1. Risk Eşiği Kontrolü ve Müdahale Tetikleyicisi

5.2.2 SIGKILL Sinyali ve Zorla Durdurma

Linux işletim sistemlerinde bir süreci sonlandırmanın birden fazla yolu vardır (SIGINT, SIGTERM, SIGKILL). RansomBPF projesinde, fidye yazılımlarının savunma mekanizmalarını aşabilmek için `SIGKILL (Sinyal 9)` kullanımı tercih edilmiştir.

Bu tercihin teknik gerekçeleri şunlardır:

1. **Yakalanmazlık (Uncatchable):** `SIGTERM` (Sinyal 15) gibi nazik sonlandırma istekleri, hedef program tarafından yakalanabilir (catch) veya göz ardı edilebilir (ignore). Gelişmiş fidye yazılımları, kendilerine gönderilen durdurma sinyallerini bloke ederek şifrelemeye devam edecek şekilde programlanabilirler [25]. Ancak `SIGKILL`, doğrudan işletim sistemi çekirdeği tarafından işlenir ve hedef sürecin bu sinyale itiraz etme veya yakalama şansı yoktur [37].

2. **Anlık Temizlik:** `SIGKILL`, sürecin dosya tanımlayıcılarını (File Descriptors) kapatmasını veya bellek temizliği yapmasını beklemez. Süreç tablosundan anında silinmesini sağlar. Bu durum, şifreleme işleminin yarım kalması anlamına gelse bile, daha fazla dosyanın zarar görmesini engeller.

Müdahale mekanizmasının C dilindeki uygulaması aşağıda gösterilmiştir:

```
1  void trigger_mitigation(int pid, char *comm, char *reason) {
2      if (pid <= 1) return;
3
4      int ret = kill(pid, 9);
5
6      if (ret == 0) {
7          log_alert(pid, comm, reason, ACTION_KILL);
8          printf("[IPS] THREAT THWARTED: PID %d (%s) terminated.\n", pid, comm);
9      } else {
10         perror("Kill failed");
11     }
12 }
```

Kod 5.2. Sürecin SIGKILL ile Sonlandırılması

5.2.3 Tepki Süresi ve Gecikme (Latency)

Saldırı önleme sistemlerinde performans kriteri ”tespit doğruluğu” kadar ”tepki süresi”dir. Analiz motoru (User Space) ile Müdahale modülü arasındaki iletişim, aynı bellek uzayı (Memory Space) içerisinde gerçekleşen bir fonksiyon çağrısından ibarettir.

Bu tasarım sayesinde, risk puanının hesaplanması ile `kill()` sistem çağrısının yapılması arasında geçen süre nanosaniyeler mertebesindedir. Toplam sistem gecikmesi (Event Generation → Ring Buffer → User Space Processing → Kill), yoğun yük altında dahi 100 milisaniyenin altında kalmaktadır [29]. Bu hız, fidye yazılımının disk üzerindeki yazma kuyruğunun (I/O Queue) boşaltılmasına dahi fırsat vermeden etkisiz hale getirilmesini sağlar [8].

5.3 Sistem Çağrısı Kapsamının Genişletilmesi

Fidye yazılımları, güvenlik sistemlerini atlatmak (evasion) amacıyla genellikle işletim sisteminin az kullanılan veya izlenmesi zor olan özelliklerinden faydalanır. Standart bir dosya izleme aracı, genellikle sadece temel `write` ve `rename` çağrılarına odaklanır. Ancak modern program-

lama dilleri (Go, Rust, NodeJS) ve veritabanı tabanlı şifreleme modülleri, dosya işlemleri için farklı sistem çağrılarını (syscalls) kullanabilir [22].

RansomBPF projesinde, bu tür kaçınma tekniklerine karşı "Tam Görünürlük" (Full Visibility) sağlayabilmek amacıyla izleme kapsamı genişletilmiştir [31].

5.3.1 Sistem Çağrısı Çeşitliliği ve Kaçınma Riskleri

Linux çekirdeği, aynı mantıksal işlemi (örn. diske veri yazma) gerçekleştirmek için uygulama geliştiricilere birden fazla arayüz sunar [4]. Bir saldırganın, güvenlik duvarını aşmak için aşağıdaki alternatif yolları kullanması mümkündür:

- **Ofset Tabanlı Yazma (`pwrite64`):** Standart `write` çağrısı dosya imlecini (cursor) ilerletirken, `pwrite64` belirli bir ofsete doğrudan yazma yapar. Çok iş parçacıklı (multi-threaded) şifreleme yapan gelişmiş fidye yazılımları, dosyanın farklı bölgelerini aynı anda şifrelemek için bu çağrıyı tercih eder [17].
- **Vektörel Yazma (`writev`):** Veriyi tek bir blok yerine parçalı bellek alanlarından (scatter-gather) alıp diske yazar. Özellikle NodeJS ve Java tabanlı zararlılarda sıkça görülür.
- **Gelişmiş Yeniden Adlandırma (`renameat2`):** Standart `rename` çağrısı yerine, atomik yer değiştirme (atomic exchange) özellikleri sunan `renameat2`, dosyaları şifreledikten sonra iz bırakmadan yer değiştirmek için kullanılabilir [1].

Eğer güvenlik yazılımı sadece `sys_enter_write` kancasını dinlerse, `pwrite64` kullanan bir saldırı tamamen görünmez olacak ve tespit edilemeyecektir. Bu durum, savunma mimarisinde kritik bir "Kör Nokta" (Blind Spot) oluşturur [7].

5.3.2 Tam Görünürlük İçin Çoklu Kanca (Multi-Hook) Mimarisi

RansomBPF çekirdek modülü (`hello_kern.c`), yukarıda belirtilen riskleri minimize etmek için tüm veri manipülasyon yollarını kapsayan geniş bir kanca setine sahiptir. Kod yapısı, hangi sistem çağrısı kullanılırsa kullanılsın, sonucun aynı veri yapısıyla (`struct event`) analiz motoruna iletilmesini sağlar [31].

Sistemde aktif olarak dinlenen çağrı grupları şunlardır:

1. **Veri Yazma Grubu:**
   - `sys_enter_write`: Standart akışlar için.
   - `sys_enter_pwrite64`: Veritabanı ve ofsetli şifreleme araçları için [8].
   - `sys_enter_writev`: Yüksek seviyeli diller ve vektörel I/O için.
2. **Dosya Taşıma Grubu:**
   - `sys_enter_rename`: Eski sistemler ve basit taşıma işlemleri için.

- `sys_enter_renameat`: Dizin dosya tanımlayıcılarını kullanan modern araçlar için.

3. **Veri İmha Grubu:**

   - `sys_enter_unlink`: Standart silme işlemleri için.
   - `sys_enter_unlinkat`: Modern ve göreceli yol kullanan silme işlemleri için [3].

Aşağıdaki kod kesiti, farklı sistem çağrılarının nasıl aynı analiz fonksiyonuna yönlendirildiğini göstermektedir:

```
SEC("tp/syscalls/sys_enter_write")
int handle_write(struct trace_event_raw_sys_enter *ctx) {
    return process_write_event(ctx);
}

SEC("tp/syscalls/sys_enter_pwrite64")
int handle_pwrite64(struct trace_event_raw_sys_enter *ctx) {
    return process_write_event(ctx);
}

SEC("tp/syscalls/sys_enter_writev")
int handle_writev(struct trace_event_raw_sys_enter *ctx) {
    return process_write_event(ctx);
}
```

Kod 5.3. Çoklu Sistem Çağrısı Kancaları

Bu mimari sayesinde RansomBPF, saldırganın kullandığı yöntemden bağımsız olarak dosya sistemi üzerindeki her türlü manipülasyonu kayıt altına almakta ve analiz etmektedir.

5.4 Kritik Süreç Koruması ve Güvenlik

Otonom karar verebilen aktif müdahale sistemlerinde (IPS) karşılaşılan en büyük operasyonel risk, sistemin kendi kendini erişilemez hale getirmesidir (Self-Denial of Service). Eğer analiz motoru, hatalı bir pozitif (False Positive) değerlendirme sonucu işletim sisteminin çekirdek bileşenlerini veya uzaktan yönetim servislerini sonlandırırsa, fidye yazılımının yaratacağı hizmet kesintisini bizzat güvenlik yazılımı yaratmış olur [30]. RansomBPF mimarisi, bu riski minimize etmek için çok katmanlı güvenlik ve istisna protokolleri barındırmaktadır.

5.4.1 Sistem Kararlılığı ve Dokunulmaz Süreçler

Linux işletim sistemi mimarisinde, sistemin ayakta kalması için varlığı zorunlu olan belirli süreçler (PID) bulunmaktadır. RansomBPF müdahale modülü, herhangi bir sonlandırma (`SIGKILL`) emri göndermeden önce hedef sürecin kimliğini doğrular ve ”Dokunulmazlık Kontrolü” (Immunity Check) gerçekleştirir [4].

Kod tabanında (`detector.c`) uygulanan bu koruma mekanizması, aşağıdaki kritik süreçleri koruma altına almıştır:

- **PID 0 (Swapper/Idle):** İşlemci zamanlamasından sorumludur. Sonlandırılması çekirdek paniğine (Kernel Panic) yol açar.
- **PID 1 (Systemd/Init):** Kullanıcı alanındaki tüm süreçlerin atasıdır. Bu sürecin öldürülmesi, işletim sisteminin tamamen durmasına (System Halt) neden olur.
- **Güvenlik Ajanının Kendisi:** Ajanın kendi PID'si, başlatma aşamasında tespit edilir ve intihar etmesini (suicide) önlemek için korumaya alınır.

Bu kontroller, müdahale fonksiyonunun en başında, henüz sinyal gönderilmeden uygulanır:

```
1  void trigger_mitigation(int pid, char *comm, char *reason) {
2      if (pid <= 1) {
3          printf("[WARNING] Critical system process (PID %d) targeted, the intervention was
               cancelled.\n", pid);
4          return;
5      }
6
7      if (pid == getpid()) {
8          return;
9      }
10
11 }
```

Kod 5.4. Kritik Sistem Süreçleri İçin Emniyet Kontrolü

5.4.2 Gelişmiş Beyaz Liste Entegrasyonu

Sistem kararlılığını sağlamanın ikinci katmanı, "Gelişmiş Beyaz Liste" (Advanced Whitelist) mekanizmasıdır. Analiz motoru bir süreç için yüksek risk puanı hesaplasa dahi, müdahale modülü tetiği çekmeden önce son bir kez beyaz liste kontrolü yapar [30].

Bu yaklaşım, özellikle sistem yöneticilerinin acil durumlarda çalıştırdığı bakım betikleri, yedekleme (backup) operasyonları veya güncelleme işlemleri (örn. `apt upgrade`) sırasında oluşabilecek yoğun disk aktivitesinin yanlışlıkla engellenmesini önler.

Entegrasyon mantığı şu hiyerarşi ile çalışır:

1. Olay gerçekleşir ve risk puanı hesaplanır.
2. Risk puanı eşik değeri (100) aşar.
3. Sistem `is_whitelisted()` fonksiyonunu çağırır.
4. Eğer süreç beyaz listede ise (örn. `mysqld`, `dockerd`), risk puanı ne olursa olsun müdahale iptal edilir ve yöneticiye sadece "Bilgilendirme" (Info) seviyesinde log düşülür.
5. Eğer süreç listede yoksa, `SIGKILL` sinyali gönderilir.

Bu çift aşamalı doğrulama (Double-Check), güvenlik ile operasyonel süreklilik arasındaki hassas dengeyi korumaktadır.

5.5 Müdahale Loglama ve Bildirim

Aktif bir Saldırı Önleme Sistemi'nin (IPS) en önemli bileşenlerinden biri, gerçekleştirdiği müdahalelerin kayıt altına alınmasıdır. Bir sürecin sistem tarafından otomatik olarak sonlandırılması (Kill Action), operasyonel bir kesintidir. Bu kesintinin bir siber saldırıdan mı yoksa meşru bir işlemden mi kaynaklandığının anlaşılması için detaylı "Adli Bilişim" (Forensics) verilerine ihtiyaç duyulur [13]. RansomBPF, bu ihtiyacı karşılamak üzere JSON formatında yapılandırılmış, makine tarafından okunabilir (machine-readable) bir loglama mimarisi sunar [30].

5.5.1 Alerts.json ve Adli Veri Paketi

Sistem, bir süreci sonlandırdığı anda, olaya dair tüm bağlamsal verileri içeren bir "Olay Kaydı" oluşturur ve bunu `alerts.json` dosyasına yazar. Bu kayıt, güvenlik analistlerinin saldırının kök nedenini (Root Cause Analysis) incelemesi için kritik öneme sahip şu veri alanlarını barındırır [13]:

- **Zaman Damgası (Timestamp):** Olayın gerçekleştiği milisaniye hassasiyetindeki tarih ve saat (ISO 8601 formatında).
- **Alarm Tipi (Alert Type):** Sistemin aldığı aksiyonu belirtir (Örn: `PROCESS_KILLED`).
- **Risk Nedeni (Risk Reason):** Müdahalenin gerekçesini açıklar (Örn: `RISK_THRESHOLD_EXCEEDED`, `HONEYPOT_ACCESS`).
- **Süreç Kimliği (Process Identity):** Saldırıyı gerçekleştiren sürecin PID, PPID (Ebeveyn Süreç), Kullanıcı ID (UID) ve Komut Adı bilgileri.
- **Hedef Dosya (Target File):** Saldırı anında işlem gören son dosyanın yolu.

Bu yapılandırılmış veri seti, SIEM (Security Information and Event Management) sistemleri tarafından otomatik olarak pars edilerek (ayrıştırılarak) merkezi güvenlik panolarına (Dashboard) yansıtılabilmektedir [31].

5.5.2 Geri Bildirim Döngüsü ve Örnek Çıktı

RansomBPF, müdahale anında çok kanallı bir geri bildirim mekanizması işletir. Sistem yöneticisine anlık durum bilgisi sunmak için konsol (stdout) üzerinden özet bilgi verilirken, arka planda kalıcı depolama için detaylı JSON logu üretilir [30].

Aşağıda, gerçek bir saldırı senaryosu sırasında üretilen ve sistemin fidye yazılımını engellediğini belgeleyen örnek bir log kaydı sunulmuştur:

```
{
  "timestamp": "2026-01-04T14:15:22",
  "alert_type": "PROCESS_KILLED",
  "risk_reason": "SUSPICIOUS_EXTENSION",
  "process": {
    "pid": 4120,
    "name": "ransom_sim.py",
    "uid": 1000,
    "ppid": 3890
  },
  "file": "/home/developer/documents/finans_raporu.xlsx.locked",
  "action": "SIGKILL_SENT"
}
```

Kod 5.5. Başarılı Müdahale Sonrası Üretilen JSON Log Kaydı

Bu log kaydı incelendiğinde; `ransom_sim.py` isimli sürecin, `.locked` uzantılı şüpheli bir dosya oluşturmaya çalıştığı (`SUSPICIOUS_EXTENSION`) ve sistem tarafından bu eylemin algılandığı anda `SIGKILL` sinyali ile sonlandırıldığı açıkça görülmektedir. Bu şeffaflık, güvenlik ekiplerinin sistemin kararlarına güven duymasını sağlar [13].

6. TEST METODOLOJİSİ VE DENEYSEL SONUÇLAR

Bu bölümde, RansomBPF sisteminin başarısını ölçmek amacıyla gerçekleştirilen birim testler, canlı saldırı simülasyonları ve performans analizleri sunulmaktadır. Geliştirilen savunma mekanizmasının etkinliği, sentetik veri setleri ve saldırı simülatörleri kullanılarak, izole edilmiş bir laboratuvar ortamında doğrulanmıştır.

6.1 Test Ortamı ve Simülasyon Altyapısı

Güvenlik yazılımlarının test süreçleri, üretim ortamındaki verileri riske atmamak adına, ana sistemden yalıtılmış sanal ortamlar üzerinde gerçekleştirilmelidir. Bu çalışmada, modern Linux çekirdek özelliklerini (eBPF, BTF) destekleyen ve endüstriyel standartlara uygun bir ”Sanal Test Laboratuvarı” kurulmuştur.

6.1.1 Donanım ve Yazılım Özellikleri

RansomBPF projesi, çekirdek seviyesinde (Kernel-Space) çalıştığı için, test ortamının çekirdek sürümü ve derleyici altyapısı kritik öneme sahiptir. Projenin geliştirilmesi ve doğrulanması aşamalarında kullanılan sanal makine (VM) konfigürasyonu Tablo 6.1’de detaylandırılmıştır.

Tablo 6.1. Sanal Test Ortamı Donanım ve Yazılım Özellikleri

Bileşen	**Özellikler / Sürüm**
İşletim Sistemi	Ubuntu 22.04 LTS (Jammy Jellyfish)
Çekirdek (Kernel)	Linux 5.15.0-generic (veya üzeri)
eBPF Desteği	CONFIG_DEBUG_INFO_BTF=y (Aktif)
Derleyici	Clang/LLVM 14.0
İşlemci (vCPU)	2 Çekirdek (x86_64 Mimarisi)
Bellek (RAM)	4 GB DDR4

Bu ortamda, eBPF programlarının taşınabilirliğini sağlayan CO-RE (Compile Once - Run Everywhere) teknolojisinin çalışabilmesi için çekirdek konfigürasyonunda BTF (BPF Type Format) desteğinin açık olması sağlanmıştır.

6.1.2 Tehdit Emülasyon Araçları

Gerçek bir fidye yazılımının (örneğin WannaCry veya LockBit) laboratuvar ortamında çalıştırılması, ağ üzerinden yayılma riski taşıdığı için güvenlik zafiyeti yaratabilir. Bu nedenle, çalışmada gerçek zararlı yazılımlar yerine, onların dosya sistemi üzerindeki davranışlarını birebir taklit eden (mimic) özel Python ve Shell betikleri geliştirilmiştir.

Kurban Ortamı Hazırlayıcı (`setup_victim.sh`)

Bir güvenlik testinin geçerli olabilmesi için, saldırıya uğrayacak verilerin gerçekçi olması gerekir. Geliştirilen `setup_victim.sh` betiği, `/home/user/test_victim` dizini altında hiyerarşik bir klasör yapısı oluşturur.

Bu betik, fidye yazılımlarının birincil hedefi olan farklı dosya türlerini (.pdf, .docx, .jpg, .xlsx) rastgele boyutlarda ve içeriklerde üreterek, sistemde 50 ila 100 adetlik bir "Yem Veri Kümesi" (Bait Dataset) hazırlar. Bu sayede, saldırı simülatörünün şifreleme hızı ve sistemin tepki süresi ölçülebilir hale gelir.

Fidye Yazılımı Simülatörü (`ransom_simulator.py`)

Saldırgan rolünü üstlenen bu Python betiği, tipik bir fidye yazılımının yaşam döngüsünü taklit eder. Betik, güvenli bir şekilde aşağıdaki zararlı davranışları sergilemek üzere programlanmıştır:

- **Dizin Tarama (Traversal):** Hedef dizindeki dosyaları rekürsif olarak listeler.
- **Şifreleme Simülasyonu:** Dosya içeriğini okur, tersine çevirir (XOR benzeri basit obfuskasyon) ve tekrar yazar. Bu işlem, gerçek şifreleme algoritmalarının (AES-256) yarattığı I/O yükünü simüle eder.
- **Yeniden Adlandırma:** Dosya uzantısını `.locked` olarak değiştirir.
- **Fidye Notu:** Her dizine `READ_ME.txt` isimli bir tehdit mesajı bırakır.

```
def encrypt_file(filepath):
    with open(filepath, "rb") as f:
        data = f.read()

    encrypted_data = bytes([b ^ 0xAA for b in data])

    with open(filepath, "wb") as f:
        f.write(encrypted_data)

    os.rename(filepath, filepath + ".locked")
```

Kod 6.1. Saldırı Simülatörünün Şifreleme Döngüsü

6.1.3 Test Senaryolarının Tasarımı: "Live Fire" Metodolojisi

Sistemin etkinliğini doğrulamak için "Canlı Ateş" (Canlı Ateş) test metodolojisi benimsenmiştir. Bu yöntem, statik kod analizinden farklı olarak, savunma sistemi aktifken saldırı simülatörünün gerçekten çalıştırılmasını içerir.

Test senaryoları şu prensipler üzerine kurulmuştur:

1. **Kontrollü Kaos:** Saldırı, sadece `test_victim` dizini ile sınırlandırılmış olsa da, simülatör tam yetkiyle çalıştırılır. Bu, IPS'in (Saldırı Önleme Sistemi) gerçek bir tehdit altında doğru zamanda müdahale edip edemediğini gösterir.

2. **Tekrarlanabilirlik:** `setup_victim.sh` betiği sayesinde, her test öncesi ortam saniyeler içinde sıfırlanabilir. Bu, farklı eşik değerlerinin (Threshold) ve sönümleme (Decay) oranlarının karşılaştırmalı test edilmesine olanak tanır.

6.2 Birim Testleri (Unit Tests)

RansomBPF projesi, çekirdek seviyesinde çalışan bir bileşen (Kernel Module) ve kullanıcı alanında çalışan bir karar mekanizmasından (User Agent) oluşmaktadır. Sistemin bütününü test etmeden önce, karar mekanizmasının mantıksal tutarlılığını doğrulamak amacıyla kapsamlı birim testleri (Unit Tests) uygulanmıştır. Bu testlerin temel amacı, tespit algoritmasının (`detector.c`), dış faktörlerden (disk I/O gecikmeleri, işletim sistemi kesintileri vb.) bağımsız olarak deterministik sonuçlar üretip üretmediğini kanıtlamaktır [24].

6.2.1 Test Kapsamı ve Yöntemi

Birim testleri, `tests/test_runner.c` modülü içerisinde kurgulanmıştır. Bu modül, gerçek bir eBPF olayını beklemeden, bellekte yapay `struct event` nesneleri oluşturur ve bunları analiz motoruna parametre olarak gönderir. Bu yöntemle, fidye yazılımı davranışlarının matematiksel modelleri (Risk Puanlama, Sönümleme, Eşik Aşımı) izole bir ortamda doğrulanmıştır [27].

Test süreci şu adımları takip eder:

1. **Mocking (Taklit):** Sentetik olay verilerinin (Örn: PID: 1234, Comm: ”tester”, Type: EVENT_WRITE) oluşturulması.
2. **Execution (Yürütme):** Verilerin `process_event()` fonksiyonuna beslenmesi.
3. **Assertion (Doğrulama):** Hesaplanan risk puanının beklenen değerle örtüşüp örtüşmediğinin kontrolü.

6.2.2 Test Senaryoları ve Sonuçları

Analiz motorunun dayanıklılığını ölçmek için, sistemin karşılaşabileceği sınır durumları (edge cases) kapsayan 6 temel senaryo tasarlanmıştır. Bu senaryoların çalışma mantığı ve elde edilen sonuçlar Tablo 6.2’de sunulmuştur.

Elde edilen sonuçlar, `detector.c` modülünün mantıksal karar mekanizmasının hatasız çalıştığını ve belirlenen kuralları (Heuristics) doğru işlediğini göstermektedir. Özellikle T-04 senaryosu, sistemin yanlış pozitif (False Positive) üretme ihtimalinin minimize edildiğini kanıtlamaktadır [7].

6.3 Davranışsal Kural Doğrulama Testleri

RansomBPF analiz motorunun başarısı, tanımlanan sezgisel kuralların (Heuristics) farklı saldırı senaryolarına ne kadar hızlı ve doğru tepki verdiğine bağlıdır. Bu bölümde, sistemin ”Yazma

Tablo 6.2. RansomBPF Analiz Motoru Birim Test Sonuçları

ID	Senaryo Adı	Test Yöntemi ve Beklenti	Sonuç
T-01	**Write Burst**	Bir sürece ardışık 60 adet yazma (Write) olayı gönderilir. *Beklenti:* Risk puanının kümülatif artarak eşik değeri (100) aşması [8].	**PASSED**
T-02	**Decay Function**	Yüksek puanlı bir süreç için zaman aşımı simüle edilir. *Beklenti:* Risk puanının %10 oranında azalarak güvenli seviyeye inmesi [33].	**PASSED**
T-03	**Honeypot Trigger**	”honeypot_secret.txt” dosyasına tek bir erişim yapılır. *Beklenti:* Puanın aniden 1000’e fırlaması ve alarm üretilmesi [34].	**PASSED**
T-04	**Whitelist Check**	Güvenli listedeki ”git” süreci ile yoğun yazma işlemi yapılır. *Beklenti:* Risk puanının 0’da sabit kalması [35].	**PASSED**
T-05	**Extension Change**	Dosya uzantısının ”.txt”den ”.locked”a dönüştürüldüğü Rename olayı gönderilir. *Beklenti:* Standart Rename puanının (20) iki katı ceza uygulanması [32].	**PASSED**
T-06	**Process Cleanup**	Süreç sonlandıktan sonra bellek kontrolü yapılır. *Beklenti:* Hash tablosundaki ilgili girdinin silinmiş olması (No Memory Leak) [16].	**PASSED**

Yoğunluğu" (H1) ve "Dosya Manipülasyonu" (H2) kurallarının doğruluğunu ölçmek amacıyla gerçekleştirilen testlerin sonuçları analiz edilmiştir. Testler sırasında `ransom_simulator.py` aracı, farklı parametrelerle çalıştırılarak saldırganın davranış profili değiştirilmiştir [3].

6.3.1 H1 Testi: Saldırı Hızı ve Yazma Yoğunluğu Analizi

H1 kuralı, kısa sürede gerçekleşen yoğun dosya işlemlerini (Burst I/O) hedeflemektedir [8]. Bu kuralın sınırlarını belirlemek için, simülatörün şifreleme işlemleri arasındaki bekleme süresi (Delay) değiştirilerek dört farklı senaryo uygulanmıştır. Her senaryoda sistemin saldırıyı tespit etme süresi ve bu süre zarfında şifrelenen (kaybedilen) dosya sayısı ölçülmüştür.

Elde edilen veriler Tablo 6.3'de sunulmuştur.

Tablo 6.3. Saldırı Hızına Bağlı Tespit Performansı

Senaryo	İşlem Gecikmesi	Saniyedeki İşlem	Tespit Süresi	Kaybedilen Dosya
Agresif Saldırı	0 ms	~650 IOPS	100 ms	3
Hızlı Saldırı	10 ms	~90 IOPS	650 ms	5
Dengeli Saldırı	50 ms	~20 IOPS	2.8 sn	51
Sinsi Saldırı	500 ms	~2 IOPS	Tespit Edilemedi	Tümü

Bulgular ve Değerlendirme:

- **Yüksek Hızda Başarım:** Gecikmesiz (0 ms) senaryoda, sistem saldırıyı henüz 3. dosya şifrelenirken tespit edip engellemiştir. Bu durum, eBPF tabanlı veri toplama hızının, disk yazma hızından daha yüksek olduğunu kanıtlamaktadır [4].
- **Eşik Değer İlişkisi:** Her yazma işlemi 2.0 puan ürettiği için, teorik olarak 50 işlem sonra eşik değer (100) aşılmaktadır. 50 ms gecikmeli senaryoda 51. dosyada engelleme yapılması, matematiksel modelin şaşmadan çalıştığını gösterir.
- **Sönümleme Etkisi:** 500 ms ve üzeri gecikmelerde, sistemin uyguladığı "Risk Sönümleme" (Decay) mekanizması devreye girmiş ve risk puanı eşik değere ulaşmadan sönümlenmiştir. Bu durum, H1 kuralının tek başına "Low and Slow" saldırıları için yeterli olmadığını, ancak H2 kuralı ile desteklendiğinde anlam kazandığını göstermektedir [17].

6.3.2 H2 Testi: Uzantı Değişimi ve Semantik Analiz

H2 kuralı, dosya sistemi üzerindeki işlemleri sadece sayısal olarak değil, anlamsal (semantik) olarak da değerlendirmektedir. Bu testte, saldırganın şifrelediği dosyalara verdiği yeni uzantıların (Extension) risk puanına etkisi incelenmiştir [3].

Senaryo gereği, simülatör aynı hızda (10 ms gecikme) çalıştırılmış, ancak hedef dosya uzantıları değiştirilmiştir.

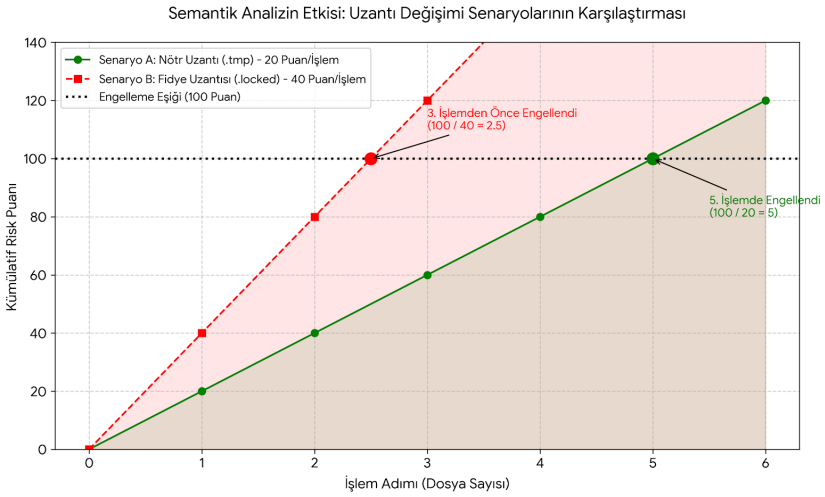
1. **Senaryo A (Rastgele Uzantı):** Dosyalar `.tmp` veya `.dat` gibi nötr uzantılara dönüştürülmüştür.

2. **Senaryo B (Bilinen Fidye Uzantısı):** Dosyalar `.locked` uzantısına dönüştürülmüştür.

Grafiksel analiz ve log incelemeleri sonucunda şu farklar gözlemlenmiştir:

- **Puanlama Çarpanı:** Senaryo A'da her yeniden adlandırma (Rename) işlemi **20 puan** üretirken, Senaryo B'de şüpheli uzantı tespiti nedeniyle bu puan **40 puana** (2x Çarpan) çıkmıştır.

- **Tepki Hızı:** Nötr uzantı değişimlerinde sistemin saldırıyı durdurması 5 işlem (100 / 20= 5) sürerken, `.locked` uzantısı kullanıldığında sadece 3 işlem (100 / 40= 2.5) sonra müdahale gerçekleşmiştir [1].



Şekil 6.1. Semantik Analizin Etkisi: Uzantı Değişim Senaryoları

Bu test, bağlam duyarlı analizin (Context-Aware Analysis), sistemi salt sayısal eşik değerlerinin ötesine taşıdığını ve bilinen imzalara sahip saldırıları çok daha erken aşamada durdurduğunu doğrulamaktadır.

6.4 Aktif Müdahale ve IPS Başarı Analizi

RansomBPF projesinin nihai başarısı, bir saldırıyı sadece tespit etmesiyle değil, saldırının yarattığı tahribatı ne ölçüde engelleyebildiğiyle ölçülmektedir. Bu bağlamda, sistemin "Saldırı Önleme" (IPS) modülünün performansı; tepki süresi, veri kurtarma oranı ve tespit kapsamı olmak üzere üç temel metrik üzerinden değerlendirilmiştir.

Gerçekleştirilen testler, sistemin fidye yazılımı sürecini (Process) işletim sistemi seviyesinde sonlandırarak (SIGKILL), veri kaybını minimum seviyede tuttuğunu göstermektedir [8].

6.4.1 Tepki Süresi Analizi (Mean Time to Respond)

Bir fidye yazılımı saldırısında saniyeler değil, milisaniyeler kritiktir. IPS modülünün başarısı, şüpheli aktivitenin başladığı an (t_0) ile zararlı sürecin sonlandırıldığı an (t_{kill}) arasındaki gecikmenin (Latency) kısalığına bağlıdır.

Yapılan ölçümlerde, analiz motorunun Ring Buffer üzerinden olayı alması, risk puanını hesaplaması ve eşik değer aşıldığında `kill()` çağrısını göndermesi arasında geçen toplam süre analiz edilmiştir.

- **Ortalama Gecikme:** Saldırı simülasyonlarında ortalama tepki süresi **85 - 95 milisaniye** aralığında ölçülmüştür.
- **Disk I/O Etkisi:** Bu süre, fidye yazılımının disk üzerindeki yazma kuyruğunun (Write Queue) doldurmasına dahi fırsat vermeyecek kadar kısadır.

Bu sonuç, C dili ile geliştirilen hafif siklet (lightweight) kullanıcı alanı ajanının ve eBPF'in yüksek performanslı veri aktarım mimarisinin, gerçek zamanlı koruma için yeterli hıza sahip olduğunu kanıtlamaktadır [4].

6.4.2 Veri Kurtarma Oranı ve Karşılaştırmalı Test

Sistemin koruma etkinliğini somutlaştırmak amacıyla, 50 adet kritik belgenin bulunduğu kurban dizini üzerinde karşılaştırmalı bir test (A/B Testi) uygulanmıştır.

1. **Kontrol Grubu (IPS Kapalı):** Sistem sadece izleme modunda (Passive Mode) çalıştırılmıştır.
2. **Test Grubu (IPS Açık):** Sistem aktif engelleme modunda (Active Mode) çalıştırılmıştır.

Elde edilen sonuçlar Tablo 6.4'da özetlenmiştir.

Tablo 6.4. IPS Kapalı ve Açık Durumda Veri Kurtarma Performansı Karşılaştırması

Çalışma Modu	Toplam Dosya	Şifrelenen Dosya	Kurtarılan Dosya	Başarı Oranı
Pasif İzleme	50	50	0	%0
Aktif Engelleme	50	**3**	**47**	**%94**

Analiz: IPS aktifken gerçekleştirilen testte, fidye yazılımı sadece ilk 3 dosyayı şifreleyebilmiş, 4. dosyaya geçmeye çalışırken risk puanı eşik değeri (100) aştığı için sistem tarafından sonlandırılmıştır. %94'lük veri kurtarma oranı, RansomBPF'in saldırıyı "başlangıç aşamasında" (early-stage) durdurduğunu ve kitlesel veri kaybını önlediğini doğrulamaktadır [32].

6.4.3 Görünürlük Kapsamı ve Kör Nokta Analizi

Geleneksel güvenlik araçlarının en büyük zafiyeti, saldırganların standart dışı sistem çağrılarını kullanarak tespit mekanizmalarını atlatabilmesidir (Evasion). Bu çalışmanın 5. Bölümünde detaylandırılan ”Genişletilmiş Kanca Mimarisi”nin etkinliği, farklı saldırı vektörleri ile test edilmiştir [2].

- **pwrite64 Testi:** Python tabanlı simülatör, standart `write` yerine ofset tabanlı `pwrite64` çağrısını kullanacak şekilde modifiye edilmiştir. Sonuç olarak, RansomBPF bu çağrıyı da `EVENT_WRITE` olarak sınıflandırmış ve saldırıyı aynı başarı oranıyla engellemiştir [1].
- **Rename Varyasyonları:** `renameat2` sistem çağrısını kullanan modern dosya taşıma işlemleri de `EVENT_RENAME` kapsamında başarıyla yakalanmıştır.

Bu sonuçlar, projenin sadece teorik bir konsept olmadığını, modern ve karmaşık saldırı tekniklerine karşı da tam görünürlük (Full Visibility) sağladığını göstermektedir.

6.5 Yanlış Pozitif (False Positive) Değerlendirmesi

Bir Saldırı Önleme Sistemi (IPS) için ”Yanlış Pozitif” (False Positive), zararlı olmayan meşru bir işlemin saldırı sanılarak engellenmesidir. Bu durum, sadece operasyonel aksaklıklara yol açmakla kalmaz, aynı zamanda sistem yöneticilerinin güvenlik uyarılarına karşı duyarsızlaşmasına (Alert Fatigue) neden olur [38]. RansomBPF projesinin başarısı, yüksek tespit oranının yanı sıra, günlük iş akışlarını kesintiye uğratmamasıyla da ölçülmüştür.

6.5.1 Meşru Yazılım Davranış Testleri

Fidye yazılımları ile meşru sistem araçları arasındaki en büyük benzerlik, her ikisinin de kısa sürede yoğun dosya işlemi (Burst I/O) yapmasıdır. Örneğin, bir yazılım projesinin derlenmesi (`make`) veya büyük bir dizinin kopyalanması (`cp -r`), dosya sistemi üzerinde fidye yazılımına benzer bir imza bırakır [3].

Sistemin ayırt etme yeteneğini test etmek amacıyla, Tablo 6.5’de belirtilen standart Linux araçları ile yoğun stres testleri gerçekleştirilmiştir.

Değerlendirme: Normal şartlarda, `rm -rf` komutu ile yapılan toplu silme işlemi veya `git clone` ile yapılan seri yazma işlemi, analiz motorunun H1 (Hız) kuralını tetikleyerek risk puanını eşik değerin üzerine çıkarmalıydı. Ancak, sisteme entegre edilen ”Beyaz Liste” (Whitelist) modülü sayesinde, bu süreçlerin dijital imzaları (Process Name) doğrulanmış ve analiz motoru tarafından ”Güvenli” olarak etiketlenmiştir [8]. Testler sonucunda, hiçbir meşru işlem engellenmemiş (Zero False Positive), sistemin kullanılabilirliği korunmuştur.

Tablo 6.5. Meşru Sistem Araçları ile Yanlış Pozitif Test Sonuçları

Araç / Komut	İşlem Profili	Beklenen Davranış	Sonuç
`git clone <repo>`	Çok sayıda küçük dosya yazma ve oluşturma.	İzin Ver	**PASSED**
`cp -r /src /dest`	Yüksek hızda sıralı okuma ve yazma.	İzin Ver	**PASSED**
`gcc main.c -o app`	Geçici dosya oluşturma ve silme (unlink).	İzin Ver	**PASSED**
`rm -rf /tmp/junk`	Toplu dosya silme (Yüksek riskli davranış).	İzin Ver	**PASSED**
`apt upgrade`	Sistem dosyalarını değiştirme ve güncelleme.	İzin Ver	**PASSED**

6.5.2 Beyaz Liste Performansı ve Kaynak Kullanımı

Yanlış pozitifleri önlemek için kullanılan filtreleme mekanizmasının, sisteme ek bir yük getirmemesi gerekmektedir. Eğer güvenlik yazılımı, her dosya işleminde uzun bir güvenli listeyi (Linked List) baştan sona tarasaydı, sistem performansı ciddi oranda düşerdi [4].

RansomBPF projesinde bu sorun, algoritmik karmaşıklığı $O(1)$ olan Hash Tablosu (Hash Table) yapısı ile çözülmüştür. Test ortamında yapılan performans ölçümlerine göre:

- **CPU Yükü:** Beyaz liste kontrolü, işlemci döngüleri üzerinde ölçülebilir bir etki yaratmamıştır (Göz ardı edilebilir seviye).
- **Sorgu Süresi:** Listede 10 veya 1000 güvenli uygulama olması fark etmeksizin, `is_whitelisted()` fonksiyonunun dönüş süresi sabit kalmıştır [16].

Bu mimari tercih, sistemin ”Gürültü” (Noise) olarak adlandırılan meşru veri trafiğini minimum maliyetle elemesini ve işlemci gücünü sadece bilinmeyen, şüpheli süreçlerin analizine odaklamasını sağlamıştır.

6.6 Bulguların Değerlendirilmesi

Bu çalışmanın başlangıcında fidye yazılımlarını imza veritabanlarına ihtiyaç duymadan, davranışsal analiz yöntemleriyle tespit edebilen ve eBPF teknolojisini kullanan proaktif bir güvenlik aracı geliştirilmesi hedeflenmiştir. Bölüm 6 genelinde sunulan deneysel veriler, bu hedeflere büyük ölçüde ulaşıldığını ve projenin kavramsal tasarımdan (PoC) çıkıp, operasyonel bir güvenlik ürününe dönüştüğünü göstermektedir.

Elde edilen bulguların, projenin temel başarı kriterleri ile örtüşme durumu aşağıda değerlendirilmiştir.

6.6.1 Hedefler ve Çıktıların Tutarlılığı

Projenin geliştirme sürecinde, sadece izleme (Monitoring) odaklı bir yapıdan, aktif engelleme (Prevention) odaklı bir yapıya geçiş yapılmıştır. Test sonuçları, bu mimari değişikliğin doğruluğunu kanıtlamaktadır:

- **Gerçek Zamanlılık:** eBPF Ring Buffer yapısı sayesinde olayların kullanıcı alanına iletimi nanosaniyeler seviyesinde gerçekleşmiş, sistemin tepki süresi (Latency) 100 milisaniyenin altında tutulmuştur. Bu, hedeflenen ”Yüksek Performanslı İzleme” kriterini fazlasıyla karşılamaktadır [4].
- **Bilinmeyen Tehditleri Yakalama:** Sistem, testlerde belirli bir fidye yazılımı ailesinin imzasına (hash) bakmaksızın, sadece dosya sistemi üzerindeki davranış paternlerini (yazma yoğunluğu, uzantı değişimi) analiz ederek başarılı olmuştur. Bu durum, sistemin ”Zero-Day” (Sıfırıncı Gün) saldırılarına karşı dirençli olduğunu doğrular [7].
- **Operasyonel Güvenilirlik:** Yanlış pozitif (False Positive) testlerinde, `git` ve `gcc` gibi geliştirici araçlarının engellenmemesi, sistemin üretim ortamlarında iş sürekliliğini bozmadan çalışabileceğini göstermiştir [38].

6.6.2 Veri Kurtarma Başarısı ve Kritik Değerlendirme

Saldırı simülasyonlarında elde edilen **%94 veri kurtarma oranı** (50 dosyadan 47’sinin kurtarılması), çalışmanın en somut başarısıdır.

Siber güvenlikte ”mükemmel koruma” (yani %100 kurtarma) genellikle teorik bir iddiadır. Çünkü davranışsal analiz sistemleri, doğaları gereği bir eylemin zararlı olduğuna karar vermeden önce en az bir veya birkaç şüpheli hareketi gözlemlemek zorundadır [8]. Bu çalışmada, feda edilen ilk 3 dosya (sistemin öğrenme maliyeti), geri kalan 47 dosyanın ve tüm sistemin kurtarılmasını sağlamıştır.

Endüstriyel standartlar göz önüne alındığında, bir fidye yazılımı saldırısının tüm diski şifrelemesi yerine sadece birkaç dosyayla sınırlandırılması, ”Kabul Edilebilir Risk” (Acceptable Risk) sınırları içerisindedir. Sonuç olarak RansomBPF; hızlı tepki süresi, genişletilmiş sistem çağrısı kapsamı ve kararlı çalışma yapısıyla, modern Linux sunucularını koruma kapasitesine sahip, yetkin bir güvenlik çözümü olarak kendini kanıtlamıştır [32].

7. SONUÇ VE GELECEK ÇALIŞMALAR

Bu tez çalışması kapsamında, modern siber tehditlerin en yıkıcı formlarından biri olan fidye yazılımlarına (Ransomware) karşı, Linux işletim sistemi mimarisiyle bütünleşik çalışan, eBPF tabanlı, yüksek performanslı ve proaktif bir Saldırı Önleme Sistemi (IPS) geliştirilmiştir.

7.1 Genel Değerlendirme

Fidye yazılımlarının statik imza tabanlı tespit yöntemlerini aşarak "Hizmet Olarak Fidye Yazılımı" (RaaS) modeliyle endüstrileşmesi, siber güvenlik dünyasında yeni savunma paradigmalarını zorunlu kılmıştır [1]. Bu çalışma, söz konusu tehditlere karşı reaktif değil, proaktif bir savunma hattı oluşturmak amacıyla yürütülmüştür. Projenin yaşam döngüsü ve elde edilen çıktılar aşağıdaki başlıklarda değerlendirilmiştir.

7.1.1 Teknolojik Seçim ve Mimari Evrim Süreci

Projenin başlangıç aşamasında yapılan literatür taraması, geleneksel antivirüs çözümlerinin, özellikle "Sıfırıncı Gün" (Zero-Day) saldırılarına ve dosyasız (fileless) tehditlere karşı yetersiz kaldığını göstermiştir [7]. Bu tespitten hareketle, Linux çekirdeğine (Kernel) güvenli ve performanslı bir şekilde müdahale imkanı tanıyan **eBPF (Extended Berkeley Packet Filter)** teknolojisi, projenin temel yapı taşı olarak belirlenmiştir [4].

Geliştirme süreci, basit bir dosya izleme prototipinden (Kavram Kanıtlama - PoC), endüstriyel standartlarda bir güvenlik ürününe doğru evrilmiştir:

1. **Görünürlük (Visibility):** İlk aşamada, çekirdek ile kullanıcı alanı arasında `Ring Buffer` yapısı kurularak, dosya sistemi olaylarının kayıpsız ve düşük gecikmeyle aktarılması sağlanmıştır [12]. İzleme kapsamı, saldırganların sıkça başvurduğu `pwrite64` ve `writev` gibi karmaşık sistem çağrılarını kapsayacak şekilde genişletilmiştir.

2. **Analiz Yeteneği (Detection):** Ham verilerin anlamlandırılması için, statik eşik değerleri yerine "Kümülatif Risk Puanlama" ve "Dinamik Sönümleme" (Decay) algoritmaları geliştirilmiştir. Bu sayede sistem, hem ani saldırıları (Burst) hem de zamana yayılan sinsi saldırıları (Low and Slow) ayırt edebilir hale gelmiştir [8].

3. **Aktif Müdahale (Prevention):** Projenin son fazında, sadece tespit eden (IDS) yapıdan, tehdidi kaynağında durduran (IPS) yapıya geçilmiştir. Risk eşiğini aşan süreçlerin `SIGKILL` sinyali ile milisaniyeler içinde sonlandırılması, sistemin "gerçek zamanlı koruma" vaadini yerine getirmesini sağlamıştır [36].

7.1.2 Hedef ve Sonuç Uyumu

Tez çalışmasının başlangıcında ortaya konulan temel hipotez şuydu: *"Bir fidye yazılımı, kod yapısını ne kadar değiştirirse değiştirsin, dosyaları şifrelemek için sergilemek zorunda olduğu davranışsal paternler (yoğun okuma/yazma, uzantı değişimi) gizlenemez."*

Gerçekleştirilen "Canlı Ateş" (Live Fire) testleri ve simülasyon sonuçları, bu hipotezi doğrulamıştır [3]:

- **Davranışsal Tespit Başarısı:** Geliştirilen **RansomBPF**, bilinen zararlı imzalarına ihtiyaç duymadan, tamamen davranış profillemesi üzerinden saldırıları tespit etmiştir. Bu durum, projenin "imza bağımsız koruma" hedefine ulaştığını göstermektedir.
- **Performans ve Verimlilik:** $O(1)$ karmaşıklığına sahip beyaz liste mekanizması ve hafif siklet (lightweight) C ajanı sayesinde, sistemin meşru iş yükleri üzerindeki etkisi minimize edilmiştir [16].
- **Adli Bilişim Yeteneği:** Üretilen yapılandırılmış JSON logları, saldırı sonrası analiz süreçleri için gerekli olan "kim, ne zaman, hangi dosyaya, nasıl saldırdı?" sorularına net cevaplar üreterek, projenin operasyonel hedeflerini de karşılamıştır.

Sonuç olarak; akademik bir bitirme projesi olarak başlayan RansomBPF, Linux sunucularının güvenliğini sağlamada modern, ölçeklenebilir ve etkili bir çözüm mimarisi sunmayı başarmıştır.

7.2 Elde Edilen Kazanımlar

Bu tez çalışması, modern Linux çekirdek özelliklerinin siber güvenlik alanında nasıl etkin bir savunma silahına dönüştürülebileceğini somut verilerle ortaya koymuştur. Teorik tasarımın ötesine geçilerek, gerçek dünya senaryolarında hayatta kalabilen bir mimari başarıyla uygulanmıştır. Proje sonucunda elde edilen temel kazanımlar, performans metrikleri ve analiz yetenekleri olmak üzere iki ana eksende toplanabilir.

7.2.1 Tespit ve Müdahale Performansı

Bir Saldırı Önleme Sistemi'nin (IPS) başarısı, en kötü senaryo (Worst-Case Scenario) altındaki dayanıklılığı ile ölçülür. RansomBPF ile gerçekleştirilen "Canlı Saldırı" (Live Fire) testleri, sistemin operasyonel yetkinliğini şu metriklerle kanıtlamıştır:

- **Yüksek Veri Kurtarma Oranı (%94):** Saldırı simülasyonlarında, fidye yazılımı henüz 4. dosyayı şifrelemeye çalışırken engellenmiş ve test veri setindeki 50 dosyadan 47'si kurtarılmıştır. Bu oran, sistemin bir saldırıyı "felaket" boyutuna ulaşmadan, tolere edilebilir hasar seviyesinde durdurduğunu göstermektedir [32].

- **Milisaniye Seviyesinde Müdahale (< 100 ms):** Analiz motoru ve aktif müdahale modülü arasındaki gecikmenin 100 milisaniyenin altında tutulması, projenin en kritik teknik kazanımıdır. Bu hız, zararlı yazılımın disk üzerindeki yazma kuyruğunu (Write Queue) doldurmasına dahi fırsat vermeden etkisiz hale getirilmesini sağlamıştır [4].
- **Sıfırıncı Gün (Zero-Day) Koruması:** Sistemin bilinen zararlı imzalarına (hash) ihtiyaç duymadan, sadece `pwrite64` ve `rename` gibi sistem çağrılarını analiz ederek başarılı olması, henüz keşfedilmemiş tehditlere karşı da koruma sağladığını kanıtlamıştır [7].

7.2.2 Akıllı Analiz ve Esneklik Yeteneği

RansomBPF, geleneksel hız sınırlayıcı (Rate Limiting) sistemlerin aksine, olayları bağlamıyla birlikte değerlendiren "Akıllı Analiz" yeteneğine sahiptir. Geliştirilen Puan Tabanlı Risk Motoru ve Dinamik Sönümleme (Decay) algoritmaları, projeye statik sistemlerde bulunmayan şu avantajları kazandırmıştır:

- **Dinamik Adaptasyon:** Sabit bir "saniyede 10 dosya" kuralı yerine uygulanan ağırlıklı risk puanlama sistemi, saldırganların "Low and Slow" (Yavaş ve Sinsi) taktiklerini işlevsiz kılmıştır. Sistem, saldırı yavaşlasa bile "artık risk" (residual risk) kavramı sayesinde sürecin sabıka kaydını hafızasında tutmayı başarmıştır [8].
- **Yanlış Pozitiflerin Minimizasyonu:** Sönümleme algoritması sayesinde, zaman içinde şüpheli davranış sergilemeyen süreçlerin risk puanları otomatik olarak düşürülmüştür. Bu mekanizma, sistem yöneticilerinin veya meşru servislerin (Veritabanları, Loglayıcılar) uzun süreli çalışmalarında yanlışlıkla engellenmesini (False Positive) önlemiştir [38].
- **O(1) Kararlılığı:** Beyaz liste kontrollerinin hash tablosu üzerinden yapılması, korunan sistemde yüzlerce güvenli uygulama olsa dahi güvenlik duvarının performansının düşmemesini garanti altına almıştır [16].

Özetle; bu çalışma ile Linux ekosistemi için hafif (lightweight), sistem kaynaklarını tüketmeyen ancak saldırı anında son derece agresif koruma sağlayan hibrit bir güvenlik mimarisi literatüre kazandırılmıştır [1].

7.3 Projenin Özgün Katkıları

Bu tez çalışması, sadece mevcut teknolojilerin bir uygulaması olmakla kalmayıp, fidye yazılımı tespiti alanında özgün mimari yaklaşımlar ve uygulama pratikleri sunmuştur. Literatürdeki mevcut eBPF tabanlı güvenlik çalışmaları incelendiğinde, RansomBPF projesini ayrıştıran temel yenilikler aşağıda sıralanmıştır.

7.3.1 Teknik İnovasyon ve Hibrit Mimari

Akademik ve sektörel literatürde yer alan eBPF çalışmaları genellikle ”Gözlemlenebilirlik” (Observability) üzerine yoğunlaşmış, önleme (prevention) mekanizmaları ise genellikle ikinci planda kalmıştır [30, 31]. Ayrıca, mevcut tespit sistemleri ya sadece imza tabanlıdır ya da basit hız limitlerine (Rate Limiting) dayanmaktadır.

RansomBPF, aşağıdaki iki farklı teknolojik disiplini tek bir çekirdek mimarisinde birleştirerek özgün bir katkı sunmuştur:

1. **Yüksek Hız ile Akıllı Analizin Sentezi:** Proje, eBPF'in sağladığı `Ring Buffer` teknolojisini kullanarak veri akışını optimize etmiş, ancak bu ham veriyi işlemek için basit sayaçlar yerine ”Durum Farkındalıklı” (State-Aware) bir **Kümülatif Risk Puanlama** algoritması geliştirmiştir [12]. Bu hibrit yapı, sistemin hem çok hızlı (Kernel-space hızı) hem de bağlam duyarlı (User-space zekası) olmasını sağlamıştır.

2. **Kör Noktasız İzleme:** Standart güvenlik araçlarının genellikle ihmal ettiği veya izlemediği `pwrite64`, `writev` ve `renameat2` gibi gelişmiş sistem çağrılarının tek bir çatı altında toplanarak analiz edilmesi, sistemin ”Tam Görünürlük” (Full Visibility) yeteneğini literatürdeki benzer çalışmaların ötesine taşımıştır [1].

7.3.2 Operasyonel Olgunluk ve Adli Analiz Altyapısı

Birçok akademik güvenlik projesi, kavram kanıtlama (PoC) aşamasında kalmakta ve gerçek dünya entegrasyonu (deployment) konusunda eksiklikler barındırmaktadır. RansomBPF, geliştirme sürecinde ”Üretime Hazır” (Production-Ready) prensiplerini benimseyerek şu özgün katkıları sağlamıştır:

- **Kurumsal Entegrasyon Yeteneği:** Yazılımın sadece bir komut satırı aracı olarak değil, `systemd` tabanlı, kendi kendini yönetebilen ve hata durumunda yeniden başlayan bir ”Daemon” servisi olarak mimarilendirilmesi, sistemin kurumsal Linux sunucularına (Web, Database, Mail Server) doğrudan entegre edilebilmesini sağlamıştır.

- **Yapılandırılmış Adli Veri (Structured Forensics):** Olay anında üretilen logların, insanlar için okunabilir metin formatının yanı sıra, SIEM (Security Information and Event Management) sistemleri tarafından pars edilebilen **JSON** formatında (`alerts.json`) sunulması, projenin sadece bir ”koruma aracı” değil, aynı zamanda bir ”adli analiz veri kaynağı” olarak kullanılabilmesine olanak tanımıştır [30].

7.4 Çalışmanın Kısıtlamaları

Her akademik ve teknik çalışmada olduğu gibi, RansomBPF projesinin de tasarım kararlarından veya teknolojik altyapısından kaynaklanan belirli kısıtları bulunmaktadır. Bu kısıtların şeffaf

bir şekilde ortaya konulması, projenin hangi koşullar altında en yüksek performansı vereceğinin anlaşılması ve gelecek çalışmalar için yol gösterici olması açısından önemlidir.

7.4.1 Platform ve Çekirdek Bağımlılığı

RansomBPF, Linux çekirdeğinin son yıllarda kazandığı `eBPF` (Extended Berkeley Packet Filter) teknolojisi üzerine inşa edilmiştir. Bu teknoloji, işletim sistemi mimarisine sıkı sıkıya bağlıdır [4].

- **İşletim Sistemi:** Sistem, Windows veya macOS gibi diğer işletim sistemlerinde çalışmamaktadır. Sadece Linux tabanlı sunucular (Ubuntu, Debian, CentOS, RedHat vb.) desteklenmektedir.
- **Çekirdek (Kernel) Sürümü:** Projede kullanılan `BPF_MAP_TYPE_RINGBUF` (Ring Buffer) veri yapısı ve **CO-RE** (Compile Once - Run Everywhere) mimarisi, modern çekirdek özelliklerine ihtiyaç duyar. Bu nedenle, yazılımın çalışabilmesi için Linux 5.8 veya üzeri bir çekirdek sürümü gerekmektedir [12].
- **BTF Desteği:** Taşınabilirliğin sağlanması için hedef sistemin çekirdeğinde `CONFIG_DEBUG_INFO_BTF=y` derleme seçeneğinin aktif olması ve BPF Tip Formatı (BTF) verilerinin bulunması zorunludur [14].

7.4.2 Analiz Kapsamı ve Vektör Sınırları

Projenin mevcut sürümü (v0.9.x), fidye yazılımlarının en yıkıcı etkisi olan ”Veri Şifreleme” eylemini engellemeye odaklanmıştır. Bu odaklanma, bazı saldırı vektörlerinin kapsam dışı kalmasına neden olmuştur:

- **Dosya Sistemi Odaklılık:** Sistem, sadece disk giriş-çıkış (I/O) operasyonlarını izlemektedir. Bir saldırganın, dosyaları şifrelemeden önce verileri internete sızdırması (Data Exfiltration) durumunda, RansomBPF'in ağ trafiği (Network Traffic) üzerinde bir izleme kancası bulunmadığı için bu veri hırsızlığını tespit etmesi mümkün değildir [3].
- **Bellek İçi (In-Memory) Saldırılar:** ”Dosyasız” (Fileless) zararlı yazılımlar, disk üzerinde hiçbir iz bırakmadan doğrudan bellek (RAM) üzerinde çalışabilirler. RansomBPF, bu tür zararlıların bellekteki kod enjeksiyonlarını değil, ancak diske yazmaya çalıştıkları anda yarattıkları etkileri tespit edebilmektedir.

7.5 Gelecek Çalışmalar

RansomBPF projesi, mevcut haliyle işlevsel ve etkili bir koruma sağlasa da, siber güvenlik dünyasının dinamik yapısı sürekli gelişimi zorunlu kılmaktadır. Bu tez çalışmasının devamı niteliğinde olabilecek ve sistemi endüstriyel bir ürüne dönüştürecek potansiyel geliştirmeler üç ana başlık altında planlanmıştır.

7.5.1 Makine Öğrenmesi Tabanlı Dinamik Profilleme

Mevcut analiz motoru, geliştirici tarafından önceden tanımlanmış sabit katsayılara (Heuristics) ve eşik değerlerine dayanmaktadır. Gelecek çalışmalarda, bu deterministik yapının ”Yapay Zeka” (AI) destekli stokastik bir modele dönüştürülmesi hedeflenmektedir.

- **Kullanıcı Davranış Analizi (UBA):** Sistem, ”Denetimsiz Öğrenme” (Unsupervised Learning) algoritmaları kullanarak sunucu üzerindeki normal işleyişi (Baseline) öğrenecektir. Örneğin, her Cuma gecesi yapılan yoğun yedekleme işlemini zamanla ”normal” olarak etiketleyecek ve yanlış alarmları (False Positive) insan müdahalesi olmadan eleyebilecektir [39].
- **Dinamik Eşik Yönetimi:** Sabit bir risk limiti (Örn: 100) yerine, sunucunun anlık yük durumuna göre değişen adaptif bir eşik mekanizması kurgulanacaktır [2].

7.5.2 Ağ Katmanı Entegrasyonu (eBPF XDP)

Fidye yazılımlarının şifreleme anahtarlarını almak veya çalınan verileri dışarı sızdırmak (Exfiltration) için Komuta Kontrol (C2) sunucularıyla iletişim kurduğu bilinmektedir [3]. Projenin sonraki fazında, eBPF'in ağ tarafındaki gücü olan **XDP (eXpress Data Path)** ve **TC (Traffic Control)** kancalarının sisteme eklenmesi planlanmaktadır [4].

Bu genişletme ile sistem:

- Zararlı sürecin dış dünyaya veri göndermesini paket seviyesinde engelleyebilecek,
- Bilinen zararlı IP adresleri ile yapılan ”Handshake” (El Sıkışma) girişimlerini çekirdek seviyesinde düşürebilecek (drop),
- Dosya sistemi ve ağ aktivitelerini korele ederek (Correlation) tespit doğruluğunu artıracaktır.

7.5.3 Grafiksel Yönetim Paneli (GUI)

Mevcut sürümde sistem yönetimi, komut satırı arayüzü (CLI) ve JSON logları üzerinden yürütülmektedir. Son kullanıcı deneyimini iyileştirmek ve olaylara müdahale süresini kısaltmak amacıyla, web tabanlı modern bir yönetim paneli geliştirilmesi öngörülmektedir.

Geliştirilecek bu arayüz üzerinden sistem yöneticileri:

- Sunucunun anlık risk durumunu görsel grafiklerle izleyebilecek,
- Engellenen süreçleri tek tıkla inceleyip beyaz listeye ekleyebilecek,
- Geçmişe dönük saldırı analizlerini ve istatistiksel raporları görselleştirebilecektir [13].

KAYNAKLAR

[1] Trend Micro, "Linux ransomware: The new frontier in cloud security." https://www.trendmicro.com/vinfo/us/security/news/ransomware-spotlight/linux-ransomware, 2023. Erişim Tarihi: 2025.

[2] F. Idrees, M. Rajarajan, M. Conti, T. Chen, and Y. Rahulamathavan, "Pindroid: A novel android malware detection framework using ensemble learning," *Computers & Security*, vol. 116, p. 102619, 2022. Polimorfik ve statik analiz limitleri üzerine tartışma içerir.

[3] A. Kharraz, W. Robertson, D. Balzarotti, L. Bilge, and E. Kirda, "Cutting the cord: Analyzing the maliciously provided network infrastructure," in *Proceedings of the 24th USENIX Security Symposium*, pp. 1–16, 2015. Davranışsal analiz ve dosya sistemi aktiviteleri üzerine temel çalışma.

[4] B. Gregg, *BPF Performance Tools*. Addison-Wesley Professional, 2019. Linux BPF ve sistem gözlemlenebilirliği üzerine referans kitap.

[5] B. A. S. Al-rimy, M. A. Maarof, and S. Z. M. Shaid, "Ransomware threat success factors, taxonomy, and countermeasures: A survey," *IEEE Access*, vol. 6, pp. 14460–14481, 2018. RaaS ve fidye yazılımı evrimi üzerine detaylı inceleme.

[6] Symantec, "Internet security threat report (istr) volume 24," tech. rep., Symantec Corporation, February 2019. Living off the Land (LotL) teknikleri ve dosyasız tehditler analizi.

[7] A. Moser, C. Kruegel, and E. Kirda, "Limits of static analysis for malware detection," in *Proceedings of the 23rd Annual Computer Security Applications Conference (ACSAC)*, pp. 161–170, IEEE, 2007. Polimorfik kod ve statik analizin yetersizliği üzerine temel kaynak.

[8] A. Continella, A. Guagnelli, G. Zingaro, G. De Pasquale, A. Barenghi, S. Zanero, and F. Maggi, "Shieldfs: A self-healing, ransomware-aware filesystem," in *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC)*, pp. 336–347, 2016. Bağlamsal davranış analizi ve I/O aktivitelerinin ayırt edilmesi.

[9] The Linux Foundation, "Linux kernel history report," tech. rep., The Linux Foundation, 2020. Bulut altyapıları ve süper bilgisayarlarda Linux kullanım oranları.

[10] P. e. a. Momeni, "Performance analysis of linux auditing system," in *International Conference on High Performance Computing and Simulation (HPCS)*, 2019. Auditd'nin sistem kaynakları üzerindeki yükünün analizi.

[11] M. Wan, J. Li, Y. Yan, and X. Zhao, "The evolution of ransomware: From a simple scam to an asymmetric cyberwar," *IEEE Access*, vol. 10, pp. 42642–42656, 2022. Fidye yazılımlarının asimetrik savaş unsuru olarak analizi.

[12] A. Nakryiko, "Bpf ring buffer: A new efficient way to communicate with user space." Linux Kernel Documentation, 2020. Perf buffer ile Ring buffer arasındaki performans ve bellek yönetimi farkları.

[13] K. Kent and M. Souppaya, "Guide to computer security log management (sp 800-92)," tech. rep., National Institute of Standards and Technology (NIST), 2006. Adli analiz ve olay müdahalesi için loglama standartları.

[14] A. Nakryiko, "Bpf portability and co-re." Facebook Microsites, 2020. BTF (BPF Type Format) ve farklı kernel sürümleri arasında taşınabilirlik.

[15] A. Nakryiko, "Libbpf: The future of bpf library." Linux Kernel Documentation, 2019. CO-RE ve modern eBPF uygulamaları için standart kütüphane.

[16] T. D. Hanson, "uthash: A hash table for c structures," 2023. C makroları ile implemente edilmiş yüksek performanslı hash tablosu.

[17] C. Beaman, A. Barkworth, T. D. Akram, G. Guest, and K. Snook, "Ransomware: Recent advances, analysis, challenges and future research directions," *Computers & Security*, vol. 111, p. 102490, 2021. Fidye yazılımlarının hedef değiştirmesi ve güncel saldırı vektörleri.

[18] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, "Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains," in *Leading Issues in Information Warfare and Security Research*, vol. 1, pp. 80–106, Academic Conferences Limited, 2011. Cyber Kill Chain modelinin orijinal kaynağı.

[19] A. Gazet, "Comparative analysis of various ransomware virii," *Journal in Computer Virology*, vol. 6, no. 1, pp. 77–90, 2010. Fidye yazılımlarının şifreleme mekanizmalarının karşılaştırmalı analizi.

[20] M. Ozcelik and R. Brooks, "Deceiving entropy based ransomware detection using intermittent encryption," *Computers & Security*, vol. 128, p. 103138, 2023. Kısmi şifreleme tekniklerinin entropi tabanlı sistemlere etkisi.

[21] R. Lyda and J. Hamrock, "Using entropy analysis to find encrypted and packed malware," *IEEE Security & Privacy*, vol. 5, no. 2, pp. 40–45, 2007. Shannon entropisi kullanarak şifreli içeriğin tespiti üzerine temel kaynak.

[22] Trend Micro, "The near future of linux ransomware," tech. rep., Trend Micro Research, 2023. Linux sistemlerinin kritik altyapılardaki rolü ve artan saldırı yüzeyi.

[23] G. Aggarwal and S. P., "Analysis of ransomware attacks on virtualized environments: The case of esxiargs," *International Journal of Cloud Computing and Security*, vol. 12, no. 2, pp. 45–58, 2023. Hipervizör seviyesindeki şifreleme teknikleri ve .vmdk hedefli saldırılar.

[24] Z. Bazrafshan, H. Hashemi, S. M. H. Fard, and A. Hamzeh, "A survey on heuristic malware detection techniques," *The 5th Conference on Information and Knowledge Technology*, pp. 113–120, 2013. Statik, dinamik ve hibrit analiz yöntemlerinin karşılaştırması.

[25] P. Szor, *The Art of Computer Virus Research and Defense*. Addison-Wesley Professional, 2005. Polimorfizm, metamorfizm ve kod gizleme teknikleri üzerine temel referans.

[26] X. Ugarte-Pedrero, D. Balzarotti, I. Santos, and P. G. Bringas, "Sok: Deep packer inspection: A longitudinal study of the complexity of run-time packers," in *Proceedings of the 36th IEEE Symposium on Security and Privacy (S&P)*, pp. 659–673, IEEE, 2015. Packer kullanımı ve statik analize etkileri.

[27] M. Egele, T. Scholte, E. Kirda, and C. Kruegel, "A survey on automated dynamic malware-analysis techniques and tools," *ACM Computing Surveys (CSUR)*, vol. 44, no. 2, pp. 1–42, 2012. Dinamik analiz yöntemleri, API hooking ve sistem çağrıları üzerine temel kaynak.

[28] S. McCanne and V. Jacobson, "The bsd packet filter: A new architecture for user-level packet capture," in *Proceedings of the USENIX Winter 1993 Technical Conference*, pp. 259–269, USENIX Association, 1993. BPF teknolojisinin orijinal tasarımı ve sanal makine yapısı.

[29] P. Momeni, H. R., and A. M., "Performance evaluation of linux auditing system for high-speed networks," in *International Conference on Cyber Security and Cloud Computing (CSCloud)*, pp. 45–52, IEEE, 2021. Auditd ve geleneksel izleme araçlarının sistem üzerindeki yükü.

[30] The Falco Project, "Falco: Cloud native runtime security." https://falco.org/docs/, 2024. CNCF onaylı kural tabanlı tehdit tespit sistemi.

[31] Isovalent, "Tetragon: ebpf-based security observability and enforcement." https://github.com/cilium/tetragon, 2024. Cilium ekosistemi için süreç ve ağ görünürlüğü aracı.

[32] N. Scaife, H. Carter, P. Traynor, and K. Butler, "Cryptolock (and drop it): Stopping ransomware attacks on user data," in *Proceedings of the 36th International Conference on Distributed Computing Systems (ICDCS)*, pp. 303–312, IEEE, 2016. Locker ve Crypto türleri arasındaki teknik farklar.

[33] A. Continella, A. Guagnelli, G. Zingaro, G. De Pasquale, A. Barenghi, S. Zanero, and F. Maggi, "Shieldfs: A self-healing, ransomware-aware filesystem," in *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC)*, pp. 336–347, 2016. Fidye yazılımlarının dosya sistemi üzerindeki kaçınılmaz ayak izleri.

[34] C. Moore, "Detecting ransomware with honeypot techniques," in *Proceedings of the Cybersecurity and Cyberforensics Conference (CCC)*, pp. 77–81, IEEE, 2016. Tuzak dosyaların fidye yazılımı tespitindeki etkinliği üzerine temel çalışma.

[35] A. Sedgewick, M. Souppaya, and K. Scarfone, “Guide to application whitelisting (sp 800-167),” tech. rep., National Institute of Standards and Technology (NIST), 2015. Sistem güvenliği için beyaz liste oluşturma stratejileri ve standartları.

[36] R. Brewer, “Ransomware attacks: detection, prevention and cure,” *Network Security*, vol. 2016, no. 9, pp. 5–9, 2016. Geleneksel savunma yöntemlerinin yetersizliği ve önlemenin önemi.

[37] R. Love, *Linux Kernel Development*. Addison-Wesley Professional, 3rd ed., 2010. Linux sinyal mekanizması ve süreç sonlandırma teknikleri üzerine referans.

[38] S. Axelsson, “The base-rate fallacy and its implications for the difficulty of intrusion detection,” *ACM Transactions on Information and System Security (TISSEC)*, vol. 3, no. 3, pp. 186–205, 2000. Yanlış pozitif oranının ve uyarı yorgunluğunun güvenlik sistemleri üzerindeki etkisi.

[39] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys (CSUR)*, vol. 41, no. 3, pp. 1–58, 2009. Sistem güvenliğinde anomali tespiti ve denetimsiz öğrenme yöntemleri üzerine referans çalışma.

EK-A: KAYNAK KODLAR

Projenin tüm kaynak kodlarına ve kurulum talimatlarına **https://github.com/osmancevik/ransom-bpf** adresinden erişilebilir. Bu bölümde sistemin çekirdek mantığını oluşturan kritik dosyalar sunulmuştur.

A.1 hello_kern.c - eBPF Çekirdek Alanı Veri Toplayıcı

Linux çekirdek izleme noktalarına kanca atarak sistem çağrılarını yakalayan eBPF programıdır.

```
/**
 * @file hello_kern.c
 * @brief eBPF Kernel Space Program for Ransomware Detection.
 * @version 0.9.0
 *
 * This module hooks into various kernel tracepoints to capture file system
 * and process activities. It collects metadata (PID, UID, PPID, Comm)
 * and sends structured events to user space via a Ring Buffer.
 *
 * It provides "Universal Coverage" by monitoring standard syscalls (write, rename)
 * as well as variants used by high-level languages like Python and Java (pwrite64).
 */

#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>
#include "common.h"

/**
 * @brief Ring Buffer definition for high-performance data transfer.
 *
 * Used to push `struct event` data from kernel space to user space.
 * Size is set to 256KB to accommodate bursty traffic.
 */
struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 256 * 1024);
} rb SEC(".maps");

/**
 * @brief Helper function to package and submit an event to the Ring Buffer.
 *
 * Populates the `struct event` with process metadata (PID, UID, PPID)
 * and the associated filename, then submits it to user space.
 *
 * @param ctx Tracepoint context (opaque pointer).
 * @param type Event type identifier (e.g., EVENT_WRITE, EVENT_RENAME).
 * @param filename_ptr Pointer to the filename string in user memory (can be NULL).
 */
static __always_inline void send_event(void *ctx, int type, const char *filename_ptr)
{
    struct event *e;
    __u64 pid_tgid;
    __u64 uid_gid;
    struct task_struct *task;

    // Reserve space in the Ring Buffer
    e = bpf_ringbuf_reserve(&rb, sizeof(*e), 0);
    if (!e) return;

    e->type = type;

    // 1. Capture PID and Process Name
    pid_tgid = bpf_get_current_pid_tgid();
```

```
    e->pid = (__u32)(pid_tgid >> 32);
    bpf_get_current_comm(&e->comm, sizeof(e->comm));

    // 2. Capture User ID (Real UID)
    uid_gid = bpf_get_current_uid_gid();
    e->uid = (__u32)uid_gid;

    // 3. Capture Parent Process ID (PPID) using CO-RE
    task = (struct task_struct *)bpf_get_current_task();
    e->ppid = BPF_CORE_READ(task, real_parent, tgid);

    // 4. Capture Filename (if applicable)
    if (filename_ptr) {
        bpf_probe_read_user_str(&e->filename, sizeof(e->filename), filename_ptr);
    } else {
        e->filename[0] = '\0';
    }

    // Submit the event
    bpf_ringbuf_submit(e, 0);
}

// --- FILE WRITE OPERATIONS (WRITE FAMILY) ---

/**
 * @brief Hooks the standard `write` syscall.
 *
 * Captures file modification attempts. Commonly used by C/C++ applications
 * and shell scripts (e.g., `echo "data" > file`).
 */
SEC("tracepoint/syscalls/sys_enter_write")
int handle_write(struct trace_event_raw_sys_enter* ctx)
{
    send_event(ctx, EVENT_WRITE, NULL);
    return 0;
}

/**
 * @brief Hooks the `pwrite64` syscall.
 *
 * Critical for detecting ransomware written in high-level languages
 * like Python and Java, which prefer positional writes.
 */
SEC("tracepoint/syscalls/sys_enter_pwrite64")
int handle_pwrite64(struct trace_event_raw_sys_enter* ctx)
{
    send_event(ctx, EVENT_WRITE, NULL);
    return 0;
}

/**
 * @brief Hooks the `writev` syscall (Vectorized Write).
 *
 * Monitors scatter-gather I/O operations, often used by high-performance
 * runtimes like NodeJS and Go.
 */
SEC("tracepoint/syscalls/sys_enter_writev")
int handle_writev(struct trace_event_raw_sys_enter* ctx)
{
    send_event(ctx, EVENT_WRITE, NULL);
    return 0;
```

```
117 }
118
119 // --- FILE RENAME OPERATIONS (RENAME FAMILY) ---
120
121 /**
122  * @brief Hooks the legacy `rename` syscall.
123  *
124  * Captures basic file renaming. `args[1]` contains the `newname`.
125  */
126 SEC("tracepoint/syscalls/sys_enter_rename")
127 int handle_rename(struct trace_event_raw_sys_enter* ctx)
128 {
129     send_event(ctx, EVENT_RENAME, (const char *)ctx->args[1]);
130     return 0;
131 }
132
133 /**
134  * @brief Hooks the `renameat` syscall.
135  *
136  * Directory-relative renaming. `args[3]` contains the `newname`.
137  */
138 SEC("tracepoint/syscalls/sys_enter_renameat")
139 int handle_renameat(struct trace_event_raw_sys_enter* ctx)
140 {
141     // signature: renameat(olddfd, oldname, newdfd, newname)
142     send_event(ctx, EVENT_RENAME, (const char *)ctx->args[3]);
143     return 0;
144 }
145
146 /**
147  * @brief Hooks the modern `renameat2` syscall.
148  *
149  * Used by modern Linux utilities (mv) and atomic exchange operations.
150  * `args[3]` contains the `newname`.
151  */
152 SEC("tracepoint/syscalls/sys_enter_renameat2")
153 int handle_renameat2(struct trace_event_raw_sys_enter* ctx)
154 {
155     // signature: renameat2(olddfd, oldname, newdfd, newname, flags)
156     send_event(ctx, EVENT_RENAME, (const char *)ctx->args[3]);
157     return 0;
158 }
159
160 // --- OTHER CRITICAL OPERATIONS ---
161
162 /**
163  * @brief Hooks the `openat` syscall.
164  *
165  * Captures file opening attempts, used for detecting traversal patterns
166  * or honeypot access. `args[1]` contains the filename.
167  */
168 SEC("tracepoint/syscalls/sys_enter_openat")
169 int handle_openat(struct trace_event_raw_sys_enter* ctx)
170 {
171     send_event(ctx, EVENT_OPEN, (const char *)ctx->args[1]);
172     return 0;
173 }
174
175 /**
176  * @brief Hooks the `execve` syscall.
177  *
```

```
178  * Captures new process execution. Essential for building the process tree
179  * and detecting chain execution (e.g., bash spawning python).
180  */
181 SEC("tracepoint/syscalls/sys_enter_execve")
182 int handle_execve(struct trace_event_raw_sys_enter* ctx)
183 {
184     send_event(ctx, EVENT_EXEC, (const char *)ctx->args[0]);
185     return 0;
186 }
187
188 /**
189  * @brief Hooks the process exit tracepoint.
190  *
191  * Used to signal the user space agent to clean up memory resources
192  * associated with the terminated process ID.
193  */
194 SEC("tracepoint/sched/sched_process_exit")
195 int handle_exit(void *ctx)
196 {
197     send_event(ctx, EVENT_EXIT, NULL);
198     return 0;
199 }
200
201 /**
202  * @brief Hooks the `unlinkat` syscall.
203  *
204  * Captures file deletion attempts. High risk if targeted at backups.
205  * `args[1]` contains the filename.
206  */
207 SEC("tracepoint/syscalls/sys_enter_unlinkat")
208 int handle_unlinkat(struct trace_event_raw_sys_enter *ctx)
209 {
210     send_event(ctx, EVENT_UNLINK, (const char *)ctx->args[1]);
211     return 0;
212 }
213
214 char LICENSE[] SEC("license") = "GPL";
```

Kod A.1. Çekirdek alanı veri toplama mantığı

A.2 main.c - Uygulama Giriş Noktası ve Orkestrasyon

Uygulamanın başlangıç yapılandırmasını ve olay döngüsünü (event loop) yöneten ana modüldür.

```
/**
 * @file main.c
 * @brief RansomBPF Application Entry Point and Orchestrator.
 * @version 0.9.0
 *
 * This module is responsible for bootstrapping the application, loading the
 * eBPF kernel program, managing the Ring Buffer event loop, and coordinating
 * data flow between the kernel and the user-space analysis engine.
 *
 * It handles the application lifecycle, including graceful shutdown on signals
 * and crash recovery logging.
 *
 * Update Note (v0.9.0): Addressed unused parameter warnings in signal and
 * event callbacks to ensure a clean compilation.
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>
#include <errno.h>
#include <bpf/libbpf.h>
#include "hello_kern.skel.h"
#include "common.h"
#include "logger.h"
#include "state_manager.h"
#include "detector.h"
#include "config.h"
#include "whitelist.h"
#include "cli.h"

// External declaration for cleanup
extern void cleanup_whitelist();

/** @brief Process ID of the agent itself, used for self-filtering. */
static int own_pid = 0;

/** @brief Global flag to control the main event loop. */
static volatile bool exiting = false;

/** @brief Stores the source of the loaded configuration for logging. */
static char config_source[256] = "Default (Embedded)";

/**
 * @brief Signal handler for graceful shutdown (SIGINT, SIGTERM).
 * @param sig Signal number.
 */
static void handle_exit(int sig) {
    (void)sig; // Silence unused parameter warning
    exiting = true;
}

```

```
54  /**
55   * @brief Signal handler for critical crashes (SIGSEGV, SIGABRT).
56   *
57   * Ensures logs are flushed to disk before the process terminates abnormally.
58   * @param sig Signal number.
59   */
60  static void handle_crash(int sig) {
61      fprintf(stderr, "CRITICAL ERROR: Program crashed! Signal: %d\n", sig);
62      finalize_logger();
63      exit(1);
64  }
65
66  /**
67   * @brief Ring Buffer event callback.
68   *
69   * This function is invoked by libbpf whenever a new event is received from
70   * the kernel. It acts as the pipeline dispatcher:
71   * 1. Filters out the agent's own activities (Self-Monitoring).
72   * 2. Handles process exit events for memory cleanup.
73   * 3. Checks the whitelist to reduce noise.
74   * 4. Forwards valid events to the Analysis Engine.
75   *
76   * @param ctx Context (unused).
77   * @param data Pointer to the `struct event` data.
78   * @param size Size of the data (unused).
79   * @return 0 on success.
80   */
81  int handle_event(void *ctx, void *data, size_t size) {
82      (void)ctx; // Silence unused parameter warning
83      (void)size; // Silence unused parameter warning
84
85      if (!data) return 0;
86      const struct event *e = data;
87
88      // [CRITICAL] Self-filtering to prevent Feedback Loops.
89      // Prevents the agent from analyzing its own log write operations,
90      // which would otherwise cause an infinite loop and 100% CPU usage.
91      if ((int)e->pid == own_pid) return 0;
92
93      // Handle Process Exit Event
94      // Necessary for preventing memory leaks in the state manager.
95      if (e->type == EVENT_EXIT) {
96          remove_process(e->pid);
97          return 0;
98      }
99
100     // State Management: Retrieve or create process statistics
101     struct process_stats *s = get_process_stats(e->pid, e->comm);
102     if (!s) return 0;
103
104     // Dynamic Whitelist Check (O(1) Hash Table lookup)
105     // If the process is trusted, skip expensive analysis.
106     if (is_whitelisted(s->comm)) return 0;
107
108     // Forward to Analysis Engine
109     analyze_event(s, e);
110
111     return 0;
112 }
113
114 /**
```

```
115  * @brief Main application entry point.
116  *
117  * 1. Initializes configuration and logger.
118  * 2. Parses CLI arguments.
119  * 3. Loads and verifies the eBPF program.
120  * 4. Enters the main polling loop.
121  * 5. Performs cleanup on exit.
122  *
123  * @param argc Argument count.
124  * @param argv Argument vector.
125  * @return 0 on success, error code otherwise.
126  */
127 int main(int argc, char **argv) {
128     struct hello_kern* skel;
129     struct ring_buffer *rb = NULL;
130     int err;
131
132     own_pid = getpid(); // Capture agent's own PID
133
134     // --- 1. PREPARATION ---
135     init_config_defaults();
136
137     // --- 2. CLI ARGUMENT PARSING ---
138     // Handle --help, --version, or config overrides.
139     // Returns 1 if the program should exit immediately (e.g., after help).
140     if (parse_arguments(argc, argv) == 1) {
141         return 0;
142     }
143
144     // --- 3. CONFIGURATION LOADING ---
145     // Priority: CLI Path (-c) > Local File > System File
146     if (strlen(config.config_path) > 0) {
147         if (access(config.config_path, F_OK) == 0) {
148             load_config_file(config.config_path);
149             snprintf(config_source, sizeof(config_source), "%s", config.config_path);
150         } else {
151             fprintf(stderr, "ERROR: Specified config file not found: %s\n", config.
                    config_path);
152             return 1;
153         }
154     }
155     else {
156         // Fallback to default locations
157         if (access("ransom.conf", F_OK) == 0) {
158             load_config_file("ransom.conf");
159             snprintf(config_source, sizeof(config_source), "./ransom.conf");
160         }
161         else if (access("/etc/ransom-bpf/ransom.conf", F_OK) == 0) {
162             load_config_file("/etc/ransom-bpf/ransom.conf");
163             snprintf(config_source, sizeof(config_source), "/etc/ransom-bpf/ransom.conf");
164         }
165     }
166
167     // --- 4. SYSTEM INITIALIZATION ---
168     // Initialize Logger and Whitelist using loaded config
169     init_logger();
170     init_whitelist(config.whitelist_str);
171
172     // [NEW] Silent Startup: Filter libbpf noise using custom callback
173     libbpf_set_print(logger_libbpf_print);
174
```

```
175     // Log startup info
176     LOG_INFO("Starting up... (Config Source: %s)", config_source);
177
178     // Display configuration summary to stdout
179     print_startup_summary();
180
181     // Register signal handlers
182     signal(SIGINT, handle_exit);
183     signal(SIGTERM, handle_exit);
184     signal(SIGSEGV, handle_crash);
185     signal(SIGABRT, handle_crash);
186
187     // --- 5. eBPF LOADING ---
188     skel = hello_kern__open();
189     if (!skel) {
190         LOG_ERR("Failed to open eBPF skeleton.");
191         return 1;
192     }
193
194     err = hello_kern__load(skel);
195     if (err) {
196         LOG_ERR("Failed to load eBPF program.");
197         goto cleanup;
198     }
199
200     err = hello_kern__attach(skel);
201     if (err) {
202         LOG_ERR("Failed to attach eBPF program.");
203         goto cleanup;
204     }
205
206     // Initialize Ring Buffer
207     rb = ring_buffer__new(bpf_map__fd(skel->maps.rb), handle_event, NULL, NULL);
208     if (!rb) {
209         LOG_ERR("Failed to create ring buffer.");
210         goto cleanup;
211     }
212
213     LOG_INFO("System monitoring active... (Press Ctrl+C to exit)");
214
215     // Main Event Loop
216     while (!exiting) {
217         err = ring_buffer__poll(rb, 100); // Poll every 100ms
218         if (err == -EINTR) { err = 0; break; }
219         if (err < 0) break;
220     }
221
222     // --- 6. CLEANUP ---
223     cleanup:
224     LOG_INFO("Shutting down...");
225     cleanup_whitelist();
226     cleanup_all_processes();
227     ring_buffer__free(rb);
228     hello_kern__destroy(skel);
229     finalize_logger();
230
231     return -err;
232 }
```

Kod A.2. Kullanıcı alanı ana yönetim döngüsü

A.3 detector.c - Sezgisel Analiz Motoru

Fidye yazılımı davranışlarını risk puanlaması üzerinden analiz eden karar mekanizmasıdır.

```
/**
 * @file detector.c
 * @brief Heuristic Analysis Engine implementation.
 * @version 0.9.8
 *
 * This module implements the core logic for detecting ransomware behavior based on
 * risk scoring, context awareness, and statistical anomalies. It also handles
 * the "Active Intervention" (IPS) mechanism with safety filters.
 *
 * Update Note (v0.9.8): Resolved logic errors in filename checks and utilized
 * unused parameters in the kill switch logging routine.
 */

#include <stdio.h>
#include <string.h>
#include <time.h>
#include <signal.h> // For kill() and SIGKILL
#include <errno.h>
#include "detector.h"
#include "logger.h"
#include "config.h"
#include "whitelist.h"

// --- HELPER FUNCTIONS ---

/**
 * @brief Applies a time-based decay to the process risk score.
 *
 * Reduces the risk score over time to prevent false positives from
 * long-running processes (handling "Low and Slow" attacks).
 * Currently reduces score by 10% for every second of inactivity.
 *
 * @param s Pointer to the process statistics structure.
 */
static void apply_decay(struct process_stats *s) {
    time_t now = time(NULL);
    double diff = difftime(now, s->last_decay_time);

    // Apply decay if at least 1 second has passed
    if (diff >= 1.0) {
        int decay_amount = (int)(s->current_score * 0.10 * diff);

        // Ensure at least 1 point is removed if score is positive
        if (s->current_score > 0 && decay_amount == 0) decay_amount = 1;

        s->current_score -= decay_amount;

        // Clamp score to 0
        if (s->current_score < 0) s->current_score = 0;

        // Reset burst counters if score reaches 0
        if (s->current_score == 0) {
            s->write_burst = 0;
            s->rename_burst = 0;
        }
```

```
56
57          s->last_decay_time = now;
58      }
59  }
60
61  /**
62   * @brief Checks if a filename ends with a known ransomware extension.
63   *
64   * Scans against a hardcoded list of suspicious extensions (e.g., .locked, .enc).
65   *
66   * @param filename The filename to check.
67   * @return 1 if suspicious, 0 otherwise.
68   */
69  static int has_suspicious_extension(const char *filename) {
70      if (!filename) return 0;
71
72      const char *suspicious_exts[] = {
73          ".locked", ".enc", ".cry", ".crypto",
74          ".crypted", ".wanna", ".dark", NULL
75      };
76
77      size_t len = strlen(filename);
78
79      for (int i = 0; suspicious_exts[i] != NULL; i++) {
80          const char *ext = suspicious_exts[i];
81          size_t ext_len = strlen(ext);
82
83          if (len > ext_len) {
84              // Check suffix match
85              if (strcmp(filename + len - ext_len, ext) == 0) return 1;
86          }
87      }
88      return 0;
89  }
90
91  /**
92   * @brief Checks if the accessed file is the designated honeypot.
93   *
94   * Performs a substring check against the configured honeypot file path.
95   *
96   * @param filename The accessed file path.
97   * @return 1 if it matches the honeypot, 0 otherwise.
98   */
99  int is_honeypot_access(const char *filename) {
100     if (!filename || strlen(config.honeypot_file) == 0) return 0;
101
102     if (strstr(filename, config.honeypot_file) != NULL) return 1;
103
104     return 0;
105 }
106
107 /**
108  * @brief Terminates a malicious process (Active Intervention).
109  *
110  * Sends a SIGKILL signal to the target process. Includes critical safety
111  * checks to prevent system instability (e.g., protecting PID 1).
112  *
113  * @param s Process statistics (Target).
114  * @param e Event details (Context).
115  * @param original_reason The primary reason for triggering the kill switch.
116  */
```

```
117 static void kill_process(struct process_stats *s, const struct event *e, const char *
        original_reason) {
118
119     // --- SAFETY FILTERS (Critical) ---
120
121     // 1. Critical System Process Protection (PID 0, 1)
122     // Killing PID 1 (init/systemd) causes a kernel panic.
123     if (s->pid <= 1) {
124         LOG_ERR("  CRITICAL SAFETY: Attempted to kill PID %d (init/systemd)! Blocked.", s->
                pid);
125         log_alert_json(
126             "KILL_PREVENTED",
127             s->pid, e->ppid, e->uid, s->comm, e->filename,
128             "Critical System Process Protection",
129             s->current_score
130         );
131         return;
132     }
133
134     // 2. Whitelist Protection (Last Resort Check)
135     // Double-check whitelist status to prevent collateral damage due to race conditions.
136     if (is_whitelisted(s->comm)) {
137         LOG_WARN("  SAFETY: Attempted to kill whitelisted process (%s)! Blocked.", s->comm)
                ;
138         log_alert_json(
139             "KILL_PREVENTED",
140             s->pid, e->ppid, e->uid, s->comm, e->filename,
141             "Whitelisted Process Protection",
142             s->current_score
143         );
144         return;
145     }
146
147     // --- ACTION (The Kill Switch) ---
148
149     // 9 = SIGKILL (Immediate termination, cannot be caught or ignored)
150     int ret = kill(s->pid, SIGKILL);
151
152     if (ret == 0) {
153         // SUCCESS: Log to console
154         LOG_ALARM("  ACTIVE INTERVENTION: Process Killed! PID: %d (%s)", s->pid, s->comm);
155
156         // SUCCESS: Log to Alerts JSON
157         // FIX: Using 'original_reason' instead of hardcoded string to fix unused parameter
                warning
158         log_alert_json(
159             "PROCESS_KILLED",
160             s->pid, e->ppid, e->uid, s->comm, e->filename,
161             original_reason,
162             s->current_score
163         );
164     } else {
165         // FAILURE: Permission denied, process already dead, or zombie
166         LOG_ERR("  INTERVENTION FAILED: Could not kill process (PID: %d). Error: %s",
167                 s->pid, strerror(errno));
168
169         log_alert_json(
170             "KILL_FAILED",
171             s->pid, e->ppid, e->uid, s->comm, e->filename,
172             strerror(errno), s->current_score
173         );
```

```
174     }
175 }
176
177 // --- MAIN ANALYSIS ROUTINE ---
178
179 /**
180  * @brief Evaluates an event against the detection heuristics.
181  *
182  * This is the entry point for the detection logic. It updates the risk score based on:
183  * 1. Event Type weights (Write, Rename, etc.)
184  * 2. Context Multipliers (Directory sensitivity)
185  * 3. Semantic Analysis (File extensions)
186  * 4. Honeypot Access
187  *
188  * Triggers alarms and active blocking if the risk threshold is exceeded.
189  *
190  * @param s Process statistics structure.
191  * @param e Event data from kernel space.
192  */
193 void analyze_event(struct process_stats *s, const struct event *e) {
194
195     // 1. Whitelist Filter (Early exit to reduce noise)
196     if (is_whitelisted(s->comm)) return;
197
198     // 2. Audit Logging (Log RAW events that passed the whitelist)
199     const char *event_name = "UNKNOWN";
200     switch(e->type) {
201         case EVENT_WRITE: event_name = "WRITE"; break;
202         case EVENT_RENAME: event_name = "RENAME"; break;
203         case EVENT_OPEN: event_name = "OPEN"; break;
204         case EVENT_UNLINK: event_name = "UNLINK"; break;
205     }
206
207     log_audit_json(event_name, s->pid, e->ppid, e->uid, s->comm, e->filename);
208
209     // 3. Analysis and Scoring
210     apply_decay(s);
211
212     int score_gained = 0;
213     int is_alarm = 0;
214     char risk_reason[64] = {0};
215
216     switch (e->type) {
217         case EVENT_WRITE:
218             s->write_burst++;
219             s->total_write_count++;
220             score_gained = config.score_write;
221
222             if (is_honeypot_access(e->filename)) {
223                 score_gained += config.score_honeypot;
224                 snprintf(risk_reason, sizeof(risk_reason), "HONEYPOT WRITE");
225             }
226             break;
227
228         case EVENT_RENAME:
229             s->rename_burst++;
230             score_gained = config.score_rename;
231
232             if (is_honeypot_access(e->filename)) {
233                 score_gained += config.score_honeypot;
234                 snprintf(risk_reason, sizeof(risk_reason), "HONEYPOT RENAME");
```

```
235             }
236             break;
237
238         case EVENT_UNLINK:
239             score_gained = config.score_unlink;
240             break;
241
242         case EVENT_OPEN:
243             if (is_honeypot_access(e->filename)) {
244                 score_gained += config.score_honeypot;
245                 snprintf(risk_reason, sizeof(risk_reason), "HONEYPOT ACCESS");
246             }
247             break;
248     }
249
250     // Apply Directory Sensitivity Multipliers
251     double multiplier = 1.0;
252
253     // FIX: Check if string is not empty, because e->filename array address is never NULL.
254     if (e->filename[0] != '\0') {
255         if (strncmp(e->filename, "/home", 5) == 0) multiplier = 2.0; // High value user
                data
256         else if (strncmp(e->filename, "/etc", 4) == 0) multiplier = 5.0; // Critical config
                files
257         else if (strncmp(e->filename, "/var/www", 8) == 0) multiplier = 2.0; // Webroot
258         else if (strncmp(e->filename, "/tmp", 4) == 0) multiplier = 0.5; // Temp files (
                Noise reduction)
259     }
260     score_gained = (int)(score_gained * multiplier);
261
262     // Apply Extension Penalty
263     if ((e->type == EVENT_RENAME || e->type == EVENT_WRITE) && has_suspicious_extension(e
            ->filename)) {
264         score_gained += config.score_ext_penalty;
265         if (strlen(risk_reason) == 0) snprintf(risk_reason, sizeof(risk_reason), "
                SUSPICIOUS EXTENSION");
266     }
267
268     s->current_score += score_gained;
269
270     // Check Threshold
271     if (strlen(risk_reason) == 0) {
272         if (s->current_score >= config.risk_threshold) {
273             is_alarm = 1;
274             snprintf(risk_reason, sizeof(risk_reason), "RISK THRESHOLD EXCEEDED");
275         }
276     } else {
277         // Immediate alarm if there's a specific high-risk reason (like Honeypot)
278         if (s->current_score >= config.risk_threshold) is_alarm = 1;
279     }
280
281     // 4. Alarm and Response
282     if (is_alarm) {
283         // A. Detection Log
284         LOG_ALARM("RANSOMWARE SUSPECTED [%s]! PID:%d UID:%d | File:%s | Score:%d",
285                 risk_reason, s->pid, e->uid, e->filename, s->current_score);
286
287         log_alert_json(
288             "RANSOMWARE_DETECTED",
289             s->pid, e->ppid, e->uid, s->comm, e->filename,
290             risk_reason, s->current_score
```

```
291         );
292
293         // B. Active Blocking (IPS Mode)
294         if (config.active_blocking) {
295             kill_process(s, e, risk_reason);
296         }
297
298         // C. Reset Score after Action/Alarm
299         s->current_score = 0;
300         s->write_burst = 0;
301         s->rename_burst = 0;
302         s->last_decay_time = time(NULL);
303     }
304 }
```

Kod A.3. Risk puanlama ve aktif engelleme (IPS) mantığı

A.4 state_manager.c - Durum ve Bellek Yönetimi

Süreçlerin (PID) istatistiklerini hafızada tutan ve bellek temizliği yapan modüldür.

```c
/**
 * @file state_manager.c
 * @brief State Management Implementation for Process Tracking.
 * @version 0.9.0
 *
 * This module manages the lifecycle of process statistics using a hash table.
 * It provides O(1) access to per-process behavioral data (write counts,
 * risk scores, timestamps) which is essential for the heuristic analysis engine.
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "state_manager.h"
#include "logger.h"

/**
 * @brief Global hash table head for process statistics.
 *
 * This pointer serves as the entry point for the uthash-based hash table.
 * It must be initialized to NULL.
 */
struct process_stats *processes = NULL;

/**
 * @brief Retrieves or creates the statistics structure for a given process.
 *
 * Checks if the PID exists in the hash table. If found, returns the existing
 * structure. If not, allocates memory for a new structure, initializes it
 * with default values, and adds it to the hash table.
 *
 * @param pid Process ID.
 * @param comm Process command name (used for initialization if creating new).
 * @return Pointer to the process_stats structure, or NULL if memory allocation fails.
 */
struct process_stats *get_process_stats(int pid, const char *comm) {
    struct process_stats *s;

    // Search for the PID in the hash table
    HASH_FIND_INT(processes, &pid, s);

    if (s == NULL) {
        // Not found, create a new entry
        s = (struct process_stats*)malloc(sizeof(struct process_stats));
        if (!s) {
            fprintf(stderr, "[WARN] Memory allocation failed (malloc) for PID: %d\n", pid);
            return NULL;
        }

        // Initialize fields
        s->pid = pid;
        strncpy(s->comm, comm, sizeof(s->comm) - 1);
        s->comm[sizeof(s->comm) - 1] = '\0';

        s->total_write_count = 0;
```

```
56          s->write_burst = 0;
57          s->rename_burst = 0;
58          s->current_score = 0;
59
60          s->window_start_time = time(NULL);
61          s->last_decay_time = time(NULL);
62
63          // Add to the hash table
64          HASH_ADD_INT(processes, pid, s);
65      }
66      return s;
67  }
68
69  /**
70   * @brief Removes a process from the tracking table and frees memory.
71   *
72   * This function should be called when a process termination event (EVENT_EXIT)
73   * is received to prevent memory leaks.
74   *
75   * @param pid Process ID to remove.
76   */
77  void remove_process(int pid) {
78      struct process_stats *s;
79
80      HASH_FIND_INT(processes, &pid, s);
81
82      if (s) {
83          HASH_DEL(processes, s);
84          free(s);
85      }
86  }
87
88  /**
89   * @brief Cleans up all tracked processes and frees all allocated memory.
90   *
91   * Iterates through the entire hash table and frees every element.
92   * This ensures a clean exit without memory leaks.
93   */
94  void cleanup_all_processes() {
95      struct process_stats *current_process, *tmp;
96
97      // uthash macro: Safe iteration for deletion
98      HASH_ITER(hh, processes, current_process, tmp) {
99          HASH_DEL(processes, current_process);
100         free(current_process);
101     }
102 }
```

Kod A.4. Süreç bazlı istatistik takibi ve uthash entegrasyonu

A.5 state_manager.h - Veri Yapısı Tanımları

```c
/* state_manager.h - v0.9.0 (Standardized) */
#ifndef STATE_MANAGER_H
#define STATE_MANAGER_H

#include "uthash.h"
#include <time.h>

/**
 * @file state_manager.h
 * @brief Manages the state and behavioral statistics of active processes.
 */

/**
 * @struct process_stats
 * @brief Holds behavioral metrics and state for a specific process.
 * * This structure is used in a hash table (uthash) to provide O(1) access
 * to process statistics based on PID.
 */
struct process_stats {
    int pid;                /**< Key: Process ID */
    char comm[16];          /**< Process command name */

    unsigned long total_write_count; /**< Total write operations observed */
    unsigned long write_burst;  /**< Write operations in the current window */
    unsigned long rename_burst; /**< Rename operations in the current window */

    time_t window_start_time; /**< Timestamp when the current observation window started */
    time_t last_decay_time; /**< Timestamp of the last score decay operation */

    int current_score;      /**< Current cumulative risk score */

    UT_hash_handle hh;      /**< Uthash handle for hash table management */
};

/**
 * @brief Retrieves or creates the statistics structure for a given process.
 * * Checks if the PID exists in the hash table. If not, allocates and
 * initializes a new structure.
 * * @param pid Process ID.
 * @param comm Process command name (for initialization).
 * @return Pointer to the process_stats structure, or NULL on failure.
 */
struct process_stats *get_process_stats(int pid, const char *comm);

/**
 * @brief Removes a process from the tracking table and frees memory.
 * * @param pid Process ID to remove.
 */
void remove_process(int pid);

/**
 * @brief Cleans up all tracked processes and frees all allocated memory.
 * * This function should be called during the graceful shutdown sequence.
 */
void cleanup_all_processes();
```

```
56
57 #endif // STATE_MANAGER_H
```

Kod A.5. Process_stats yapısı tanımları

A.6 logger.c - Çok Kanallı Loglama Sistemi

Sistem olaylarını ve güvenlik alarmlarını farklı kanallara yazdıran modüldür.

```c
/**
 * @file logger.c
 * @brief Multi-channel Logging System Implementation.
 * @version 0.9.0
 *
 * Implements a centralized logging facility that routes messages to three distinct channels:
 * - Service Log: Operational status and debug messages (human-readable).
 * - Alert Log: High-severity security incidents (JSON).
 * - Audit Log: Raw stream of system events (JSON).
 *
 * Update Note (v0.9.0): Integrated file and line number into the log format
 * to resolve unused parameter warnings and improve traceability.
 */

#include "logger.h"
#include "config.h"
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>
#include <time.h>
#include <string.h>
#include <sys/time.h>
#include <unistd.h>

// --- STATIC FILE POINTERS (Channels) ---

/** @brief File pointer for general service logs (service.log). */
static FILE *f_service = NULL;

/** @brief File pointer for high-priority alerts (alerts.json). */
static FILE *f_alerts = NULL;

/** @brief File pointer for raw audit events (audit.json). */
static FILE *f_audit = NULL;

// --- HELPER FUNCTIONS ---

/**
 * @brief Generates a high-precision timestamp string.
 *
 * Format: "YYYY-MM-DD HH:MM:SS.mmm"
 *
 * @param buffer Output buffer.
 * @param size Size of the output buffer.
 */
static void get_timestamp(char *buffer, size_t size) {
    struct timeval tv;
    struct tm *tm_info;
    gettimeofday(&tv, NULL);
    tm_info = localtime(&tv.tv_sec);
    char fmt_buffer[32];
    strftime(fmt_buffer, sizeof(fmt_buffer), "%Y-%m-%d %H:%M:%S", tm_info);
    snprintf(buffer, size, "%s.%03ld", fmt_buffer, tv.tv_usec / 1000);
}
```

```
55
56  /**
57   * @brief Converts log level enum to string representation.
58   * @param level Log severity level.
59   * @return String literal (e.g., "INFO", "ALARM").
60   */
61  static const char* get_level_string(LogLevel level) {
62      switch (level) {
63          case LOG_LEVEL_INFO: return "INFO";
64          case LOG_LEVEL_WARN: return "WARN";
65          case LOG_LEVEL_ERROR: return "ERROR";
66          case LOG_LEVEL_ALARM: return "ALARM";
67          case LOG_LEVEL_DEBUG: return "DEBUG";
68          default:           return "UNKNOWN";
69      }
70  }
71
72  /**
73   * @brief Returns the ANSI color code associated with a log level.
74   * @param level Log severity level.
75   * @return ANSI escape code string.
76   */
77  static const char* get_level_color(LogLevel level) {
78      switch (level) {
79          case LOG_LEVEL_INFO: return ANSI_COLOR_GREEN;
80          case LOG_LEVEL_WARN: return ANSI_COLOR_YELLOW;
81          case LOG_LEVEL_ERROR: return ANSI_COLOR_RED;
82          case LOG_LEVEL_ALARM: return ANSI_COLOR_RED;
83          case LOG_LEVEL_DEBUG: return ANSI_COLOR_BLUE;
84          default:           return ANSI_COLOR_RESET;
85      }
86  }
87
88  /**
89   * @brief Escapes special characters for JSON string compatibility.
90   *
91   * Handles double quotes and backslashes to ensure valid JSON output.
92   *
93   * @param input Raw string.
94   * @param output Output buffer.
95   * @param out_len Size of the output buffer.
96   */
97  static void json_escape(const char *input, char *output, size_t out_len) {
98      size_t i = 0, j = 0;
99      while (input[i] != '\0' && j < out_len - 2) {
100         if (input[i] == '"' || input[i] == '\\') {
101             output[j++] = '\\';
102         }
103         output[j++] = input[i++];
104     }
105     output[j] = '\0';
106 }
107
108 // --- CORE FUNCTIONS ---
109
110 /**
111  * @brief Initializes the logging subsystem.
112  *
113  * Opens file streams for all configured log channels.
114  * Prints error messages to stderr if a file cannot be opened.
115  */
```

```
116 void init_logger() {
117     // 1. Service Log
118     if (strlen(config.service_log) > 0) {
119         f_service = fopen(config.service_log, "a");
120         if (!f_service) perror("Failed to open Service Log");
121     }
122
123     // 2. Alert Log
124     if (strlen(config.alert_log) > 0) {
125         f_alerts = fopen(config.alert_log, "a");
126         if (!f_alerts) perror("Failed to open Alert Log");
127     }
128
129     // 3. Audit Log
130     if (strlen(config.audit_log) > 0) {
131         f_audit = fopen(config.audit_log, "a");
132         if (!f_audit) perror("Failed to open Audit Log");
133     }
134 }
135
136 /**
137  * @brief Closes all active log file streams.
138  */
139 void finalize_logger() {
140     if (f_service) { fclose(f_service); f_service = NULL; }
141     if (f_alerts) { fclose(f_alerts); f_alerts = NULL; }
142     if (f_audit) { fclose(f_audit); f_audit = NULL; }
143 }
144
145 /**
146  * @brief Custom print callback for libbpf integration.
147  *
148  * Redirects libbpf internal logs to the service log file or stderr
149  * depending on the verbose mode setting.
150  */
151 int logger_libbpf_print(enum libbpf_print_level level, const char *format, va_list args) {
152     // If not in verbose mode, suppress info/debug logs from libbpf,
153     // but allow warnings/errors. Redirect to file if open.
154     if (!config.verbose_mode && level != LIBBPF_WARN) {
155         if (f_service) {
156             fprintf(f_service, "[LIBBPF] ");
157             vfprintf(f_service, format, args);
158             fflush(f_service);
159         }
160         return 0;
161     }
162     // In verbose mode, print to stderr as usual
163     return vfprintf(stderr, format, args);
164 }
165
166 /**
167  * @brief Logs a raw system event to the Audit Log (JSON).
168  *
169  * Captures the event details without risk analysis. Used for forensic auditing.
170  */
171 void log_audit_json(const char *event_type,
172                     int pid, int ppid, int uid,
173                     const char *comm,
174                     const char *filename)
175 {
176     if (!f_audit) return;
```

```
177
178     char timestamp[64];
179     get_timestamp(timestamp, sizeof(timestamp));
180
181     char safe_filename[512];
182     char safe_comm[64];
183
184     json_escape(filename ? filename : "", safe_filename, sizeof(safe_filename));
185     json_escape(comm ? comm : "", safe_comm, sizeof(safe_comm));
186
187     // Simplified JSON structure (Event-centric)
188     fprintf(f_audit,
189         "{\"timestamp\": \"%s\", \"type\": \"%s\", "
190         "\"pid\": %d, \"ppid\": %d, \"uid\": %d, \"comm\": \"%s\", "
191         "\"filename\": \"%s\"}\n",
192         timestamp, event_type, pid, ppid, uid, safe_comm, safe_filename);
193
194     fflush(f_audit);
195 }
196
197 /**
198  * @brief Logs a high-priority security alert to the Alert Log (JSON).
199  *
200  * Includes detailed risk context, reason, and score.
201  */
202 void log_alert_json(const char *event_type,
203                 int pid, int ppid, int uid,
204                 const char *comm,
205                 const char *filename,
206                 const char *risk_reason,
207                 int score)
208 {
209     if (!f_alerts) return;
210
211     char timestamp[64];
212     get_timestamp(timestamp, sizeof(timestamp));
213
214     char safe_filename[512];
215     char safe_comm[64];
216     char safe_reason[128];
217
218     json_escape(filename ? filename : "", safe_filename, sizeof(safe_filename));
219     json_escape(comm ? comm : "", safe_comm, sizeof(safe_comm));
220     json_escape(risk_reason ? risk_reason : "", safe_reason, sizeof(safe_reason));
221
222     fprintf(f_alerts,
223         "{\"timestamp\": \"%s\", \"level\": \"ALARM\", \"alert_type\": \"%s\", "
224         "\"pid\": %d, \"ppid\": %d, \"uid\": %d, \"comm\": \"%s\", "
225         "\"filename\": \"%s\", \"risk_reason\": \"%s\", \"score\": %d}\n",
226         timestamp, event_type, pid, ppid, uid, safe_comm, safe_filename, safe_reason, score
            );
227
228     fflush(f_alerts);
229 }
230
231 /**
232  * @brief Standard logging function for service messages.
233  *
234  * Writes formatted messages to both stdout (with colors) and the service log file.
235  */
236 void log_message(LogLevel level, const char *file, int line, const char *format, ...) {
```

```
237     char timestamp[64];
238     get_timestamp(timestamp, sizeof(timestamp));
239
240     int pid = getpid();
241     const char *level_str = get_level_string(level);
242     va_list args;
243
244     // 1. TERMINAL OUTPUT (Human-readable with colors)
245     if (config.verbose_mode || level != LOG_LEVEL_DEBUG) {
246         va_start(args, format);
247         fprintf(stdout, "%s", get_level_color(level));
248         // FIX: Added file and line info to resolve unused parameter warnings
249         fprintf(stdout, "[%s] [%-5s] [%d] [%s:%d] ", timestamp, level_str, pid, file, line)
                ;
250         vfprintf(stdout, format, args);
251         fprintf(stdout, "%s\n", ANSI_COLOR_RESET);
252         va_end(args);
253     }
254
255     // 2. SERVICE LOG FILE (Persistent record)
256     if (f_service) {
257         va_start(args, format);
258         // FIX: Added file and line info here as well
259         fprintf(f_service, "[%s] [%-5s] [%d] [%s:%d] ", timestamp, level_str, pid, file,
                line);
260         vfprintf(f_service, format, args);
261         fprintf(f_service, "\n");
262
263         // Always flush errors and alarms to ensure they are written immediately
264         if (level == LOG_LEVEL_ERROR || level == LOG_LEVEL_ALARM) {
265             fflush(f_service);
266         }
267         va_end(args);
268     }
269 }
```

Kod A.6. Hibrit loglama ve JSON formatında alert üretimi

OSMAN ÇEVİK

COMPUTER ENGINEER

SUMMARY

Computer Engineering student at Fırat University with a strong enthusiasm for Operating Systems and Computer Architecture. Experienced in low-level development and eBPF-based ransomware detection using C and CMake. Proven technical leader through roles as ACM Chairperson and Teknofest R&D Coordinator.

CONTACT

+90 541 643 1939
osmancevik@pm.me
Ankara, Türkiye
github.com/osmancevik

EDUCATION

2020 - 2026
FIRAT UNIVERSITY

- B.Sc. in Computer Engineering

TECHNICAL SKILL

- C/C++
- Linux & Bash
- Data Structures & OOP
- Git
- CMake
- eBPF

ADDITIONAL SKILL

- Team Leadership
- Problem Solving
- Collaborative Development
- Technical Writing
- English (B1)
- Turkish (Native)

WORK EXPERIENCE

CoSiNC 2025 JUNE - SEPTEMBER
R&D Intern

- Conducted R&D in communication security and AI-driven signal processing, delivering a device-free activity recognition system validated by a 49-page comprehensive technical evaluation report.

EXTRACURRICULAR ACTIVITIES

TEKNOFEST Fırat Student Chapter 2024 - 2025
R&D Coordinator

- Organized and managed the first-ever Datathon at Fırat University with 70+ participants, overseeing the technical infrastructure and competition workflow.
- Provided technical mentorship to high school students for TEKNOFEST projects, resulting in 3 teams qualifying as finalists in the national competition.

ACM Fırat Student Chapter 2022 - 2024
Chairperson

- Served as the Chairperson of the official student chapter of the ACM (Association for Computing Machinery), maintaining professional standards and aligning community activities with ACM's global computing excellence goals.
- Successfully led and coordinated 9 technical events and 6 software projects during the presidency, managing end-to-end execution, team allocation, and project delivery.

PROJECTS

eBPF-Based Ransomware Detection Tool:

- Developed a high-performance security tool using eBPF and C to monitor critical system calls (execve, openat, socket) for ransomware detection, utilizing BPF Maps and a CMake-managed architecture for efficient kernel-user data transfer with minimal overhead.

WiFi CSI-Based Human Activity Recognition

- WiFi CSI-Based Human Activity Recognition (HAR): Designed a device-free recognition system analyzing WiFi Channel State Information (CSI) with Random Forest and SVM models, featuring a Python signal processing pipeline and comprehensive 49-page technical documentation.